

GUARDING THE SIGNAL: SECURE MESSAGING WITH REVERSE FIREWALLS

Yevgeniy Dodis¹, Bernardo Magri², Noah Stephens-Davidowitz^{*3}, and Yiannis Tselekounis⁴

¹New York University

²The University of Manchester and Primev

³Cornell University

⁴Royal Holloway, University of London

June 13, 2025

Abstract

Secure messaging protocols allow users to communicate asynchronously over untrusted channels with strong guarantees of privacy, authenticity, forward secrecy, and post-compromise security. However, traditional security analyses of these protocols assume complete trust in the hardware and software of honest participants, overlooking a significant class of real-world threats known as *subversion attacks*. These attacks alter cryptographic algorithms to compromise security, by exfiltrating secrets or creating vulnerabilities that are often undetected.

The notion of reverse firewalls (EC’15), aims at protecting against subversion attacks by introducing a third party, called a “reverse firewall” (RF), which sits between a party and the outside world and modifies its outgoing and incoming messages in a way such that, even if the party’s machine has been corrupted (in a way that maintains functionality), security is still preserved. Importantly, the firewall shares no private information with the parties, and parties put no more trust in the firewall than they do in the communication channel. In this work, we address the existing gap in secure messaging and subversion attacks by presenting several key contributions:

1. We design the first subversion-resilient secure messaging protocol based on the model of RF. Our protocol is based on the Signal protocol—the current state-of-the-art in two-party secure messaging, though it lacks subversion resilience—and achieves subversion resilience with only constant overhead over Signal.
2. We develop a subversion-resilient version of the X3DH protocol in the RF model. X3DH is a core component that facilitates secure initial key agreement in Signal’s protocol.
3. We introduce and formalize the notion of Continuous Key Agreement with Tamper Detection, an essential concept for subversion-resilient secure messaging. Our notion enables parties to continuously agree on keys, even in the presence of active adversaries capable of partially tampering with the key exchange transcript. We present a construction of our notion and prove its subversion resilience in the model of RF.

^{*}This work was supported by NSF Grants Nos. CCF-2122230 and CCF-2312296, a Packard Foundation Fellowship, and a generous gift from Google.

Contents

1	Introduction	2	4	The CKA Protocol	31
1.1	Our Contributions	3	4.1	Protocol Operations	31
1.2	Technical Overview	4	4.2	CKA Security	36
1.3	Related Work	8	4.3	Subversion Resilience	39
2	Preliminaries	9	5	Secure Messaging Scheme	44
2.1	Game-Based Security and Notation	9	5.1	Security of the SM Scheme	47
2.2	Basic Building Blocks	10	5.2	Subversion Resilience of the SM Scheme	53
2.3	Diffie-Hellman Assumptions	12	6	Subversion-Resilient X3DH	61
2.4	Secure Messaging	13	6.1	Protocol Description	62
2.5	Malleable Commitments	18	6.2	Security	63
2.6	Forward-Secure AEAD	20	6.3	Subversion Security for X3DH	72
2.7	PRF-PRNGs	22	A	Comparison with [32]	79
3	Our Model	24	B	Security Model for Signal-Conforming AKE Protocols	80
3.1	Reverse Firewalls	24	C	Proof of Alice’s X3DH Firewall	83
3.2	Secure Messaging	26			
3.3	Continuous Key Agreement with Tamper Detection	28			

1 Introduction

End-to-end encrypted secure messaging is widely used by billions of people worldwide, forming the foundation of popular messaging applications like Signal, WhatsApp, Apple’s iMessage, and Meta’s Messenger, enabling clients to communicate securely and asynchronously over untrusted networks and server infrastructures. Here, the term “secure” encompasses security guarantees requiring that the current protocol messages remain secure against any adversary that controls all network traffic and can leak all parties’ secret states in the past – captured by the notion of *post-compromise security* (PCS) – and in the future – captured by the notion of *forward secrecy* (FS).

Subversion Resilience. The traditional analysis of cryptographic schemes and protocols often assumes complete security of the hardware and software utilized by honest participants—an assumption frequently challenged by practical experiences. The pioneering work by Young and Yung in kleptography [65, 66], building upon Simmons’ earlier exploration of subliminal channels [59, 60], marked the first formal consideration of adversaries stealthily modifying cryptographic algorithms to breach security. This research area, revitalized by Bellare, Paterson, and Rogaway [10], focuses on the threat of mass surveillance through algorithm substitution attacks. Their investigations reveal the vulnerability of randomized, stateless encryption schemes when an adversary covertly swaps the encryption algorithm for a version designed to silently extract secret keys or plaintexts. Real-world instances of subversion attacks,¹ such as the contentious standardization of Dual_EC_DRBG [46], the

¹In our work we use the term “subversion” to refer to any attack that tampers arbitrarily with the cryptographic implementation, although the term was popularized by [10] to refer to algorithm substitution attacks.

Chinese hack chip [55] and more recently the Crypto AG scandal [50], underscore these theoretical concerns by highlighting that powerful actors often find subverting cryptographic schemes to be a more successful attack strategy than trying to violate the security of such schemes in some other way. Thus, there is a pressing need for cryptographic systems to be designed with resilience against such subversions in mind.

It is not hard to see that, without additional assumptions, parties cannot possibly hope to maintain any reasonable notion of security if they are executing an arbitrarily compromised implementation of a cryptographic protocol. E.g., the compromised implementation might simply reveal all of the parties’ secrets in its first message. In order to circumvent this, Mironov and Stephens-Davidowitz [51] introduced a model in which an additional party called a (cryptographic) *reverse firewall* sits between a party (e.g., on the party’s router) and the outside world. The firewall has the ability to modify a party’s messages (both incoming and outgoing—without, of course, breaking the correctness of the protocol). Intuitively, a firewall is considered secure for a party if it guarantees that the party preserves its security properties—such as authenticity and confidentiality in secure messaging—even when the party’s implementation is replaced by a compromised version. In such cases, the firewall safeguards the party’s secrets, preventing unauthorized exposure or breaches despite the presence of the corrupted implementation.² Crucially, the firewall is *not* a trusted party and in particular does not share any secrets with parties.

Several works have demonstrated how to construct reverse firewalls for a variety of cryptographic primitives (e.g., [32, 25, 17, 39, 22, 23, 24]). Among these, the most relevant to our work is the contribution by Dodis, Mironov, and Stephens-Davidowitz [32], which demonstrated how to construct a protocol using reverse firewalls that achieves the notion of *message transmission*, a security concept known to be *weaker* than secure messaging. Achieving message transmission in the presence of subversion attacks was already quite surprising, since such protocols are known to be impossible [10] in a world without reverse firewalls. In fact, the security properties achieved by [32] are not nearly as strong as the ones required in secure messaging protocols [1]. For example, message transmission does not model state leakage attacks (so does not account for forward secrecy or post-compromise security), allows heavy interaction for every message (unlike non-interactive secure messaging protocols like Signal), and does not provide any correctness guarantees against active attackers, even with no corruptions and if honest messages are eventually delivered.³ Thus, we ask:

“Can we construct efficient secure-messaging protocols secure against subversion attacks?”

1.1 Our Contributions

In this work we present the first *subversion resilient secure messaging (SM) protocol*, in the model of reverse firewalls, achieving a constant overhead relative to Signal’s protocol, which is the current state of the art in two-party SM. In more detail, we present the following results:

Continuous Key Agreement (CKA) with tamper detection: We introduce the notion of *CKA with tamper detection*, which enables parties to continuously agree on keys, even against *active adversaries*⁴ that are allowed to *partially tamper* with the CKA transcript. As we argue in the next

²See Sec. 1.2 for a detailed discussion.

³For a detailed comparison between the current work and [32], cf. Sec. A.

⁴The original notion [1] considers passive adversaries.

section, our new notion is *essential* for achieving subversion resilient secure messaging.

Subversion resilient CKA protocol: We construct a subversion resilient CKA protocol with tamper detection, in the reverse firewall model, assuming the inverse computational Diffie Hellman assumption and an idealized hash function modeled as a random oracle. This protocol forms the main technical novelty of our work, and is only a constant factor slower than the extremely simple (non-subversion-resilient) CKA protocol of Signal.

Subversion resilient SM protocol: We build a subversion resilient secure messaging scheme in the reverse firewall model, based on *subversion resilient CKA with tamper detection*,⁵ forward-secure authenticated encryption with associated data (FS-AEAD) with unique ciphertexts, and a “hashing primitive” called PRF-PRNG [1].⁶ Our scheme achieves only a *constant overhead* over Signal’s protocol [48], and maintains the non-interactive (“send-and-go”) nature of Signal.

Subversion resilient X3DH-based protocol: Finally, we present a subversion resilient version of the X3DH protocol [49] in the reverse firewall model, which is a core protocol component that enables parties to securely agree on the initial key for Signal’s protocol. We prove security of our scheme in the model of “Signal-conforming authenticated key exchange” of [42]. Our result is in the random oracle model and assumes the hardness of the Gap Diffie Hellman (GDH) problem, inverse CDH problem, existentially unforgeable signatures (with uniqueness) and pseudorandom functions.⁷

Taken together, our results define a *fully subversion-resilient secure messaging protocol* in the style of the Signal protocol, including the initial key agreement, where we present a firewall-friendly version of X3DH, and the double ratchet part, all working together to maintain the non-interactive nature of the resulting messaging protocol, while preserving advanced security notions, such as FS and PCS. Our protocol prevents subverted implementations from leaking information, e.g., through backdoored PRGs, rejection sampling, bad randomness attacks, and devices that fail to generate enough entropy.

Compared to [32], our protocols realize a stronger functionality, i.e., continuous vs one-time, achieve immediate decryption, and stronger security (including FS and PCS), as well as deniability, without relying on signatures and pairings. In addition, our protocol achieves recovery against denial of service attacks (while [32] doesn’t) and PCS is achieved in one round, as opposed to [32] that requires four rounds.

1.2 Technical Overview

We now discuss the challenges in building subversion-resilient SM and we present the high-level technical ideas behind our definitions and constructions.

Subversion resilience and reverse firewalls [51, 32]. Our starting point is the notion of reverse firewalls (RF) [51]. According to [51], firewalls are required to satisfy three properties: (1) they should be *functionality maintaining*, in the sense that the composition between any protocol

⁵Modulo technicalities, our composition follows the template of [1], with the key novelty being our new CKA notion with tamper detection.

⁶This is a two-input function that acts as a pseudorandom function w.r.t. its first input, and as a pseudorandom generator w.r.t. its second input.

⁷We leave the construction of a subversion-resilient PQ-X3DH as future work.

and its firewall should realize the same functionality as the protocol itself, (2) they should *preserve security*, which requires the composition between any protocol and its firewall to achieve the notion of security that the original protocol provides, even when the machine of the party that sits behind the firewall gets corrupted, and (3) they should be *exfiltration resistant*, in the sense that a corrupted machine should not be able to leak information to other parties through the firewall. For instance, in a secure messaging protocol between parties A, B, exfiltration resistance for A against B requires that A leaks no information to B (other than information that is meant to be communicated to B via the functionality of the protocol).

Functionality maintaining adversaries. It is not hard to see that exfiltration resistance against parties within the same protocol session is impossible to achieve if we allow arbitrary corruptions. For instance, in a secure messaging protocol session between A and B, A can simply encrypt and send to B part of her private state (as part of her next message), and such leakage cannot be prevented by a firewall that does not share a secret key with A. In the current work, and following [51, 32, 25, 17, 39, 22, 23, 24], we focus on corrupted implementations of parties that *maintain protocol functionality*, i.e., the final protocol outputs should be consistent with the inputs provided by the parties. In the context of secure messaging, the functionality requires B to output/receive the message that is provided by A as input, therefore any functionality-maintaining corruption must actually send this message, otherwise users can possibly detect (e.g., using out of band channels) that the implementation is not functioning correctly.⁸

However, in our context, even this is not quite sufficient. Roughly speaking, in prior constructions of subversion-resilient protocols, if any party “noticed” a “malformed message” (e.g., if B received a message that did not authenticate properly), they would simply abort and therefore break functionality. So, by restricting attention to subversions that maintain functionality, such prior work effectively restricted attention to subversions that never send such malformed messages. Notice that this type of restriction is necessary, since (as was already observed in [51]) if a corrupted implementation of A could selectively choose whether to send a malformed message, she could use this as a channel to leak A’s information to B. However, aborting after receiving a single bad message is an extremely undesirable behavior because it allows for a very simple and devastating denial of service attack. In particular, if B simply aborts and refuses all future communications with A after receiving a single malformed message, then an adversary can prevent A and B from having any future communication by simply injecting a single malformed message into their conversation. Prior work simply did not model this type of attack and therefore was vulnerable to denial of service attacks.

An additional contribution of this work is a new more restrictive notion of functionality maintaining corruptions, which we call *explainability* and which we show is sufficient even in our much stronger context. Intuitively, the messages sent by a corrupted implementation of A are *explainable* if there exist coins r such that an honest implementation of A would produce the same messages if its random coins were fixed to r (and all other messages in the protocol were unchanged). Notice that this still allows for quite a wide class of attacks, such as bad randomness attacks, rejection sampling attacks, and various kinds of backdoors. In fact, nearly all prior constructions of subversion-resilient protocols have the property that explainability is implied by being functionality maintaining because any unexplainable message from A would cause B to abort.

⁸Note the model of reverse firewalls does not capture side-channel or timing attacks.

Signal and subversion resilience. Signal provides *post-compromise security* and *forward secrecy* via the notion of continuous key-agreement [1] (CKA), which enables parties to non-interactively refresh their key material. The protocol requires the entire CKA transcript to be authenticated, however, in order to achieve subversion resilience, firewalls must modify the parties’ outgoing messages, in order to prevent exfiltration of information. Assuming one of the parties is subverted, say A, it is not hard to see that, since A decides her next CKA key without interacting with B, if the firewall modifies an outgoing message T to some T^* , then B would compute the next CKA key using T^* , which is unknown to A. Therefore, it is unclear how the CKA state would evolve without violating correctness (assuming fresh keys are computed non-interactively as in Signal). Furthermore, the entire CKA transcript is authenticated via symmetric authenticated encryption by the higher layer protocol, and therefore it is unclear how to sanitize the parties’ outgoing messages without breaking the authenticity of messages. Given the above, we opt for a solution that computes keys *interactively* and *allows partial modification of the transcript*, without breaking correctness.

New notion: CKA with tamper detection. We address the limitations of CKA, by introducing CKA with tamper detection, that diverges from the original notion of [1] in the sense that is defined with regard to active adversaries, that are allowed to *partially tamper* with the CKA transcript (the original notion considers only passive adversaries). To overcome this considerable extra power of the attacker, we will require two additional properties from our CKA protocols: *tamper detection* and *state recovery*. Tamper detection requires that for unauthenticated messages sent at epoch i , if the adversary mauls them in a way that violates correctness (i.e., parties’ future CKA states and keys diverge), mauling will be detected by parties within a certain (protocol specific) number of epochs. State recovery then allows parties to engage in a subprotocol that enables them to compute consistent CKA states, assuming the adversary remains passive for a number of rounds (equal to 4 for our instantiation). Notice, this seemingly strong assumption *does not introduce any extra restrictions* on the attacker as compared to Signal [1], where it is already required for post-compromise security.⁹ As a result, active adversaries allowed in the secure messaging literature can only prevent the CKA state to evolve forward,¹⁰ and our modeling of CKA will manage to preserve this key property, despite giving more power to the attacker. Our definition is formally presented in [Sec. 3.3](#).

Subversion resilient CKA protocol. The key ingredient in our construction is the one-time subversion resilient key-agreement protocol of [32] against passive adversaries. One possible approach in constructing a one-time protocol is by rerandomizing the messages of the Diffie-Hellman key-agreement protocol, in which A first sends g^a , B then replies with g^b , and the shared key is g^{ab} . A firewall for Bob in this protocol can simply replace Bob’s incoming message A by A^α for uniformly random α and replace Bob’s outgoing message B by B^α so that both Alice and Bob eventually agree to the key $g^{\alpha ab}$ and Bob’s outgoing message is uniformly random. However, as pointed out in [32], this protocol is not subversion resilient (for Bob), since it allows a subverted implementation of Bob to influence the key, e.g., by repeatedly resampling b until specific bits of the final key $A^{\alpha b}$ are fixed to some desirable value. To prevent this attack, in the protocol of [32], A first commits to

⁹It is a well known limitation of Signal — accepted by practitioners — that parties can only recover from corruptions if the adversary remains passive for a number of rounds.

¹⁰Looking ahead, in this case parties can still securely exchange messages, using the symmetric ratcheting layer (of course, unless compromised, and PCS did not kick in).

g^a , B sends g^b , and finally A sends the opening g^a of the commitment. This protocol, referred below as SRKA, requires the commitment scheme to be *rerandomizable* and *malleable*, so that the firewall can rerandomize both the commitment, as well as the committed group element, in order to prevent exfiltration of private information.

Our protocol’s execution involves four interleaved (but independent) sessions of SRKA in order to use SRKA to continuously update a key. In particular, session 1 relates to the CKA key of the current epoch. With every CKA message sent by a party, say A, the party sends $T := (C, \text{op}, A)$, where op is an opening to the commitment sent by A in its previous CKA message (initiating session 2), A is a response to the last commitment sent by B in his last message (initiating session 3), and C is a commitment that initiates a new session of SRKA (session 4). T denotes the unauthenticated part of the CKA message, which the adversary is allowed to tamper with. By appropriately authenticating the DH keys derived from commitments and responses (this refers to the authenticated part of the CKA message denoted by T_a that the adversary is not allowed to tamper with at the CKA level), we guarantee that if the adversary tampers with the unauthenticated part of the transmitted CKA messages in a way that leads to diverging CKA keys, this will be detected by parties, which then engage in a four round “resend” protocol that enables them to resend the unauthenticated parts of messages sent within two epochs from the epoch that tamper detection took place. If the adversary remains passive during “resend” or if it mauls the communication in a way that doesn’t lead to diverging states, parties will then compute their next CKA key securely. The parties simply execute the “resend” protocol until they compute the next CKA key consistently.

At a high level, due to the rerandomization of commitments, openings and responses made by the firewall, and the fact that the authenticated part of the CKA message is deterministic, the CKA transcript is not leaking any information. Our protocol is formally presented in detail in [Sec. 4](#), and its CKA security and subversion resilience are formally proved in [Sec. 4.2](#) and [Sec. 4.3](#), respectively.

We note that, in the presence of an additional layer of encryption, such as TLS, it is not straightforward how the firewall would rerandomize the traffic, and we leave this as an interesting open problem.

Subversion resilient secure messaging. To construct secure messaging (and not just CKA), we follow the paradigm of [1] and combine CKA, forward-secure authenticated encryption with associated data (FS-AEAD) and PRF-PRNG, where the latter is a two-input function that acts as a pseudorandom function on its first input, and as a pseudorandom generator on its second input. In order for our construction to be subversion resilient, we require CKA to be subversion resilient and FS-AEAD to satisfy the uniqueness property (see below).

At a high level, the messages output by the “send” operation of our SM protocol consist of the CKA output messages and the FS-AEAD ciphertexts. Assuming subversion resilience of the CKA component, the CKA transcript leaks no information. However, as already discussed above, the adversary could possibly try to exfiltrate information by creating a channel between A and B, i.e., by encrypting arbitrary messages related to the parties’ private state. The uniqueness property of encryption (requiring that, for every message m , associated data ad and key sk , there is exactly one ciphertext c such that $\text{Dec}(\text{sk}, c, \text{ad}) = m$), together with the fact that subverted parties should be functionality maintaining, prevents the creation of this type of channel: if a subverted client encrypts a message m' other than the message m that the sender wishes to send, due to the uniqueness property, the output of decryption would be m' , thus breaking functionality.

Subversion resilient X3DH. The initiating key exchange mechanism for the Signal protocol, the original X3DH protocol [48], leverages a Diffie-Hellman authenticated key exchange (AKE) that incorporates both long-term and ephemeral keys to increase security against key compromise risks. X3DH distinctively employs a server in a “mailbox” role to store and forward messages between parties allowing for asynchronous communication, a feature that distinguishes it from traditional AKE protocols. This unique aspect of X3DH, underlining its departure from standard AKE practices by ensuring the initiator’s first message does not depend on the receiver’s identity, was formally defined as a “Signal-conforming AKE” in [42].

The main technical challenge behind constructing a subversion resilient version of X3DH is that, as we discussed above, subversion resilience requires some sort of interaction between A and B, while X3DH prohibits this interaction (i.e., parties interact only with the server). In order to circumvent this limitation, we introduce a “firewall-friendly” version of X3DH, that only requires one additional communication round between the parties and the server (parties still don’t interact with each other). At a high level, whenever A uploads her ephemeral key g^x to the server (as in the original X3DH protocol), A’s firewall rerandomizes it to $g^{x\alpha}$, and then the server responds with a value $r \neq 0$, which, as an incoming message to A, is rerandomized by the firewall to αr . Given the above, A’s final ephemeral key is set to $g^{x\alpha r}$, which then A signs and uploads to the server. Note that the firewall can check whether A uploads the correct sanitized key since it knows g^x , r that was chosen by the server, and α chosen by itself. We prove that the firewall’s rerandomization process, together with the fact that A signs her key with a signature scheme that produces *unique signatures* prevents exfiltration, even if r is maliciously chosen by the server. We prove security of our scheme in the model of “Signal-conforming AKE” by [42], capturing multiple parties and sessions between parties, as well as all corruptions against the ephemeral and long-term keys of parties. Our scheme is presented in [Sec. 6](#).

1.3 Related Work

The concept of reverse firewalls [51] has been used to construct secure message transmission and key exchange protocols [32, 25, 17], interactive proof systems [39], multi-party computation (MPC) protocols [22, 23], and recently was extended to the UC framework [24], while a related notion [2] shows how to build collusion-free protocols. The seminal work of Bellare, Patterson, and Rogaway [10] delves into the subversion of symmetric encryption schemes via algorithm-substitution attacks (ASAs), while further research [28, 9, 5, 40, 3] expanded on these findings, investigating ASAs in other contexts. The work of [13] studies the relation between ASAs and steganography. The field of cliptography [56, 57, 58, 26, 4, 12] studies subversion in the presence of watchdogs. Finally, several works study subversion against various cryptographic primitives, including pseudorandom generators [31, 29], hash functions [38], non-interactive zero-knowledge proofs [8], and public-key encryption [6]. The work [54] investigates the construction of subversion-resilient key exchange protocols without relying on mechanisms such as reverse firewalls. While the authors present candidate constructions, they do not provide formal security proofs within the proposed security framework. Ultimately, the authors acknowledge the limitations of their approach and recognize that the reverse firewall model addresses many of the core challenges in this setting. Additionally, [47] explores subversion-resilient AKE within the reverse firewall model of [32], presenting a protocol based on the CDH assumption. However, the proposed solution is restricted to the one-time AKE setting and does not extend to continuous key exchange or secure messaging scenarios as we do in our work.

Marlinspike and Perrin [48] introduced the Double Ratchet (DR), which is the core component of Signal [61], that is based on the Off-The-Record protocol [16]. A formal analysis of DR is presented in [27, 1, 21, 14, 45]. The works [11, 53, 7, 43, 37, 44] propose protocols with stronger security, [19, 41, 18, 30] present post-quantum secure versions of X3DH, while [64, 62, 63, 41, 18, 30] analyze its offline deniability guarantees. Finally, there is a big line of works focusing on authentication [33, 34, 35, 37, 20, 36], as well as anonymity [36, 15].

2 Preliminaries

In the current section we introduce the notation and the cryptographic tools that will be used throughout the paper.

2.1 Game-Based Security and Notation

All security definitions in this work are game-based, i.e., they consider games executed between a challenger and an adversary. The games have one of the following formats:

- **Unpredictability games:** First, the challenger executes the special init procedure, which sets up the game. Subsequently, the attacker is given access to a set of oracles that allow it to interact with the scheme in question. The goal of the adversary is to provoke a particular, game-specific *winning* condition. The *advantage* of an adversary $\mathcal{A}$ against a construction C in an unpredictability game Γ^C is

$$\text{Adv}_{\Gamma}^C(\mathcal{A}) := \Pr[\mathcal{A} \text{ wins } \Gamma^C].$$

- **Indistinguishability games:** In addition to setting up the game, the init procedure samples a secret bit $b \in \{0, 1\}$. The goal of the adversary is to determine the value of b . Once more, upon completion of init, the attacker interacts arbitrarily with all available oracles up to the point where it outputs a guess bit b' . The adversary *wins* the game if $b = b'$. The *advantage* of an adversary $\mathcal{A}$ against construction C in an indistinguishability game Γ is

$$\text{Adv}_{\Gamma}^C(\mathcal{A}) := 2 \cdot |\Pr[\mathcal{A} \text{ wins } \Gamma^C] - 1/2|.$$

With the above in mind, to describe a any security (or correctness) notion, one need only specify the init oracle and the oracles available to $\mathcal{A}$. The following special keywords are used to simplify the exposition of the security games:

- **req** is followed by a condition; if the condition is not satisfied, the oracle containing the keyword aborts and the game state is rolled back to the point prior to calling the oracle.
- **win** is used to declare that the attacker has won the game; it can be used for both types of games above.
- **end** disables all oracles and returns to the attacker all the values following the keyword.

Note that some games might not allow corruptions of parties if this would lead to a trivial attack (e.g., cf. Fig. 6) and the execution continues, while on others (e.g., cf. Fig. 1), the game ends and the adversary cannot win if corrupts parties in a way that security cannot recover.

Moreover, the descriptions of some games/schemes involve *dictionaries*. For ease of notation, these dictionaries are described with the *array-notation* described next, but it is important to note that they are to be implemented by a data structure whose size grows (linearly) with the number of elements *in* the dictionary (unlike arrays):

- **Initialization:** The statement $D[\cdot] \leftarrow \varepsilon$ initializes an *empty* dictionary D .
- **Adding elements:** The statement $D[i] \leftarrow v$ adds a value v to dictionary D with key i , overriding the value previously stored with key i if necessary.
- **Retrieval:** The expression $D[i]$ returns the value v with key i in the dictionary; if there are no values with key i , the value ε is returned.
- **Deletion:** The statement $D[i] \leftarrow \varepsilon$ *deletes* the value v corresponding to key i .

Finally, sometimes the random coins of certain probabilistic algorithms are made explicit. For example, $y \leftarrow A(x; r)$ means that A , on input x and with random tape r , produces output y . If r is not explicitly stated, it is assumed to be chosen uniformly at random; in this case, the notation $y \xleftarrow{\$} A(x)$ is used.

2.2 Basic Building Blocks

In this section we recall the basic building blocks used in this work.

2.2.1 Symmetric Encryption with Unique Ciphertext

First we recall the notion of unique ciphertexts for a symmetric encryption scheme as shown in [10].

Definition 1 (Unique Ciphertext [10]). *Let $\Pi = (\text{KeyGen}, \text{Enc}, \text{Dec})$ be a symmetric encryption scheme. For any possible state τ of Dec with respect to key k , any message $m \in \{0, 1\}^*$ and any associated data $a \in \{0, 1\}^*$, let $C_\Pi(k, m, a, \tau)$ be the set of all ciphertexts c such that $\text{Dec}(k, c, a, \tau)$ accepts with message m , meaning its output is (m, τ') for some τ' . We say that Π has unique ciphertexts if the set $C_\Pi(k, m, a, \tau)$ has size at most one for all k, m, a, τ . This means that, for any given key, message, associated data and state, there exists at most one ciphertext that the decryptor will decrypt to the message in question.*

Note that forward secure AEAD can be constructed generically from one-time AEAD and PRG. The PRG simply outputs a new key for one-time AEAD and a new PRG state (cf. [1]). The same idea can be applied on one time AEAD with uniqueness to make it forward secure. See for instance the one-time AEAD with uniqueness scheme of [10].

2.2.2 Unique signature scheme

Next, we recall the definition of unique signature schemes.

Definition 2 (Unique signature scheme). A unique signature scheme is a triple of algorithms $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$. KeyGen takes as input 1^λ where λ is the security parameter and outputs a public key spk and a private key ssk . Sign takes as input the secret key ssk and a plaintext m and outputs a signature σ . Ver takes as input the public key spk , a signature σ and a message m and outputs either true or false. A signature scheme Σ is correct if $\text{Ver}(\text{spk}, m, \text{Sign}(\text{ssk}, m)) = 1$. It is unique if for each plaintext m and public key spk , there is a unique signature σ such that $\text{Ver}(\text{spk}, m, \sigma) = 1$. A signature scheme is secure against adaptive chosen-message existential-forgery attacks (EU-CMA) if no adversary with access to the public key spk and a signature oracle can produce a valid signature not returned by the oracle.

2.2.3 Pseudorandom Functions

Next we recall the standard definition of pseudorandom functions (PRFs).

Definition 3 (Pseudorandom function). Let $\text{PRF} : \{0, 1\}^\lambda \times \mathcal{X} \rightarrow \mathcal{Y}$ be an efficient keyed function, where $\mathcal{X}$ and $\mathcal{Y}$ denote the domain and the range of PRF. Denote by $\mathcal{F}$ the set of all functions mapping $\mathcal{X}$ into $\mathcal{Y}$. Intuitively, a pseudorandom function (PRF) is a function that is computationally indistinguishable from a truly random function, for all efficient distinguishers. More formally, a function $\text{PRF} : \{0, 1\}^\lambda \times \mathcal{X} \rightarrow \mathcal{Y}$ is a (t, q, ε) -secure pseudorandom function, if for all adversaries $\mathcal{A}$ running in time at most t we have

$$\left| \Pr[\mathcal{A}^{\text{PRF}_k(\cdot)}(1^\lambda) = 1] - \Pr[\mathcal{A}^f(\cdot)(1^\lambda) = 1] \right| \leq \varepsilon,$$

where the randomness is taken over the choice of k and f , and $\mathcal{A}$ asks at most q queries to its oracle.

2.2.4 Authenticated Encryption

Syntax. An authenticated encryption with associated data (AEAD) scheme is a pair of algorithms $\text{AE} = (\text{Enc}, \text{Dec})$ with the following syntax:

- **Encryption:** Enc takes a key k , associated data a , and a message m and produces a ciphertext $e \leftarrow \text{Enc}(k, a, m)$.
- **Decryption:** Dec takes a key k , associated data a , and a ciphertext e and produces a message $m \leftarrow \text{Dec}(k, a, e)$.

All AEAD schemes in this paper are assumed to be deterministic, i.e., all randomness stems from the key k .

Correctness. An AEAD scheme is *correct* if for all keys k and all pairs (k, a) ,

$$\text{Dec}(k, a, \text{Enc}(k, a, m)) = m.$$

Security. In order to be used in the constructions in this paper, AEAD schemes need only satisfy one-time IND-CCA security. This is captured by the game depicted in Figure 1. It provides access to a one-time encryption oracle that returns either an encryption of the message specified by the attacker or a random ciphertext from the space $\mathcal{C}$ of all ciphertexts, depending on a randomly chosen

Oracles for AEAD Game

Parameters: AE scheme $\text{AE} = (\text{Enc}, \text{Dec})$ with key space $\mathcal{K}$.

<pre>// Initialization init $k \leftarrow \mathcal{K}$ $e^* \leftarrow \varepsilon$ $b \leftarrow \{0, 1\}$ // Decryption Oracle decrypt(a, e) if $e = e^*$ or $b = 1$ then return $\perp$ else return Dec(k, a, e)</pre>	<pre>// Encryption Oracle encrypt(a, m) if $b = 0$ then $e^* \leftarrow \text{Enc}(k, a, m)$ else $e^* \leftarrow \mathcal{C}$ return e^*</pre>
--	--

Figure 1: Oracles of the IND-CCA security game for an AEAD scheme (Enc, Dec) , where `encrypt` is a one-time oracle.

bit b . Moreover, the attacker may query a decryption oracle arbitrarily many times (except on the challenge ciphertext), which, however, always returns $\perp$ if $b = 1$. The advantage of an adversary $\mathcal{A}$ attacking an AEAD scheme AE is denoted by

Definition 4. An AEAD scheme AE is ε -one-time-CCA-secure if for all PPT attackers $\mathcal{A}$,

$$\text{Adv}_{\text{ot-cca}}^{\text{AE}}(\mathcal{A}) \leq \varepsilon.$$

2.3 Diffie-Hellman Assumptions

In this section we recall the Diffie-Hellman type assumptions used in this work.

2.3.1 Computational Diffie-Hellman (CDH) Assumption

We recall the standard computational Diffie-Hellman assumption below.

Definition 5. For a group G of order p , we say that CDH is hard in G if no probabilistic polynomial-time adversary taking input (g, g^a, g^b) where $g \xleftarrow{\$} G$ and $(a, b) \xleftarrow{\$} \mathbb{Z}_p^2$ has non-negligible probability of returning g^{ab} .

2.3.2 Inverse CDH Assumption

Next we recall the inverse CDH assumption as shown in [32].

Definition 6 (Inverse CDH [32]). For a group G of order p , we say that inverse CDH is hard in G if no probabilistic polynomial-time adversary taking input (g, g^a, g^b) where $g \xleftarrow{\$} G$ and $(a, b) \xleftarrow{\$} \mathbb{Z}_p^2$ has non-negligible probability of returning $(h, h^{1/a}, h^{1/b})$ for some element $h \in G \setminus \{1_G\}$.

2.3.3 Gap Diffie-Hellman (GDH) Assumption

Next we recall the Gap Diffie-Hellman assumption introduced in [52]

Definition 7 (GDH). *Let G be a cyclic group of prime order q with generator g . Assume that there exists an efficient DDH oracle which, on input a tuple (g, g^a, g^b, g^c) , decides whether $g^c = g^{ab}$.*

$$\mathcal{O}(g, g^a, g^b, g^c) = \begin{cases} 1, & \text{if } g^c = g^{ab}, \\ 0, & \text{otherwise,} \end{cases}$$

Then the Gap Diffie-Hellman assumption is that for every PPT adversary $\mathcal{A}$ the following probability is negligible in the security parameter.

$$\Pr_{a,b \leftarrow \mathbb{Z}_q} [\mathcal{A}^{\mathcal{O}}(g, g^a, g^b) = g^{ab}] \leq \text{negl}(\lambda).$$

Here the probability is taken over the uniformly random choices of $a, b \in \mathbb{Z}_q$ and the internal randomness of $\mathcal{A}$.

2.4 Secure Messaging

A *secure messaging (SM)* scheme allows two parties A and B to communicate securely bidirectionally and is expected to satisfy the following informal requirements. (1) *Correctness*: if no attacker interferes with the transmission or (interferes only with the unauthenticated part of the transmission), B outputs the messages sent by A in the correct order and vice versa. (2) *Immediate decryption and message-loss resilience (MLR)*: Messages must be decrypted as soon as they arrive and may not be buffered; if a message is lost, the parties do not stall. (3) *Authenticity*: While the parties' states are uncompromised (i.e., unknown to the attacker), the attacker cannot change the messages sent by them or inject new ones. (4) *Privacy*: While the parties' states are uncompromised, an attacker obtains no information about the messages sent. (5) *Forward secrecy (FS)*: All messages sent and received prior to a state compromise of either party (or both) remain hidden to an attacker. (6) *Post-compromise security (PCS, aka "healing")*: If the attacker remains passive (i.e., does not inject any corrupt messages), the parties recover from a state compromise (assuming each has access to fresh randomness). (7) *Randomness leakage/failures*: While the parties' states are uncompromised, all the security properties above except PCS hold even if the attacker completely controls the parties' local randomness. That is, good randomness is only required for PCS.

Epochs. SM schemes proceed in so-called *epochs*, which roughly correspond the "back-and-forth" between the two parties A and B. By convention, odd epoch numbers t are associated with A sending and B receiving, and the other way around for even epochs. Note, however, that SM schemes are completely asynchronous, and, hence, epochs overlap to a certain extent. Correspondingly, consider two epoch counters t_A and t_B for A and B, respectively, satisfying the following properties:

- The two counters are never more than one epoch apart, i.e., $|t_A - t_B| \leq 1$ at all times.
- When A receives a message for epoch t from B, for $t = t_A + 1$, it sets $t_A \leftarrow t$ (even). The next time A sends a message, t_A is incremented again (to an odd value).
- Similarly, when B receives a message for epoch t from A, for $t = t_B + 1$, it sets $t_B \leftarrow t$ (odd). The next time B sends a message, t_B is incremented again (to an even value).

Message indices. Within an epoch, messages are identified by a simple counter. To capture the property of immediate decryption and MLR, the receive algorithm of an SM scheme is required to output the correct epoch number and index *immediately* upon reception of a ciphertext, even when messages arrive out of order.

Corruptions and their consequences. Since SM schemes are required to be forward-secure and to recover from state compromise, any SM security game must allow the attacker to learn the state of either party at any given time. Moreover, to capture authenticity and privacy, the attacker should be given the power to inject malicious ciphertexts and to call a (say) left-or-right challenge oracle, respectively. These requirements, however, interfere as follows:

- When either party is in a compromised state, the attacker cannot invoke the challenge oracle since this would allow him to trivially distinguish.
- When either party is in a compromised state, the attacker can trivially forge ciphertexts and must therefore be barred from calling the inject oracle.
- When the receiver of messages in transmission is compromised, these messages lose all security, i.e., the attacker learns their content and can replace them by a valid forgery. Consequently, while any challenge ciphertext is in transmission, the recipient may not be corrupted. Similarly, an SM scheme must be able to deal with forgeries of compromised messages (once the parties have healed).

These issues require that the security definition keep track of ciphertexts *in transmission*, of *challenge* ciphertexts, and of *compromised* ciphertexts; this will involve some (slightly cumbersome) record keeping.

Natural SM schemes. For simplicity, SM schemes in this work are assumed to satisfy the natural requirements below.

Definition 8. An SM scheme $SM = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Rcv})$ is natural if the following criteria are satisfied:

1. Whenever Rcv outputs $m = \perp$, the state remains unchanged.
2. Any given ciphertext corresponds to an epoch t and an index i , i.e., the values (t, i) output by Rcv are an (efficiently computable) function of c .
3. Algorithm Rcv never accepts two messages corresponding to the same pair (t, i) .
4. A party always rejects ciphertexts corresponding to an epoch in which the party does not act as receiver.
5. If a party, say A , accepts a ciphertext corresponding to an epoch t , then $t_A \geq t - 1$.

Initialization and state. The initialization procedure chooses a random key and initializes the states s_A and s_B of A and B, respectively. Moreover, it defines several variables to keep track of the execution: (1) t_A and t_B are the epoch counters for A and B, respectively; (2) variables i_A and i_B count how many messages have been sent by each party in their respective current epochs; (3) t_L records the last time either party's state was leaked to the attacker and is used, together with t_A and t_B , to preclude trivial attacks; (4) the sets **trans**, **chall**, and **comp** will contain records and allow to track ciphertexts in transmission, challenge ciphertexts, and compromised ciphertexts, respectively; (5) the bit b is used to create the challenge; (6) the flags **rs** and d_{prv} are used by the security game to track whether the protocol advances to a new epoch, which is the case in which the receive operation outputs $d \neq \perp$ or $d_{\text{prv}} = \top$ ($d = \perp$ implies tamper detection against the unauthenticated part of the transcript).

Sampling if necessary. The send oracles **transmit-A** and **chall-A** allow the attacker to possibly control the random coins r of **Send**. If $r = \perp$, the function samples $r \xleftarrow{\$} \mathcal{R}$ (from some appropriate set $\mathcal{R}$), and returns (r, good) , where **good** indicates that fresh randomness is used. If, on the other hand, $r \neq \perp$, the function returns (r, bad) , indicating, via **bad**, that adversarially controlled randomness is used.

Epoch management. The epoch management function ***ep-mgmt** advances the epoch of the calling party if that party's epoch counter has a “receiving value” (even for A; odd for B) and resets the index counter. The **flag** argument is to indicate whether fresh or adversarial randomness is used. If a currently corrupted party starts a new epoch with bad randomness, the new epoch is considered corrupted. However, if it does not start a new epoch, bad randomness does not make the ciphertext corrupted. This captures that randomness should only be used for PCS (but for none of the other properties mentioned above).

Note that ***ep-mgmt** does not advance the counters to the next epoch if the receive operation outputs $d = \perp$, since the “resend” variable is set to **true**. This implies that in our protocol parties do not advance to the next epoch until the receive operation outputs $d = \top$. In this way we model tamper detection against the unauthenticated part of the transcript, in which case parties do not advance to the next epoch until they “resend” the unauthenticated part of the SM transcript and the receive operation outputs $d = \top$.

Record keeping. The game keeps track of ciphertexts in transmission, of challenge ciphertexts, and of compromised ciphertexts. Records have the format (P, t_P, i_P, m, c) , where P is the sender, t_P the epoch in which the message was sent, i_P the index within the epoch, m the message itself, and c the ciphertext.

Whenever ***record** is called, the new record is added to the set **trans**. If a party is not in a safe state, the record is also added to the set of compromised ciphertexts **comp**. If the function is called with **flag** = **chall**, the record is added to **chall**. The function ***delete** takes an epoch number and an index and removes the corresponding record from all three record keeping sets **trans**, **chall**, and **comp**.

Sometimes, it is convenient to refer to a particular record (or a set thereof) by only specifying parts of it. For example, the expression $B \notin \text{chall}$ is equivalent to there not being any record $(B, *, *, *, *)$ in the set **chall**. Similarly, $\text{trans}(B)$ is

Send oracles. Both send oracles, **transmit-A** and **chall-A**, begin with `*sam-if-nec`, which samples fresh randomness if necessary, followed by a call to `*ep-mgmt`. Observe that the flag argument is set to $\text{flag} \leftarrow \text{good}$ by `*sam-if-nec` if fresh randomness is used, and to $\text{flag} \leftarrow \text{bad}$ otherwise. Subsequently, **transmit-A** increments i_A , executes `Send`, and creates a record using $\text{flag} = \text{norm}$, indicating that this is not a challenge ciphertext. Observe that if **A** is not currently in a safe state, the record is added to **comp**. **chall-A** works similarly to **transmit-A**, except that one of the two inputs is selected according to b , and the record is saved with $\text{flag} = \text{chall}$, which will cause it to be added to the challenges **chall**. Note that **chall-A** can only be called when **A** is not in a compromised state, which is captured by the statement `req safe-chA`. The oracles for **B** are defined analogously.

Receive oracles. Two oracles are available by which the attacker can get **A** to receive a ciphertext: **deliver-A** is intended for honest delivery, i.e., to deliver ciphertexts for which the authenticated part has been created by **B**, whereas **inject-A** is used to inject forgeries. These rules are enforced by checking (via `req`) the set **trans**. In **deliver-A** the ciphertext is first passed through `Rcv`, which must correctly identify the values t , i , and m , recorded when $c.a$ was created; if it fails to do so, the correctness property is violated and the attacker immediately wins the game. In case c was a challenge, the decrypted message is replaced by $\perp$ in order to avoid trivial attacks. Note that honest delivery only requires the authenticated part of the ciphertext, denoted by $c.a$, to be in the list of honestly transmitted messages, i.e., correctness holds even if the adversary injects against the unauthenticated part of c .

For **inject-A**, again, the ciphertext is first passed through `Rcv`. Unless the ciphertext corresponds¹¹ to $(t, i) \in \text{comp}$, algorithm `Rcv` must reject it; otherwise, authenticity is violated and the attacker wins the game. The final instructions are as in **deliver-A**. Oracle **inject-A** may only be called if neither party is currently recovering from state compromise, which is taken care of by flag `safe-inj`. Note that valid injections require the authenticated part of c , $c.a$, to not belong in the set of transmitted messages. The oracles for **B** are defined analogously.

By deleting records at the end of **deliver-A** and **inject-A**, the game enforces that no replay attacks take place. For example, if the authenticated part of a ciphertext c that at some point is in **trans** is accepted twice, the second time counts as a forgery. Similarly, if two forgeries for a compromised pair (t, i) are accepted, the attacker wins as well. Note, however, that natural schemes do not allow replay attacks (cf. [Item 3](#) in [Definition 8](#)).

Corruption oracles. The corruption oracle for **A**, **corr-A**, can be called whenever no challenges are in transit from **B** to **A**, i.e., when $\text{B} \notin \text{chall}$. If corruption is allowed, all ciphertexts in transit sent by **B** become compromised. Before returning **A**'s state, the oracle updates the time of the most recent corruption. The corruption oracle **corr-B** for **B** is defined similarly. the set of all records of this type in **trans**.

2.4.1 Simplified Properties

In the current section we revisit the “simplified properties” of [1], that enable proving SM security by individually proving correctness, privacy and authenticity, and the combining all properties in [Theorem 1](#).

¹¹cf. [Item 2](#) in [Definition 8](#).

Proving SM security is facilitated by considering the three simplified properties *correctness*, *authenticity*, and *privacy*. These properties together imply full SM security, as stated at the end of this section. All three of them are obtained by modifying the original SM game (cf. Figure 5).

Correctness. For the correctness game, there are no oracles **chall-A** and **chall-B**, and only *reduced* inject oracles are available, which differ from the normal ones as follows:

- if ciphertext c does not correspond to $(t', i') \in \text{comp}$, **inject-A** and **inject-B** immediately return $(t', i', \perp)$;¹²
- the “if”-clause is removed completely.

Essentially, the reduced inject oracle only processes ciphertexts corresponding to $(t, i) \in \text{comp}$, which is unavoidable even after healing since the attacker knows the keys with which to decrypt and authenticate these messages. Observe that the only way to win the game is via the **win** instruction in **deliver-A** or **deliver-B**. The advantage of $\mathcal{A}$ in the correctness game is denoted by $\text{Adv}_{\text{corr}}^{\text{SM}}(\mathcal{A})$.

Definition 9. A secure-messaging scheme SM is $(q, q_{\text{ep}}, \varepsilon)$ -correct if for all (q, q_{ep}) -attackers $\mathcal{A}$,

$$\text{Adv}_{\text{corr}}^{\text{SM}}(\mathcal{A}) \leq \varepsilon.$$

Authenticity. In the authenticity variant of the SM game, there are no oracles **chall-A** and **chall-B**, and the **win**-instruction inside **deliver-A** and **deliver-B** is removed. Furthermore, the attacker must call **init** with two arguments t_{L}^* and t^* with $t^* \geq t_{\text{L}}^* + \delta_{\text{SM}}$, where t^* is the epoch it is trying to attack and t_{L}^* is the last corruption event before the attempt. To formally define this, consider the case where t^* is *even*, i.e., **B** is the sender and **A** the receiver in the epoch under attack; the case where t^* is odd works analogously. Then, the authenticity game additionally differs from the original as follows:

- if at any point $t_{\text{L}} \in \{t_{\text{L}}^* + 1, \dots, t^* - 1\}$, the attacker loses the game immediately;
- if **A** (the receiver in epoch t^*) is corrupted any time once $t_{\text{A}} > t_{\text{L}}^*$, the attacker loses the game immediately;
- the inject oracles are reduced (see above) *except* for ciphertexts corresponding to epoch t^* .

The advantage of $\mathcal{A}$ in the authenticity game is denoted by $\text{Adv}_{\text{auth}, \delta_{\text{SM}}}^{\text{SM}}(\mathcal{A})$.

Definition 10. A secure-messaging scheme SM is $(q, q_{\text{ep}}, \delta_{\text{SM}}, \varepsilon)$ -authentic if for all (q, q_{ep}) -attackers $\mathcal{A}$,

$$\text{Adv}_{\text{auth}, \delta_{\text{SM}}}^{\text{SM}}(\mathcal{A}) \leq \varepsilon.$$

¹²Recall that, by virtue of Property 2 in Definition 8, (t', i') output by **Rcv** are assumed to be efficiently deducible from a ciphertext c .

Privacy. The privacy variant only provides *reduced* inject oracles (see above), and the “if”-clause with the **win** instruction in **deliver-A** is removed completely. Furthermore, the attacker must call **init** with two arguments t_L^* and t^* with $t^* \geq t_L^* + \delta_{SM}$, where t^* is the epoch it is trying to attack and t_L^* is the last corruption event before the attempt. Formally, this is enforced as follows:

- if at any point $t_L \in \{t_L^* + 1, \dots, t^* - 1\}$, the attacker loses the game immediately (i.e., the outcome of the game is a uniformly random bit);
- the challenge oracle for $P \in \{A, B\}$ has an additional $t_P = t^*$ constraint in the **req**-statement.

Observe that the condition $t_P = t^*$ effectively means that only one challenge oracle is available (**chall-A** if t^* is even; **chall-B** if t^* is odd). The advantage of $\mathcal{A}$ in the privacy game denoted by $\text{Adv}_{\text{priv}, (\delta_{SM}, \Delta_{SM})}^{\text{SM}}(\mathcal{A})$.

Definition 11. A secure-messaging scheme SM is $(q, q_{ep}, \delta_{SM}, \Delta_{SM}, \varepsilon)$ -private if for all (q, q_{ep}) -attackers $\mathcal{A}$,

$$\text{Adv}_{\text{priv}, (\delta_{SM}, \Delta_{SM})}^{\text{SM}}(\mathcal{A}) \leq \varepsilon.$$

Combining the properties. The following theorem relates the simplified properties to the full SM security definition.

Theorem 1 ([1]). Assume an SM scheme SM is

- $(q, q_{ep}, \varepsilon_{\text{corr}})$ -correct,
- $(q, q_{ep}, \delta_{SM}, \varepsilon_{\text{auth}})$ -authentic, and
- $(q, q_{ep}, \delta_{SM}, \Delta_{SM}, \varepsilon_{\text{priv}})$ -private.

Then, it is also $(q, q_{ep}, \delta_{SM}, \Delta_{SM}, \varepsilon)$ -secure, where

$$\varepsilon \leq \varepsilon_{\text{corr}} + ne^2 \cdot (\varepsilon_{\text{auth}} + \varepsilon_{\text{priv}}).$$

2.5 Malleable Commitments

In the current section we briefly revisit the notion of malleable, statistically hiding, computationally binding, commitments of [32], together with an instantiation which is a variant of the Pedersen commitment [32]. Recall that for our application we require that the committed value is a group element and that the commitment can be mauled in such a way that a commitment to a group element B can be converted into a commitment to the group element B^a . Therefore, following [32], we define malleable commitments in this specific setting.

Definition 12 (Commitment scheme syntax). A commitment scheme C is a collection of four efficient algorithms $(\text{Gen}, \text{Com}, \text{Open}, \text{Ver})$, where,

- **Gen** receives 1^λ and is parameterized by a group whose order is proportional to λ , where λ is the security parameter, and outputs public parameters ρ .
- **Com** receives the public parameters ρ , a plaintext m , and randomness r , and outputs a commitment C .

- **Open** receives the public parameters ρ , a commitment C , and randomness r and outputs an opening x .
- **Ver** receives a commitment C and opening x and outputs either a plaintext m or the special symbol $\perp$.

For simplicity, we often omit references to the public parameters and the randomness.

Definition 13 (Correctness, perfect hiding, computational binding). *Let $\mathbf{C} = (\text{Gen}, \text{Com}, \text{Open}, \text{Ver})$ be a commitment scheme.*

- **Correctness:** $\mathbf{C}$ satisfies correctness if for any message m ,

$$\Pr \left[\text{Ver}(C, x) = m \mid r \leftarrow \{0, 1\}^\lambda; \rho \leftarrow \text{Gen}(1^\lambda); C = \text{Com}(\rho, m; r); x = \text{Open}(\rho, C, r) \right] = 1$$

- **Perfect hiding:** $\mathbf{C}$ is perfectly hiding if for any two messages m_1, m_2 , the distribution of $\text{Com}(m_1)$ is identical to that of $\text{Com}(m_2)$.
- **Computational binding:** $\mathbf{C}$ is computationally binding if no efficient adversary can produce a commitment C and two openings $x \neq x'$, such that $\text{Ver}(C, x) = m_1$, $\text{Ver}(C, x') = m_2$, where $m_1 \neq m_2$, $m_1, m_2 \neq \perp$.

Additional properties introduced by [32], follow.

Definition 14 (Tightness, rerandomizability, malleability). *Let $\mathbf{C} = (\text{Gen}, \text{Com}, \text{Open}, \text{Ver})$ be a commitment scheme with messages over a multiplicative group of order p .*

- **Tightness:** $\mathbf{C}$ is tight if for each commitment C and message m , there is a unique opening x such that $\text{Ver}(C, x) = m$.
- **Rerandomizability:** $\mathbf{C}$ is rerandomizable if there exists a pair of efficient algorithms $(\text{Rerand}, \text{OpenRerand})$ such that
 1. For any commitment C and uniformly random r , $\text{Rerand}(C, r)$ is uniformly random over the commitment space.
 2. For any commitment C and opening x , $\text{Ver}(C, x) = \text{Ver}(C', x')$, where $C' = \text{Rerand}(C, r)$ and $x' = \text{OpenRerand}(x, r)$.
- **Malleability:** $\mathbf{C}$ is malleable if there exists a pair of efficient algorithms $(\text{Maul}, \text{MaulOp})$ such that for any commitment C and opening x , $\text{Ver}(C, x)^a = \text{Ver}(C', x')$, where $C' = \text{Maul}(C, a)$ and $x' = \text{MaulOp}(x, a)$ for $a \in \mathbb{Z}_p$.

In the main body, we make calls to **OpenRerand** and **MaulOp** implicit.

The commitment scheme of [32]. We now revisit the instantiation of [32].

- $\text{Gen}(1^\lambda)$ returns two uniformly random non-identity group elements g, h , over a group G .
- Com on input $m \in G$ and randomness $(r, s) \in \mathbb{Z}_p^2$, returns $C := (g^r h^s, h^s m)$.
- Open simply returns the randomness (r, s) .

- Ver on input $C = (c_1, c_2)$ and (r, s) , it first checks that $c_1 = g^r h^s$. If so, it returns $h^{-s} c_2$.
- Rerand on input $C = (c_1, c_2)$ and (r', s') , returns $C' := (g^{r'} h^{s'} c_1, h^{s'} c_2)$.
- OpenRerand on input an opening (r, s) and randomness (r', s') , returns $(r + r', s + s')$.
- Maul on input $C = (c_1, c_2)$ and $a \in \mathbb{Z}_p$, returns $C' := (c_1^a, c_2^a)$.
- MaulOp on input (r, s) and a and returns (ar, as) .

Theorem 2 ([32]). *The above commitment scheme is correct, statistically hiding, tight, rerandomizable, and malleable. If the discrete log is hard over G , it is also computationally binding.*

2.6 Forward-Secure AEAD

Forward-secure authenticated encryption with associated data is a stateful primitive between a sender A and a receiver B and can be considered a single-epoch variant of an SM scheme, a fact that is also evident from its security definition, which resembles that of SM schemes.

Syntax. A *Forward-secure authenticated encryption with associated data (FS-AEAD)* is a tuple of algorithms $\text{FS-AEAD} = (\text{FS-Init-S}, \text{FS-Init-R}, \text{FS-Send}, \text{FS-Rcv})$, where

- FS-Init-S (and similarly FS-Init-R) takes a key k and outputs a state $v_A \leftarrow \text{FS-Init-S}(k)$,
- FS-Send takes a state v , associated data a , and a message m and produces a new state and a ciphertext $(v', e) \leftarrow \text{FS-Send}(v, a, m)$, and
- FS-Rcv takes a state v , associated data a , and a ciphertext e and produces a new state, an index, and a message $(v', i, m) \leftarrow \text{FS-Rcv}(v, a, e)$.

Observe that all algorithms of an FS-AEAD scheme are deterministic.

Memory management. In addition to the basic syntax above, it is useful to define the following two functions **FS-Stop** (called by the sender) and **FS-Max** (called by the receiver) for memory management:

- **FS-Stop**, given an FS-AEAD state v , outputs how many messages have been received and then “erases” the FS-AEAD session corresponding to v from memory; and
- **FS-Max**, given a state v and an integer ℓ , remembers ℓ internally such that the session corresponding to v is erased from memory as soon as ℓ messages have been received.

These features will be useful in the full protocol (cf. Section 5) to be able to terminate individual FS-AEAD sessions when they are no longer needed. Providing a formal requirement for these additional functions is omitted. Moreover, since an attacker can infer the value of the message counter from the behavior of the protocol anyway, there is no dedicated oracle included in the security game below.

Security Game for FS-AEAD

Parameters: _____

// Initialization

Init

```

 $k \xleftarrow{\$} \mathcal{K}$ 
 $v_A \leftarrow \text{FS-Init-S}(k)$ 
 $v_B \leftarrow \text{FS-Init-R}(k)$ 
 $i_A \leftarrow 0$ 
 $\text{corr}_A \leftarrow \text{false}$ 
 $\text{trans}, \text{chall}, \text{comp} \leftarrow \emptyset$ 
 $b \xleftarrow{\$} \{0, 1\}$ 

```

// Corrupt party A

Corr-A

```

 $\text{corr}_A \leftarrow \text{true}$ 
return  $v_A$ 

```

// Corrupt party B

Corr-B

```

req  $\text{chall} = \emptyset$ 
end  $(v_A, v_B)$ 

```

// Deliver B

deliver-B(a, e)

```

req  $(i, a, m, e) \in \text{trans}$  for some  $i, m$ 
 $(v_B, i', m') \leftarrow \text{FS-Rcv}(v_B, a, e)$ 
if  $(i', m') \neq (i, m)$  then
  win
if  $(i, m) \in \text{chall}$  then
   $m' \leftarrow \perp$ 
*delete( $i$ )
return  $(i', m')$ 

```

// Record

***record**(flag, a, m, e)

```

 $\text{rec} \leftarrow (i_A, a, m, e)$ 
 $\text{trans} \leftarrow \text{rec}$ 
if  $\text{flag} = \text{bad}$  or  $\text{corr}_A$  then
   $\text{comp} \leftarrow \text{rec}$ 
if  $\text{flag} = \text{chall}$  then
   $\text{chall} \leftarrow \text{rec}$ 

```

// Transmit for party A

transmit-A(a, m)

```

 $i_A \leftarrow i_A + 1$ 
 $(v_A, e) \leftarrow \text{FS-Send}(v_A, a, m)$ 
*record( $\text{good}, a, m, e$ )
return  $e$ 

```

// Challenge A

chall-A(a, m_0, m_1)

```

req  $\neg \text{corr}_A$  and  $|m_0| = |m_1|$ 
 $i_A \leftarrow i_A + 1$ 
 $(v_A, e) \leftarrow \text{FS-Send}(v_A, a, m_b)$ 
*record( $\text{chall}, a, m_b, e$ )
return  $e$ 

```

// Inject B

inject-B(a, e)

```

req  $(a, e) \notin \text{trans}$ 
 $(v_B, i', m') \leftarrow \text{FS-Rcv}(v_B, a, e)$ 
if  $m' \neq \perp$  and  $i' \notin \text{comp}$  then
  win
*delete( $i'$ )
return  $(i', m')$ 

```

// Delete

***delete**(i)

```

 $\text{rec} \leftarrow (i, a, m, e)$  for  $m, a, e$ 
  such that  $(i, a, m, e) \in \text{trans}$ 
 $\text{trans}, \text{chall}, \text{comp} \leftarrow \text{rec}$ 

```

Figure 2: Oracles corresponding to party A of the FS-AEAD security game for a scheme $\text{FS-AEAD} = (\text{FS-Init-S}, \text{FS-Init-R}, \text{FS-Send}, \text{FS-Rcv})$. After corrupting B, the parties' states are revealed to the adversary, who can then fully simulate oracle queries on its own.

Correctness and security. Both correctness and security are built into the security game depicted in Figure 2. The game is the single-epoch analogue of the SM security game (cf. Figure 5) and therefore has similarly defined oracles and similar record keeping. A crucial difference is that as soon as the receiver **B** is compromised, the game ends with a full state reveal as no more security can be provided. If only the sender **A** is compromised, the game continues and uncompromised messages must remain secure.

The advantage of an attacker $\mathcal{A}$ against an FS-AEAD scheme FS-AEAD is denoted by the expression $\text{Adv}_{\text{fs-aead}}^{\text{FS-AEAD}}(\mathcal{A})$. The attacker is parameterized by the total number of queries q it makes.

Definition 15. An FS-AEAD scheme FS-AEAD is q -secure if for all q -attackers $\mathcal{A}$,

$$\text{Adv}_{\text{fs-aead}}^{\text{FS-AEAD}}(\mathcal{A}) \leq \epsilon .$$

2.7 PRF-PRNGs

A *PRF-PRNG* resembles both a pseudo-random function (PRF) and a pseudorandom number generator with input (PRNG)—hence the name. On the one hand, as a PRNG would, a PRF-PRNG (1) repeatedly accepts inputs I and uses them to refresh its state σ and (2) occasionally uses the state, provided it has sufficient entropy, to derive a pseudo-random pair of output R and new state; for the purposes of secure messaging, it suffices to combine properties (1) and (2) into a single procedure. On the other hand, a PRF-PRNG can be used as a PRF in the sense that if the state has high entropy, the answers to various inputs I *on the same state* are indistinguishable from random and independent values.

A *PRF-PRNG* is a pair of algorithms $P = (P\text{-Init}, P\text{-Up})$, where

- $P\text{-Init}$ takes a key k and produces a state $\sigma \leftarrow P\text{-Init}(k)$, and
- $P\text{-Up}$ takes a state σ and an input I and produces a new state and an output $(\sigma', R) \leftarrow P\text{-Up}(\sigma, I)$.

Security Game for PRF-PRNG

Parameters: _____

```

// Initialization
Init
   $k \xleftarrow{\$} \mathcal{K}$ 
   $\sigma \leftarrow \text{P-Init}(k)$ 
   $\text{corr} \leftarrow \text{false}$ 
   $\text{prng}, \text{prf} \leftarrow \text{false}$ 
  sample random function  $\mathcal{F}$ 
   $b \xleftarrow{\$} \{0, 1\}$ 

// Corrupt party
corr
  req  $\neg \text{prf}$ 
  return  $\sigma$ 

// Challenge PRNG
chall-prng( $I$ )
   $I \leftarrow \text{*sam-if-nec}(I)$ 
  req  $\neg \text{corr}$  and  $\neg \text{prf}$ 
   $\text{prng} \leftarrow \text{true}$ 
   $(\sigma, R) \leftarrow \text{P-Up}(\sigma, I)$ 
  if  $b = 1$  then
     $R \xleftarrow{\$} \mathcal{R}$ 
  return  $R$ 

// Process
Process( $I$ )
   $I \leftarrow \text{*sam-if-nec}(I)$ 
   $(\sigma, R) \leftarrow \text{P-Up}(\sigma, I)$ 
  return  $R$ 

// Challenge PRF
chall-prf( $I$ )
  req  $\neg \text{corr}$  and  $\neg \text{prng}$ 
   $\text{prf} \leftarrow \text{true}$ 
   $(\sigma', R) \leftarrow \text{P-Up}(\sigma, I)$ 
  if  $b = 1$  then
     $R \leftarrow \mathcal{F}(I)$ 
  return  $(\sigma', R)$ 

*sam-if-nec( $I$ )
  if  $I = \perp$  then
     $I \xleftarrow{\$} \mathcal{I}$ 
     $\text{corr} \leftarrow \text{false}$ 
  return  $I$ 

```

Figure 3: Oracles of the PRF-PRNG security game for a scheme $\text{P} = (\text{P-Init}, \text{P-Up})$.

Security. The simple intuitive security requirement for a double-seed PRG is that $\text{P-Init}(\sigma, I)$ produce a pseudorandom value if the state σ is uncorrupted (i.e., has high entropy) or the input I is chosen uniformly from some set $\mathcal{S}$. Moreover, if the state is uncorrupted, it should have the PRF property described above. This is captured by the security definition described by Figure 3:

- **Initialization:** Procedure **Init** chooses a random bit b , initializes the PRF-PRNG with a random key, and sets two flags **prng** and **prf** to **false**: the PRNG and PRF modes are mutually exclusive and only one type of challenge may be called; the flags keep track of which. During initialization, a random function $\mathcal{F}$ with appropriate input and output lengths is chosen to create PRF challenges (see below).
- **PRNG mode:** The oracle **Process** can be called in two ways: either I is an input specified by the attacker and is simply absorbed into the state, or $I = \perp$, in which case the game chooses it randomly (inside **sam-if-nec**) and absorbs it into the state, which at this point becomes uncorrupted. Oracle **chall-prng** is works in the same fashion but creates a challenge.
- **PRF mode:** Once the state is uncompromised the attacker can decide to obtain PRF

challenges by calling **chall-prf**, which simply evaluates the (adversarially chosen) input on the current state without updating it and creates a challenge (using random function $\mathcal{F}$ when $b = 1$).

- **Corruption:** At any time, except after asking for PRF challenges, the attacker may obtain the state by calling **corr**.

The advantage of $\mathcal{A}$ in the PRF-PRNG game is denoted by $\text{Adv}_{\text{PP}}^{\text{P}}(\mathcal{A})$.

Definition 16. An PRF-PRNG P is ε -secure if for all PPT attackers $\mathcal{A}$,

$$\text{Adv}_{\text{PP}}^{\text{P}}(\mathcal{A}) \leq \varepsilon .$$

3 Our Model

In this section, we present our model for Reverse Firewalls, which generalizes the framework introduced in [51] to the setting of continuous protocols. We also introduce our model for Secure Messaging schemes, extending the formulation from [1]. Notably, our notion of correctness permits injections on the unauthenticated portion of the transcript. Finally, we introduce the novel notion of *continuous key agreement* (CKA) with *tamper detection*, which enables parties to recover their states once tampering has been detected.

3.1 Reverse Firewalls

A reverse firewall W is a stateful algorithm that maps messages to messages. For a party P and reverse firewall W , $W \circ P$ is defined as the following composition: for each incoming message to P , W is applied to it before P sees it, and the output of W is forwarded to P . Symmetrically, messages sent by P pass through W (before leaving the local network of P) and its output is sent to the corresponding recipient. W has access to all public parameters, but not to the private input or state of P .

Similarly to [51], we assume that parties execute a cryptographic protocol realizing some functionality $\mathcal{F}$ that satisfies a correctness property and security requirements $\mathbf{S}$. For instance, a functionality requirement $\mathcal{F}$ might require that parties output the same value when the protocol execution ends, which should be the case for all key-exchange protocols. In this setting, a security requirement $\mathbf{S}$ might ask that no efficient adversary can distinguish between the transcript of the protocol and a uniformly random string.

Throughout the paper, we use $\tilde{P}$ to represent functionality maintaining adversarial implementations of party P , i.e., implementations of P that still satisfy the functionality requirements of the protocol. For a protocol Π with party P , we write $\Pi_{P \rightarrow \tilde{P}}$ to represent the protocol in which the role of party P is replaced by party $\tilde{P}$, and this generalizes to $\Pi_{P_1 \rightarrow \tilde{P}_1, \dots, P_n \rightarrow \tilde{P}_n}$, to denote that the role of P_i is replaced by $\tilde{P}_i$, $i \in [n]$, in Π . For any party P , we define $P(X, a_1, \dots, a_\ell) := (b_1, \dots, b_\ell)$ as the list of messages sent by P on input X on receiving messages $a_1, \dots, a_\ell$. This is a random variable over the randomness of P . We denote by $P(X, a_1, \dots, a_\ell; r)$ the deterministic function in which the coins of P are fixed to r .

Definition 17 (Functionality preservation [51]). A reverse firewall W maintains the functionality $\mathcal{F}$ for a party P in a protocol Π if protocol Π satisfies $\mathcal{F}$, the protocol $\Pi_{P \rightarrow W \circ P}$ satisfies $\mathcal{F}$, and the protocol $\Pi_{P \rightarrow W \circ \dots \circ W \circ P}$ also satisfies $\mathcal{F}$.

In [32], whenever the adversary sends an invalid message – such as an invalid opening to a commitment, a signature that does not verify, or a ciphertext that fails to decrypt – the other party aborts, and the execution terminates. In secure messaging, this event can be easily triggered if a network adversary tampers with communication. Therefore, our protocols must recover from such denial-of-service attacks (as opposed to [32]). However, this recovery mechanism could be exploited by subverted parties to exfiltrate private information through selective failure attacks. Therefore, we require subversions to be explainable, as defined below.

Definition 18 (Explainable implementations). $\tilde{P}$ is an explainable implementation of party P , if for every input X , messages $a_1, \dots, a_\ell$, and randomness r' , there exists randomness r such that $P(X, a_1, \dots, a_\ell; r) = \tilde{P}(X, a_1, \dots, a_\ell; r')$.

The high level idea behind the above definition is highlighted as follows. Consider the messages, $(b_1, \dots, b_{\hat{\ell}})$, produced by a subverted party $\tilde{P}$, assuming the messages of the other party are $(a_1, \dots, a_\ell)$. If for some r' , $\tilde{P}$ produces b_i, b_j , $i, j \in [\hat{\ell}]$, such that b_i is a commitment and b_j is candidate opening for b_i , then, b_j should always be a valid opening w.r.t. b_i , since there exists no r for which an honest party would produce an invalid commitment-opening pair (assuming perfect correctness of the commitment scheme). A similar argument prevents the adversary from producing invalid ciphertexts at will or mounting other types of selective failure attacks to exfiltrate private information.

Definition 19 (Functionality maintaining and explainable implementations). $\tilde{P}$ is a functionality maintaining and explainable implementation of P with respect to the functionality $\mathcal{F}$ in protocol Π , if protocol Π satisfies $\mathcal{F}$, the protocol $\Pi_{P \rightarrow \tilde{P}}$ satisfies $\mathcal{F}$ and $\tilde{P}$ is an explainable subversion of P .

We now define security preservation along the lines of [51, 32].

Definition 20 (Security preservation [51]). A reverse firewall preserves the security property S for a party P in protocol Π , if protocol Π satisfies S , and for any functionality maintaining explainable $\tilde{P}$, the protocol $\Pi_{P \rightarrow W \circ \tilde{P}}$ satisfies S . I.e., the firewall enables security preservation of the security properties of Π even when the adversary has substituted P with a functionality maintaining and explainable implementation of P , $\tilde{P}$.

Exfiltration Resistance Security Game

```

EX-RE( $\lambda, \Pi, P_1, P_2, W$ ):
   $(\tilde{P}_1, \tilde{P}_2, X) \leftarrow \mathcal{A}(1^\lambda)$ 
   $b \xleftarrow{\$} \{0, 1\}$ 
  if  $b = 1$  then
     $P^* \leftarrow W \circ \tilde{P}_1$ 
  else
     $P^* \leftarrow W \circ P_1$ 
   $T \leftarrow \Pi_{P_1 \rightarrow P^*, P_2 \rightarrow \tilde{P}_2}(X)$ 
   $b' \leftarrow \mathcal{A}(T, S_{\tilde{P}_2})$ 
  return  $(b == b')$ 

```

Figure 4: The exfiltration resistance security game for a reverse firewall W and party P_1 for a protocol Π against party P_2 , with input X . $\mathcal{A}$ denotes the adversary, λ the security parameter, $S_{\tilde{P}_2}$ the state of $\tilde{P}_2$ after the run of the protocol Π on input X and T is the transcript resulting from a run of $\Pi_{P_1 \rightarrow P^*, P_2 \rightarrow \tilde{P}_2}$. By the order of quantifiers, the adversary knows the firewall's, the parties' and the protocols' code.

Definition 21 (Exfiltration resistance [32]). A reverse firewall W is exfiltration resistant for party P_1 against party P_2 in protocol Π satisfying functionality $\mathcal{F}$ if no PPT algorithm $\mathcal{A}$ that

outputs functionality maintaining and explainable $\tilde{P}_1$ and $\tilde{P}_2$, has non-negligible advantage in $\text{EX-RE}(\lambda, \mathbf{II}, P_1, P_2, W)$ (cf. [Sec. 3.1](#)).

Note that, following [\[32\]](#) as well as [\[1\]](#), the definition is single-instance.

3.2 Secure Messaging

In the current section we revisit the definition of Secure Messaging by [\[1\]](#), with the only difference being that in our notion of correctness we allow injections against the unauthenticated part of the transcript. Therefore, we aim for a stronger notion of correctness in which during message delivery, the adversary is allowed to deliver a ciphertext in which the unauthenticated part is arbitrary.¹³ Such injections do not restrict parties from exchanging messages securely, i.e., our SM notion and protocol avoids denial of service attacks. We do so by extending the syntax of the “receive” operation to also output a special flag $d \in \{\top, \perp\}$, indicating whether an injection has been detected, which happens when $d = \perp$. Note that, we can still have $d = \perp$, while decryption outputs a valid message.

Syntax. A secure-messaging (SM) scheme consists of four probabilistic algorithms $\text{SM} = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Rcv})$, where

- **Init-A** (and similarly **Init-B**) receives a key k and outputs a state $s_A \leftarrow \text{Init-A}(k)$.
- **Send** receives a state s and a message m and produces a new state and a ciphertext $(s', c) \leftarrow \text{Send}(s, m)$.
- **Rcv** receives a state s and a ciphertext c and produces a new state, an epoch number, an index, a message m and flag $d \in \{\top, \perp\}$, $(s', t, i, m, d) \leftarrow \text{Rcv}(s, c)$.

The authenticated part of a ciphertext c will be denoted by $c.a$.

The security game, which is depicted in [Figure 5](#), consists of an initialization procedure **init** and of two “send” oracles, **transmit-A** (normal transmission) and **chall-A** (challenge transmission), two “receive” oracles, **deliver-A** (honest delivery) and **inject-A** (for forged ciphertexts) and a corrupt oracle **corr-A**. Those oracles are for party A. The oracles for B are similar. Moreover, [Figure 5](#) also features an epoch-management function ***ep-mgmt**, a function ***sam-if-nec** which relates to bad randomness, and two record-keeping functions ***record** and ***delete**; these functions cannot be called by the attacker. The game is parametrized by δ_{SM} , Δ_{SM} , which relate to how fast parties recover from a state compromise, and how fast they can be corrupted after a challenge, respectively. An extensive discussion on the oracle descriptions and how the game handles epochs, message indices, record keeping and corruptions, is deferred for [Sec. 2.4](#).

The advantage of $\mathcal{A}$ against an SM scheme SM is denoted by $\text{Adv}_{\delta_{\text{SM}}, \Delta_{\text{SM}}}^{\text{SM}}(\mathcal{A})$. The attacker is parameterized by the total number of queries q it makes, and the maximum number of epochs q_{ep} it runs for.

Definition 22. A secure-messaging scheme SM is $(q, q_{\text{ep}}, \delta_{\text{SM}}, \Delta_{\text{SM}}, \varepsilon)$ -secure if for all (q, q_{ep}) -attackers $\mathcal{A}$, $\text{Adv}_{\delta_{\text{SM}}, \Delta_{\text{SM}}}^{\text{SM}}(\mathcal{A}) \leq \varepsilon$.

Note that honest delivery only requires the authenticated part of the ciphertext, denoted by $c.a$, to be in the list of honestly transmitted messages, i.e., correctness holds even if the adversary

¹³The notion of correctness of [\[1\]](#) allows the adversary to only deliver honestly generated messages.

Security Game for Secure Messaging

Parameters: $SM = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Rcv})$ with key space $\mathcal{K}$; safety parameters: δ_{SM}, Δ_{SM} .

```

// Initialization
init
   $k \xleftarrow{\$} \mathcal{K}$ 
   $s_A \leftarrow \text{Init-A}(k)$ 
   $s_B \leftarrow \text{Init-B}(k)$ 
   $(t_A, t_B) \leftarrow (0, 0)$ 
   $i_A[t_A], i_B[t_B] \leftarrow 0$ 
   $t_L \leftarrow -\infty$ 
   $\text{trans}, \text{chall}, \text{comp} \leftarrow \emptyset$ 
   $b \xleftarrow{\$} \{0, 1\}$ 
   $\text{rs} \leftarrow \text{false}, d_{\text{prv}} \leftarrow \top$ 

// Corrupt party A
corr-A
  req  $B \notin \text{chall} \wedge \text{safe-corr}_P$ 
   $\text{comp} \leftarrow \text{trans}(B)$ 
   $t_L \leftarrow \max(t_A, t_B)$ 
  return  $s_A$ 

// Deliver for A
deliver-A(c)
  req  $(B, t, i, m, c.a) \in \text{trans}$ 
  for some  $t, i, m$ 
   $(s_A, t', i', m', d) \leftarrow \text{Rcv}(s_A, c)$ 
  if  $(t', i', m') \neq (t, i, m)$  then
    win
  if  $(t, i, m) \in \text{chall}$  then
     $m' \leftarrow \perp$ 
  if  $d = \perp$  then  $\text{rs} \leftarrow \text{true}$ 
  if  $d_{\text{prv}} = \top$  then  $t_A \leftarrow \max(t_A, t)$ 
  *delete( $t, i$ );  $d_{\text{prv}} \leftarrow d$ 
  return  $(t', i', m')$ 

// Inject for A
inject-A(c)
  req  $(B, c.a) \notin \text{trans} \wedge \text{safe-inj}$ 
   $(s_A, t', i', m', d) \leftarrow \text{Rcv}(s_A, c)$ 
  if  $m' \neq \perp \wedge (B, t', i') \notin \text{comp}$  then
    win
  if  $d = \perp$  then  $\text{rs} \leftarrow \text{true}$ 
  if  $d_{\text{prv}} = \top$  then  $t_A \leftarrow \max(t_A, t')$ 
  *delete( $t', i'$ );  $d_{\text{prv}} \leftarrow d$ 
  return  $(t', i', m')$ 

// Transmit for A
transmit-A( $m, r$ )
   $(r, \text{flag}) \leftarrow \text{*sam-if-nec}(r)$ 
  *ep-mgmt( $A, \text{flag}$ )
   $i_A[t_A]++$ 
   $(s_A, c) \leftarrow \text{Send}(s_A, m; r)$ 
  *record( $A, \text{norm}, m, c$ )
  return  $c$ 

// Challenge for A
chall-A( $m_0, m_1, r$ )
   $(r, \text{flag}) \leftarrow \text{*sam-if-nec}(r)$ 
  *ep-mgmt( $A, \text{flag}$ )
  req  $\text{safe-ch}_A \wedge |m_0| = |m_1|$ 
   $i_A[t_A]++$ 
   $(s_A, c) \leftarrow \text{Send}(m_b; r)$ 
  *record( $A, \text{chall}, m_b, c$ )
  return  $c$ 

// Helpers
*ep-mgmt( $P, \text{flag}$ )
  if  $(P = A \wedge t_P \text{ even}) \vee (P = B \wedge t_P \text{ odd})$  then
    if  $\text{flag} = \text{bad} \wedge \neg \text{safe-ch}_P$  then
       $t_L \leftarrow t_P + 1$ 
    if  $\neg \text{rs}$  then
       $t_P++$ 
       $i_P[t_P] \leftarrow 0$ 
   $\text{rs} \leftarrow \text{false}$ 

*record( $P, \text{flag}, m, c$ )
   $\text{rec} \leftarrow (P, t_P, i_P, m, c)$ 
   $\text{trans} \leftarrow \text{rec}$ 
  if  $\neg \text{safe-ch}_P$  then  $\text{comp} \leftarrow \text{rec}$ 
  if  $\text{flag} = \text{chall}$  then  $\text{chall} \leftarrow \text{rec}$ 

safe-ch $_P \iff t_P \geq t_L + \delta_{SM}$ 
safe-inj  $\iff \min(t_A, t_B) \geq t_L + \delta_{SM}$ 
safe-corr $_P \iff \text{for } t : (\cdot, t, \cdot, \cdot, \cdot) \in \text{chall},$ 
  req  $t_P \geq t + \Delta_{SM}$ 

*delete( $t, i$ )
   $\text{rec} \leftarrow (P, t, i, m, c)$  for some  $P, m, c$ 
   $\text{trans}, \text{chall}, \text{comp} \leftarrow \text{rec}$ 

*sam-if-nec( $r$ )
   $\text{flag} \leftarrow \text{bad}$ 
  if  $r = \perp$  then
     $r \xleftarrow{\$} \mathcal{R}$ 
     $\text{flag} \leftarrow \text{good}$ 
  return  $(r, \text{flag})$ 

```

Figure 5: Oracles corresponding to party A of the SM security game for a scheme $SM = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Rcv})$. Oracles for B are analogous.

injects against the unauthenticated part of c , and that valid injections require the authenticated part of c , $c.a$, to not belong in the set of transmitted messages. Also, `*ep-mgmt` in the next `Send` operation does not advance the counters to the next epoch if the receive operation outputs $d = \perp$, since the “resend” variable, `rs`, is set to `true`, i.e., tampering has been detected. This implies that in our protocol parties do not advance to the next epoch until the receive operation outputs $d = \top$. In this way we model tamper detection against the unauthenticated part of the transcript, in which case parties do not advance to the next epoch until they “resend” the unauthenticated part of the SM transcript and the receive operation outputs $d = \top$. Parties advance to the next epoch when processing incoming ciphertexts, if the last `Rcv` outputs $d_{\text{prv}} = \top$, in which case t_A is incremented if t is larger than t_A , and the record corresponding to c is deleted.

3.3 Continuous Key Agreement with Tamper Detection

In the current section we present a notion of *continuous key agreement* (CKA) with *tamper detection*. Our notion enables parties to recover their states once tampering has been detected and enables us to prove a stronger notion of correctness for the higher layer protocol, in which the adversary is allowed to inject against the unauthenticated part of the transcript.

Syntax. A *continuous-key-agreement* (CKA) scheme is a vector of algorithms $\text{CKA} = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Receive})$, where

- $\gamma^A \leftarrow \text{Init-A}(k)$: receives a key k and produces an initial state $\gamma^A \leftarrow \text{Init-A}(k)$. `Init-B` is similar producing γ^B .
- $(\gamma', T', I) \leftarrow \text{Send}(\gamma)$: receives a state γ and produces a new state γ' , message T' and key I .
- $(\gamma', I, d) \leftarrow \text{Receive}(\gamma, T)$: receives a state γ and message T , and produces a new state γ' , key I and $d \in \{\perp, \top\}$.

In what follows the output of the `Send` operation, T' , will split into the pair (T, T_a) , where T_a denotes the part of the CKA message that needs to be authenticated by the higher layer protocol and therefore cannot be mauled by the adversary during the CKA security game. By $T'.ua$ we will refer to the unauthenticated part of T' . Finally, we denote by $\mathcal{K}$ the space of initialization keys k and by $\mathcal{I}$ the space of CKA keys I .

Correctness. A CKA scheme is correct if in the security game of Fig. 6, if the adversary is not injecting any messages, then A and B compute the same key in every round, with probability 1.

Security. The basic property a CKA scheme must satisfy is that conditioned on the transcript $T_1, T_2, \dots$, the keys $I_1, I_2, \dots$, look uniformly random and independent. An attacker against our CKA notion is given the power to possibly (1) control the random coins used by the sender, (2) leak the current state of either party, and (3) inject against the unauthenticated part (denoted by $T.ua$) of the transcript (injections are captured by the **Inject-A** oracle). The CKA keys for which (1) or (2) hold, need not be secure, however parties are required to recover from state compromise within δ_{CKA} epochs, which is a parameter in our security game.¹⁴ Furthermore, keys for which (1), (2), do

¹⁴[1] considers $\delta_{\text{CKA}} = 2$.

not hold, are required to be secure, even in the presence of injections against the unauthenticated part of the transcript.

The security game for CKA is depicted in Fig. 6. It begins with a call to the **Init** oracle, which samples a bit b , initializes the states of both parties, and defines epoch counters t_A and t_B . The oracle receives as input a value t^* , which determines in which round the challenge oracle may be called. δ and d are helpers variables related to the tamper detection property.

Upon completion of the initialization procedure, the attacker gets to interact arbitrarily with the remaining oracles, as long as *the calls are in a “ping-pong” order*, i.e., a call to a send oracle for A is followed by a receive or inject call for B, then by a send oracle for B, etc. The attacker only gets to use the challenge oracle for epoch t^* . No corruption or using bad randomness is allowed less than δ_{CKA} epochs before the challenge is sent (see **allow-corr**).

As in [1], our CKA security game is parametrized by Δ_{CKA} , which stands for the number of epochs that need to pass after t^* until the states do not contain secret information pertaining to the challenge. Once a party reaches epoch $t^* + \Delta_{\text{CKA}}$, its state may be revealed to the attacker (via the corresponding corruption oracle). The game ends (not made explicit) once both states are revealed after the challenge phase. The attacker wins the game if it eventually outputs a bit $b' = b$.

As mentioned above, in our work, besides Δ_{CKA} , we also consider the number of epochs required for a CKA protocol to heal after a corruption, denoted by δ_{CKA} . [1] considers $\delta_{\text{CKA}} = 2$ for all protocols, i.e., it is assumed that all CKA protocols heal after two epochs, however, due to firewall mauling, our protocols require more rounds of interaction to produce new CKA keys, thus secrets remain in the parties states longer and healing requires more rounds.

Security against active adversaries and recovery. Our notion achieves the following (additional) property: injections against the unauthenticated part of the CKA messages (implemented by the oracle **Inject-A**) that lead to diverging CKA states and keys, are detectable, with overwhelming probability. This is captured by the helper function ***bcorr-or-rec**, which requires the output of the CKA receive operation, d , to be $\perp$ iff A and B compute different keys for the next epoch (denoted by I_{next}),¹⁵ otherwise the adversary wins the game.

When the adversary injects against the unauthenticated part of a CKA message, which happens when $T.\text{ua} \neq T_{t_A}.\text{ua}$ and $T.\text{a} = T_{t_A}.\text{a}$, in a way such that $d = \perp$ (i.e., A, B, compute different next keys) δ is set to 0 (cf. the ***bcorr-or-rec** helper function), the parties resend CKA messages and the counter δ increases with every such message. The definition requires that, if the adversary remains passive for δ_{CKA} epochs, or tampers in a way that creates consistent states, then A and B should eventually compute the same key (the case of $\delta = \delta_{\text{CKA}} \wedge d = \top$), otherwise the parties reinitiate the resend process, i.e., δ is set to 0. Note that, send, receive and inject operations, increase the epoch counter only if no tampering has been detected or parties have recovered from tampering (in both cases $d = \top$). **Send** advances the epoch only if the last **Receive** outputs $d = \top$ (if $d = \perp$ parties resend their messages), while **Receive** advances the epoch if the previous **Receive** outputs $\top$, i.e., $d_{\text{prv}} = \top$, independently of whether the current operation outputs $d = \perp$. The reason is that each call to **Receive** outputs a CKA key I that has been already authenticated in a previous round, while d relates to the next CKA key. At a high level, our CKA notion enables parties to recover their CKA states after injection is detected, by simply resending the unauthenticated messages sent by the **Send** operation in prior epochs.

Definition 23. A CKA scheme CKA is $(\Delta, \delta, \varepsilon)$ -secure if for all PPT attackers $\mathcal{A}$, $\text{Adv}_{\Delta, \delta}^{\text{CKA}}(\mathcal{A}) \leq \varepsilon$.

¹⁵The CKA key of the next epoch is part of the current epoch’s CKA state for both parties.

Security Game for CKA with Tamper Detection

Parameters: CKA scheme $\text{CKA} = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Receive})$ with key space $\mathcal{K}$, and safety parameters $\delta_{\text{CKA}}, \Delta_{\text{CKA}}$.

```

// Initialization
Init( $t^*$ )
 $k \xleftarrow{\$} \mathcal{K}$ 
 $\gamma_0^A \leftarrow \text{Init-A}(k)$ 
 $\gamma_0^B \leftarrow \text{Init-B}(k)$ 
 $t_A, t_B \leftarrow 0; d, d_{\text{prv}} \leftarrow \top$ 
 $b \xleftarrow{\$} \{0, 1\}; \delta \leftarrow \perp$ 

// Corrupt party A
Corr-A
  req allow-corr or finishedA
  return  $\gamma_{t_A}^A$ 

// Challenge
Chall-A
   $t_A++$ 
  req  $t_A = t^* \wedge d = \top$ 
   $(\gamma_{t_A}^A, T_{t_A}, I_{t_A}) \leftarrow \text{Send}(\gamma_{t_A-1}^A)$ 
  if  $b = 0$  then
    return  $(T_{t_A}, I_{t_A})$ 
  else
     $I \xleftarrow{\$} \mathcal{I}$ 
    return  $(T_{t_A}, I)$ 

// Inject A
Inject-A( $T$ )
  if  $d_{\text{prv}} = \top$  then  $t_A++$ 
  req  $T.\text{ua} \neq T_{t_A.\text{ua}} \wedge T.\text{a} = T_{t_A.\text{a}}$ 
   $(\gamma_{t_A}^A, I, d) \leftarrow \text{Receive}(\gamma_{t_A-1}^A, T)$ 
   $d_{\text{prv}} \leftarrow d$ 
  *bcorr-or-rec

// Send
Send-A( $r$ )
  if  $d = \top$  then  $t_A++$ 
  if  $r \neq \varepsilon$  then
    req allow-corr
     $(\gamma, T_{t_A}, I_{t_A}) \leftarrow \text{Send}(\gamma; r)$ 
  else
     $(\gamma, T_{t_A}, I_{t_A}) \leftarrow \text{Send}(\gamma)$ 
  return  $(T_{t_A}, I_{t_A})$ 

// Receive
Receive-A
  if  $d_{\text{prv}} = \top$  then  $t_A++$ 
   $(\gamma_{t_A}^A, I, d) \leftarrow \text{Receive}(\gamma_{t_A-1}^A, T_{t_A})$ 
   $d_{\text{prv}} \leftarrow d$ 
  *bcorr-or-rec

// Tamper Detection Logic
*bcorr-or-rec
  if  $(d = \top \wedge \gamma_{t_A}^A.I_{\text{next}} \neq \gamma_{t_B}^B.I_{\text{next}}) \vee (d = \perp \wedge \gamma_{t_A}^A.I_{\text{next}} = \gamma_{t_B}^B.I_{\text{next}})$  then
    win
    if  $\delta = \perp \wedge d = \perp$  then  $\delta \leftarrow 0$ 
    if  $\delta \neq \perp$  then  $\delta++$ 
    if  $\delta = \delta_{\text{CKA}}$  then
      if  $d = \top$  then
         $\delta \leftarrow \perp$ 
      else
         $\delta \leftarrow 0$ 

// Capping Conditions
allow-corr :  $\iff \max(t_A, t_B) \leq t^* - \delta_{\text{CKA}}$ 
finishedp :  $\iff t_p \geq t^* + \Delta_{\text{CKA}}$ 

```

Figure 6: Oracles corresponding to party A of the CKA security game for a scheme $\text{CKA} = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Receive})$. The oracles for B are defined analogously.

4 The CKA Protocol

In the current section we present our CKA protocol, which is depicted in Fig. 8.

As discussed already in Sec. 1.2, the key ingredient in our construction is the one-time subversion resilient key-agreement protocol of [32], against passive adversaries. In the protocol of [32], which we will refer to as SRKA, A first commits to g^a , B sends g^b , and finally A sends the opening g^a to the commitment. This protocol requires the commitment scheme to be *rerandomizable* and *malleable*, so that the firewall can rerandomize both the commitment, as well as the committed group element.

Our protocol’s execution relates to four interleaved (but independent) sessions of SRKA. For starters, and to ease our exposition, consider the execution depicted in Fig. 7, in which A initiates a session of SRKA, by sending a commitment, C_0 , to a uniformly random group element, A_0 , to B. B responds, with the second message, by sending the response B_0 , while in addition it initiates another session of SRKA, by sending a fresh commitment C_1 . Subsequently, A computes the first key I_0 , sends a response A_1 to B’s commit, and initiates another session of SRKA. After B sends its second message, the execution converges to a point in which parties, with every “send” operation, compute the following. **Current CKA key:** parties compute the CKA key derived from the oldest SRKA session, say I , which is the key that will be used by the higher layer protocol to encrypt the party’s current message. **Next CKA key:** parties compute the next CKA key, which relates to the second oldest session of SRKA. **Response for SRKA:** samples a uniformly random group element as a response to the last commitment sent by B for the third oldest session of SRKA. **New SRKA session:** initiates a new session of SRKA by sampling a fresh group element and computing the commitment over it.

Our protocol follows the flow presented above, but it pushes into the initial states of the parties the protocol rounds until convergence to a point in which every send operation outputs: (1) an opening to a previously sent commitment, (2) a response to a commitment sent by the other party, and (3) a fresh commitment initiating a new session of SRKA. Our protocol is presented in detail in the next section.

4.1 Protocol Operations

The operations in our CKA protocol are defined as follows.

Initialization. During initialization the epoch counters, t_A, t_B , are initialized to zero. The operation generates $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a') \leftarrow G(k)$, where G is a PRG, K_1, K_2 , denote keys in G , $a, \hat{a}, a'$, are private exponents in $\mathbb{Z}_p^*$ and op and $\hat{\text{op}}$ are openings w.r.t. C . A stores $(K_1, K_2, \text{op}, a')$, which will be used for when A acts as a sender, while $\hat{C}$ is used for when A acts as a receiver. T will store sent CKA messages for up to three epochs, in order for them to be resent if injections are detected. The case for Bob is symmetric.

At a high level, the operation enables parties to initialize their states with an authenticated Diffie-Hellman (DH) K_1 of a complete session of SRKA, an unauthenticated key of another session of SRKA, as well as, commitments and openings of open SRKA sessions. I.e., the initial state of parties stores the information for the first three rounds of the execution depicted in Fig. 7, and enables A, B, to initiate the execution from that point and on.

Send. If `resend` is not set, i.e., $\gamma.\text{rs} = \varepsilon$, the operation increases the counter of the sender and computes the next CKA key by evaluating H over $(K_t, 1)$. Next the operation computes the part

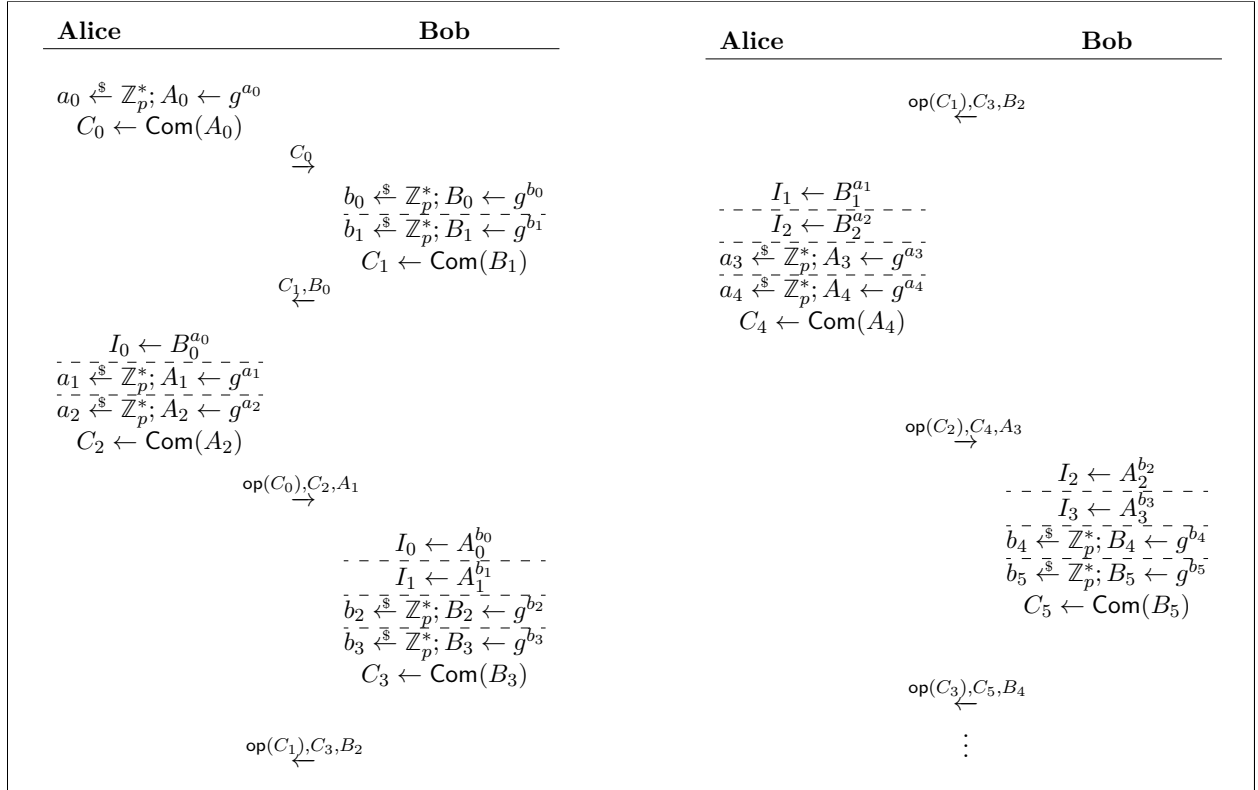


Figure 7: Interleaved executions of SRKA. The execution starts on the left-hand side of the figure.

CKA protocol

Parameters: Hash H , PRG G , commitment scheme C

// Initialization for A

Init-A(k)

```

 $\gamma^A.\text{ctr} \leftarrow 0; \gamma^A.\text{rs} \leftarrow \varepsilon$ 
 $(K_1, K_2, \cdot, \cdot, \text{op}, \hat{\text{op}}, a') \leftarrow G(k)$ 
 $\hat{C} \leftarrow C.\text{Com}(\hat{\text{op}}); \gamma^A.T[\cdot] \leftarrow \varepsilon$ 
 $\gamma^A.\text{snd} \leftarrow (K_1, K_2, \text{op}, a')$ 
 $\gamma^A.\text{rcv} \leftarrow (\cdot, \cdot, \cdot, \hat{C})$ 
return  $\gamma^A$ 

```

// Initialization for B

Init-B(k)

```

 $\gamma^B.\text{ctr} \leftarrow 0; \gamma^B.\text{rs} \leftarrow \varepsilon$ 
 $(K_1, \cdot, a, \hat{a}, \text{op}, \hat{\text{op}}, \cdot, \cdot) \leftarrow G(k)$ 
 $C \leftarrow C.\text{Com}(\text{op}); \gamma^B.T[\cdot] \leftarrow \varepsilon$ 
 $\gamma^B.\text{rcv} \leftarrow (K_1, a, \hat{a}, C)$ 
 $\gamma^B.\text{snd} \leftarrow (\cdot, \cdot, \hat{\text{op}})$ 
return  $\gamma^B$ 

```

// Helper function

*KE-sample (a')

```

 $a, \hat{a} \xleftarrow{\$} \mathbb{Z}_p, r \xleftarrow{\$} \{0, 1\}^\lambda$ 
 $A \leftarrow g^a; \hat{A} \leftarrow g^{\hat{a}}$ 
if  $t = 1 \wedge P = A$  then
   $A \leftarrow g^{a'}, a \leftarrow a'$ 
return  $(A, a, \hat{A}, \hat{a}, r)$ 

```

// Send

Send

```

if  $\gamma.\text{rs} \neq \varepsilon$  then
   $(\gamma, (T, T_a), I) \leftarrow \text{*resend.Send}(\gamma)$ 
  return  $(\gamma, (T, T_a), I)$ 
 $t \leftarrow \gamma.\text{ctr}++$ 
 $(K_t, K_{t+1}, \text{op}, a') \leftarrow \gamma.\text{snd}$ 
 $I \leftarrow H(K_t, 1)$ 
 $v \leftarrow H(K_{t+1}, 0)$ 
 $(A, a, \hat{A}, \hat{a}, r) \leftarrow \text{*KE-sample}(a')$ 
 $C \leftarrow C.\text{Com}(\hat{A}; r)$ 
 $\hat{\text{op}} \leftarrow (\hat{A}, r)$ 
 $\gamma.\text{rcv} \leftarrow (K_{t+1}, a, \hat{a}, \cdot)$ 
 $\gamma.\text{snd} \leftarrow (\varepsilon, \varepsilon, \hat{\text{op}}, \varepsilon)$ 
 $T \leftarrow (\text{op}, A, C); T_a \leftarrow v$ 
 $\gamma.T[t] \leftarrow T$ 
return  $(\gamma, (T, T_a), I)$ 

```

// Receive

Receive(T, T_a)

```

if  $T_a = \text{"init-res"} \vee \gamma.\text{rs} \neq \varepsilon$  then
  if  $T_a = \text{"init-res"}$  then
     $\gamma.\text{rs} \leftarrow \text{"init"}$ 
     $(\gamma, I, d) \leftarrow \text{*resend.Rcv}(\gamma)$ 
    return  $(\gamma, I, d)$ 
 $t \leftarrow \gamma.\text{ctr}++$ 
 $\gamma.T[t-3] \leftarrow \varepsilon$ 
 $(K_t, a, \hat{a}, C) \leftarrow \gamma.\text{rcv}$ 
 $(\text{op}, \hat{B}, \hat{C}) \leftarrow T$ 
 $(B, r) \leftarrow \text{op}$ 
 $I \leftarrow H(K_t, 1)$ 
if  $T_a \neq H(B^a, 0) \vee C.\text{Com}(B; r) \neq C$  then
   $\gamma.\text{rs} \leftarrow \text{"inj"}$ 
   $d \leftarrow \perp$ 
else
   $d \leftarrow \top$ 
 $\gamma.\text{snd} \leftarrow (B^a, \hat{B}^{\hat{a}}, \cdot, \cdot)$ 
 $\gamma.\text{rcv} \leftarrow (\cdot, \cdot, \cdot, \hat{C})$ 
return  $(\gamma, I, d)$ 

```

Figure 8: The main operations of our CKA scheme w.r.t. hash function H , PRG G and commitment scheme C .

of the output that needs to be authenticated by the higher layer protocol (and thus cannot be tampered in the CKA security game) as $v \leftarrow H(K_{t+1}, 0)$, i.e., authenticates the next CKA key, and samples fresh $(A, a, \hat{A}, \hat{a}, r)$, where, at a high level, $(A, a, \hat{A}, \hat{a})$, are the sender's contribution for two DH key-exchanges (A^a is the sender's response to a DH key-exchange session initiated already by the other party,¹⁶ while $\hat{A}$ is a newly initiated DH key-exchange), and r is the randomness for committing to $\hat{A}$. Finally, the operation updates the local state, stores the current sent T , and returns. If $\gamma.rs \neq \varepsilon$, parties execute **resend* which is discussed below.

Receive. If “resend” has not been initiated, i.e., $T_a \neq \text{“init-res”}$ and $\gamma.rs = \varepsilon$, the operation increases the epoch counter, the sent CKA message of epoch $t-3$ is erased as it has been authenticated and is not needed for resend, and after parsing the input and the receiver's state, computes the next CKA key $I \leftarrow H(K, 1)$, and stores the DH keys $B^a, \hat{B}^{\hat{a}}$, that will later enable the computation of the next two CKA keys, after checking whether $T_a = H(B^a, 0)$, i.e., if the authenticated part sent by the sender matches the one computed locally. If the parties compute different keys (for the next epoch), which happens if $T_a \neq H(B^a, 0)$, or if the opening to the commitment is invalid, $\gamma.rs$ is set to “inj”, and $d \leftarrow \perp$, which will initiate **resend* with the next “send” operation. Finally, the operation stores the newly received commitment C whose opening will be verified with the next receive operation. The operation returns the new CKA state and key, as well as $d = \perp$ or $d = \top$, depending on whether injection has been detected against the next CKA key.

4.1.1 Resending messages after tamper detection

If the execution of Receive results in setting $\gamma.rs = \text{“inj”}$, this indicates that tampering has been detected. In this case, the parties execute a recovery sub-protocol, denoted **resend*, which orchestrates a series of interactions to restore a consistent shared state. This recovery protocol is facilitated by helper functions (**resend.Send*, **resend.Rcv*), which trigger the selective re-transmission of prior commitments and responses. Recovery operates under the assumption that tampering was detected by B upon receiving a message from A, and hence B initiates the process. The recovery protocol is described below.

1. **Initialization:** The execution of **resend.Send* by B sends to A (ε, T_a) , where $T_a = \text{“init-res”}$, i.e., it signifies the initialization of **resend*.
2. **Resend commitment:** On receive (ε, T_a) , A sets $\gamma.rs \leftarrow \text{init}$. On her next **resend.Send*, sends (T, ε) , where T is the unauthenticated part of her CKA messages sent by A in epoch $t_A - 2$.
3. **Process commitment:** The next **resend.Rcv* operation by B locally executes $(\gamma, T') \leftarrow \text{StUp}(\gamma, T)$, and since $\gamma.rs = \text{“inj”}$ (cf. Fig. 8), it parses $(\cdot, \cdot, C) \leftarrow T$, sets $\gamma.rcv \leftarrow (\cdot, \cdot, C)$ and returns γ and $d = \perp$.
4. **Resend response to commitment:** On the next **resend.Send* operation, B sends $T' := T[t - 1]$ to A, i.e., resends B's response, T' , to A's last commitment.

¹⁶Note that the first response is computed deterministically via G , since the corresponding commitment/opening has been already fixed.

5. **Process response:** A executes $(\gamma, T'') \leftarrow \text{StUp}(\gamma, T')$, and since $\gamma.\text{rs} = \text{"init"}$, the operation parses $(\cdot, B, C) \leftarrow T'$, and executes $(\cdot, \cdot, a, \cdot) \leftarrow \gamma.\text{rcv}$, $(\cdot, B^a, \cdot) \leftarrow \gamma.\text{snd}$, $(\cdot, \cdot, \cdot, C) \leftarrow \gamma.\text{rcv}$, and returns γ and $d = \perp$.
6. **Send new T_a :** A sends (T, T_a) to B, where $T_a = H(B^a, 0)$ and T is the last unauthenticated part of the CKA message sent by A and sets $\gamma.\text{rs} \leftarrow \varepsilon$.
7. **Process new T_a :** Finally, on input (T, T_a) , B executes $\gamma \leftarrow \text{StUp}(\gamma, (T, T_a))$, that executes as the original “receive” operation, i.e., parses $((B, r), \hat{B}, \hat{C}) \leftarrow T$, $(\cdot, a, \hat{a}, \cdot) \leftarrow \gamma.\text{rcv}$, checks whether $\text{C.Com}(B; r) = C$ and $T_a = H(B^a, 0)$, and if not, it moves to step 1, otherwise computes $\gamma.\text{snd} \leftarrow (B^a, \hat{B}^{\hat{a}}, \cdot)$, sets $\gamma.\text{rs} \leftarrow \varepsilon$, and returns γ and $d = \top$.

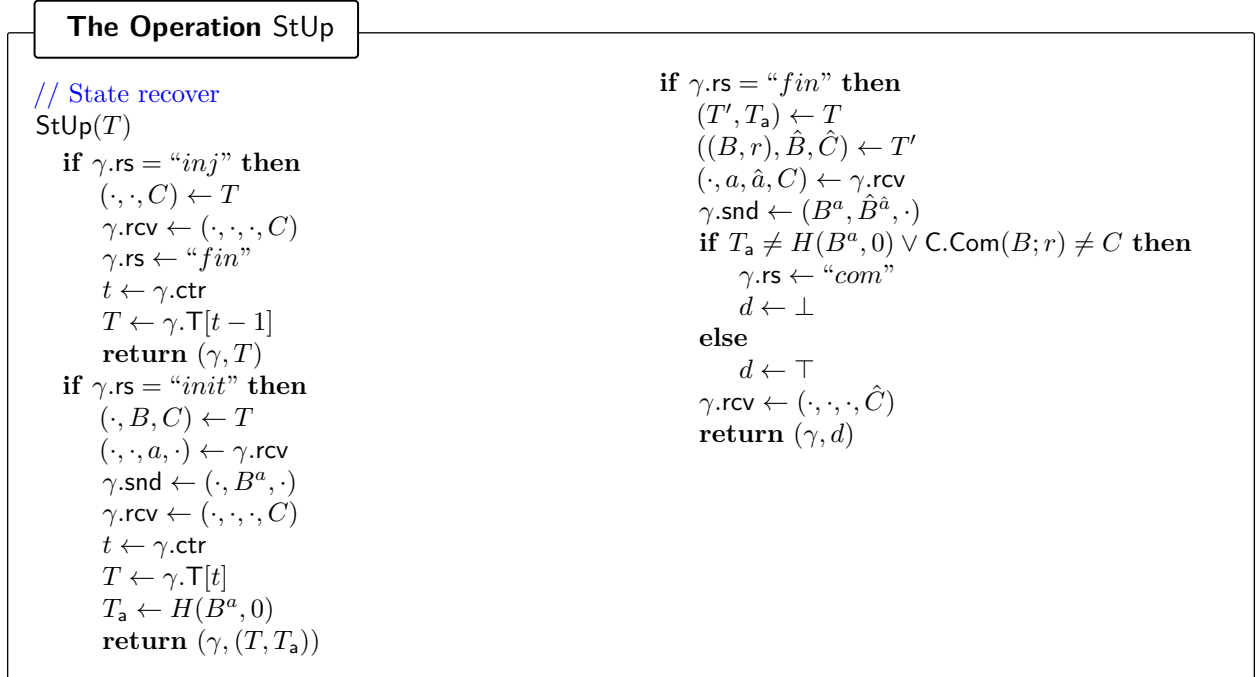


Figure 9: The main operations of our CKA scheme w.r.t. hash function H , PRG G and commitment scheme C .

The recovery process thus ensures that, if tampering is detected (e.g., through decryption errors or authentication failures), the affected commitments and responses are systematically re-sent and validated. If any step fails due to invalid or mismatched data, the protocol gracefully restarts. Importantly, once the execution terminates successfully, the parties resume from a consistent shared CKA state. If desired, they may also proceed to encrypt a fresh application message, even if the previous one was discarded during recovery.

The entire update logic is encapsulated in the helper function **StUp**, illustrated in Fig. 9, which updates the CKA state depending on the current value of $\gamma.\text{rs}$. Since our protocol executes independent sessions of the one-time protocol SRKA, the recovery mechanism operates by selectively substituting previously exchanged commitments and responses into the internal CKA state of each party.

4.2 CKA Security

In what follows we discuss the CKA security of our scheme at a high level. We do so by inspecting the private state of parties at an arbitrary epoch $i > 0$, and for each element of the state we use a subscript index to indicate the epoch that this element relates to, e.g., for a DH key K_i , the index i indicates that this is the DH key from which the key for epoch i is computed (via the computation of H), while for an exponent a_i , opening op_i or commitment C_i , the indices denote the fact that the (underlying) values *contribute* to the computation of the DH key of epoch i . For brevity, we refer to $\gamma^A.\text{ctr}$ (resp. $\gamma^B.\text{ctr}$) by ctr_A (resp. ctr_B). For an epoch $i > 0$ the state of the parties before or after a send or receive operation is depicted on Fig. 10.

CKA States				
	$\gamma^A.\text{snd}$	$\gamma^A.\text{rcv}$	$\gamma^B.\text{snd}$	$\gamma^B.\text{rcv}$
i odd, $\text{ctr}_A = \text{ctr}_B = i - 1$	$K_i, K_{i+1}, \text{op}_{i+1}$	C_{i+2}	op_{i+2}	$K_i, b_{i+1}, \hat{b}_{i+2}, C_{i+1}$
i odd, $\text{ctr}_A = i, \text{ctr}_B = i - 1$	op_{i+3}	$K_{i+1}, a_{i+2}, \hat{a}_{i+3}, C_{i+2}$	op_{i+2}	$K_i, b_{i+1}, \hat{b}_{i+2}, C_{i+1}$
i odd, $\text{ctr}_A = i, \text{ctr}_B = i$	op_{i+3}	$K_{i+1}, a_{i+2}, \hat{a}_{i+3}, C_{i+2}$	$K_{i+1}, K_{i+2}, \text{op}_{i+2}$	C_{i+3}
i even, $\text{ctr}_A = \text{ctr}_B = i - 1$	op_{i+2}	$K_i, a_{i+1}, \hat{a}_{i+2}, C_{i+1}$	$K_i, K_{i+1}, \text{op}_{i+1}$	C_{i+2}
i even, $\text{ctr}_A = i, \text{ctr}_B = i - 1$	op_{i+2}	$K_i, a_{i+1}, \hat{a}_{i+2}, C_{i+1}$	op_{i+3}	$K_{i+1}, b_{i+2}, \hat{b}_{i+3}, C_{i+2}$
i even, $\text{ctr}_A = i, \text{ctr}_B = i$	$K_{i+1}, K_{i+2}, \text{op}_{i+2}$	C_{i+3}	op_{i+3}	$K_{i+1}, b_{i+2}, \hat{b}_{i+3}, C_{i+2}$

Figure 10: The parties' states before and after executing the “send” operation that creates epoch i .

We observe that, if i is odd, i.e., epoch i is created by A via a Send operation, but A hasn't invoked the operation yet (i.e., $\text{ctr}_A = i - 1$), its $\gamma^A.\text{snd}$ state contains $(K_i, K_{i+1}, \text{op}_{i+1})$, where op_{i+1} was generated during the send operation that created epoch $i - 2$, while its $\gamma^A.\text{rcv}$ state contains the last commitment, C_{i+2} , sent by B when created epoch $i - 1$. B's send state contains op_{i+2} and receive state contains $(K_i, b_{i+1}, \hat{b}_{i+2}, C_{i+1})$. If i is odd, A has invoked the send operation (i.e., $\text{ctr}_A = i$) and B hasn't invoked the receive operation (i.e., $\text{ctr}_B = i - 1$), A's “send state” stores op_{i+3} , and the “receive state” stores $(K_{i+1}, a_{i+2}, \hat{a}_{i+3}, C_{i+2})$. B's state is as above. If i is odd, A has invoked the send operation and B has invoked the receive operation ($\text{ctr}_A = \text{ctr}_B = i$), A's state is as above and B's “receive state” stores C_{i+3} and “send state” stores $(K_{i+1}, K_{i+2}, \text{op}_{i+2})$.

The case of even i is symmetric and is depicted in Fig. 10. If the adversary challenges epoch i , we observe the following. The adversary is not allowed to corrupt any party at epoch $i - 1$, since in that case the party stores K_i from which the CKA key of epoch i is derived. The adversary is allowed to corrupt the sender that creates epoch i right after the send operation is complete, and the receiver, right after the receive operation is complete, since their states store keys, commitments, openings and exponents, that only relate to future keys, totally unrelated to K_i . In order for epoch i to be secure, the adversary should not be able to derive the related secret exponents, a_i, b_i . Note that commitments and openings do not increase the knowledge of the adversary via corruptions, since they are observed in the network traffic. Thus if at least one of a_i or b_i gets leaked, the adversary can use the observed B_i or A_i , to compute $K_i := g^{a_i b_i}$. By the protocol specification, and assuming i is odd, i.e., A creates epoch i via a Send operation, a_i has been generated by A during the Send operation that creates epoch $i - 2$, in which a_i is stored by A in order to be used by the receive operation of epoch $i - 1$, during which she computes and stores K_i . Therefore the adversary is not allowed to corrupt A at epochs $i - 1$ and $i - 2$, but is allowed to corrupt it at any epoch

$i_A \leq i - 3$, since A doesn't store any information related to a_i . The case of B is symmetric.

Given the above, and assuming the adversary challenges the epoch key of epoch t^* , the admissible corruptions and bad randomness attacks against the protocol of Fig. 8 are depicted in Fig. 11.

From the above we derive that for our protocol $\Delta_{\text{CKA}} = 0$, $\delta_{\text{CKA}} = 4$.¹⁷

On tamper detection. Observe that, if the adversary tampers with the unauthenticated part of the CKA transcript, i.e., with commitments, openings and responses to commitments, in a way such that after a session of SRKA is concluded, A computes a key $K_1 := g^{ab'}$, while B computes a key $K_2 := g^{a'b}$, such that $K_1 \neq K_2$, then this will be detected by B with probability 1, when A authenticates K_1 via $T_a = H(g^{ab'}, 0)$, in which case B will compute $T_a = H(g^{a'b}, 0)$, and the receive operation will output $d = \perp$. If the adversary tampers with the communication s.t. $K_1 = K_2$, we prove that the CKA key is indistinguishable from uniform.

Our result is formalized in the following theorem.

Theorem 3. *Let CKA be the CKA scheme of Fig. 8. Then, for any PPT adversary $\mathcal{A}$, $\text{Adv}_{\Delta, \delta}^{\text{CKA}} \leq q_{\text{ro}} \cdot \text{Adv}_{\text{inv-cdh}}^G + \text{Adv}_{\text{prg}}^G$, for $\Delta = 0$, $\delta = 4$, where q_{ro} is the number of queries to the random oracle made by $\mathcal{A}$, $\text{Adv}_{\text{inv-cdh}}^G$ denotes the advantage of $\mathcal{A}$ against the inverse CDH assumption (cf. Sec. 2.3.2), $\text{Adv}_{\text{prg}}^G$ is the advantage of $\mathcal{A}$ against the pseudorandomness property of G , and H is modeled as a random oracle.*

The proof of the above theorem relies on a sequence of hybrids. The first hybrid is the original CKA security game (cf. Fig. 6). The second hybrid substitutes the initial state of the parties by uniformly random values (instead of computing them as the output of PRG), thus the indistinguishability between the two hybrids follows from the security of PRG. The third hybrid, for the challenge epoch t^* , sets the CKA key to a uniformly random element I and the indistinguishability between the hybrids follows from the inverse CDH assumption (cf. Sec. 2.3.2). At a high level, since the adversary can tamper with the CKA transcript (i.e., the adversary can maul A's g^a (resp. B's g^b) to $g^{a'}$ (resp. $g^{b'}$)), A computes the key $g^{ab'}$, while B computes the key $g^{a'b}$, subject to $g^{ab'} = g^{a'b}$, as required by the CKA security definition, otherwise, tampering is detected and the key cannot be challenged. The reduction sets the inverse CDH challenge (g^a, g^b) to be A's and B's, respectively, CKA messages for epoch t^* , and the only way for the adversary to distinguish between the two hybrids, would be to query the random oracle H with $g^{ab'}$, thus the triple $(h, h^{1/a}, h^{1/b}) := (g^{ab'} = g^{a'b}, g^{b'}, g^{a'})$, is a valid solution to the inverse CDH problem. We present the full proof next.

Proof. Assume w.l.o.g. that t^* is odd, i.e., A sends the challenge; the case where t^* is even is handled analogously. Our proof proceeds via sequence of hybrids.

Hybrid H_0 . This hybrid is identical to the CKA security game of figure Fig. 6 w.r.t. CKA scheme of Fig. 8.

Hybrid H_1 . This hybrid executes as H_0 with the only difference being that the parties' states are initialized using uniformly random elements, i.e., the call to $G(k)$ during initialization is omitted, and $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a')$ is sampled uniformly at random from the corresponding space. The

¹⁷In fact $\delta_{\text{CKA}} = 3$ is also admissible depending on which party gets corrupted or uses bad randomness, and whether t^* is even or odd. However, $\delta_{\text{CKA}} = 4$ covers all cases.

indistinguishability between H_1 and H_0 is straightforward given the pseudorandomness property of G and the fact that its input is never exposed to the adversary. We formalize this in the following lemma.

Lemma 1. *Any PPT adversary cannot distinguish between H_0 and H_1 with advantage more than $\text{Adv}_{\text{prg}}^G$, where the probability runs over the randomness used by the two experiments.*

Proof. We define an adversary $\mathcal{B}$ that plays against the challenger of the PRG property of G as follows. $\mathcal{B}$ receives the challenge $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a')$ from the PRG challenger and uses that value to fully simulate the rest of the execution of H_0 . Observe that $\mathcal{B}$ can perfectly simulate the execution, and if the PRG challenger returns $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a')$ as the output of G , then $\mathcal{B}$ simulates H_0 , while if $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a')$ is uniformly random, then it simulates H_1 , and the distinguishing advantage is bounded by $\text{Adv}_{\text{prg}}^G$. $\square$

Hybrid H_2 . This hybrid diverges from H_1 as follows: for epoch t^* , it sets the CKA key to a uniformly random element I_{t^*} , instead of $H(g^{ab'}, 1)$, where $g^{ab'}$ is the key computed by $\mathcal{A}$ if the adversary tampers with the communication. Recall that we should have $g^{ab'} = g^{ba'}$, otherwise the key would have been rejected at epoch $t^* - 1$. We prove indistinguishability between the two hybrids via a reduction to the hardness of the inverse CDH assumption in G , assuming H is a random oracle. We formalize this idea in the following lemma.

Lemma 2. *Any PPT adversary cannot distinguish between H_1 and H_2 with advantage more than $q_{\text{ro}} \cdot \text{Adv}_{\text{inv-cdh}}^G$, where the probability runs over the randomness used by the two experiments.*

Proof. Let $\mathcal{A}$ be any PPT adversary and let (g, g^a, g^b) be a Diffie-Hellman triple output by the inverse CDH challenger. We define a reduction $\mathcal{B}$ that plays against the challenger and simulates the CKA protocol for $\mathcal{A}$ but embeds the triple into the CKA execution as follows:

- In epoch $t^* - 3$,¹⁸ where $\mathcal{B}$ generates a fresh group element B and sends a commitment of it to $\mathcal{A}$ (initiating a DH key-exchange session), $\mathcal{B}$ embeds part of the CDH challenge by sending a commitment of $B := g^b$. Assume the adversary given B outputs B' .
- In epoch $t^* - 2$, where $\mathcal{A}$ generates and sends a group element A as a reply to the DH key-exchange protocol initiated at epoch $t^* - 3$, the reduction uses $A := g^a$, sends it to the adversary who outputs A' .
- In epoch $t^* - 1$, where $\mathcal{B}$ computes $v \leftarrow H(A'^b, 0)$, the reduction computes a uniformly random v .
- In epoch t^* , where $\mathcal{A}$ (resp. $\mathcal{B}$) computes $I_{t^*} \leftarrow H(B'^a, 1)$ (resp. $I_{t^*} \leftarrow H(A'^b, 1)$), the reduction sets I_{t^*} to a uniformly random value for both parties (we know that $g^{ab'} = g^{a'b}$, otherwise the adversary would loose the CKA game in the previous epoch).

Observe that the reduction simulates perfectly the CKA execution (up to the point of returning a uniformly random element I_{t^*}) since the adversary never gets access to the private exponents a, b , of the challenge epoch. In particular, by the safety predicate, the adversary cannot control

¹⁸If the challenge key I_{t^*} is computed after an execution of `*resend`, we can simply set as t^* to be the index of “send” oracle query that initiates the DH session that computes the challenge key. Then $\mathcal{B}$ embeds B during that send operation, and the remaining values accordingly.

the randomness of B (resp. A) for epoch $t^* - 3$ (resp. $t^* - 2$), therefore, the reduction successfully embeds g^a, g^b , in the execution. Furthermore, the adversary is not allowed to corrupt A (resp. B) between epochs $t^* - 2$ (resp. $t^* - 3$) and t^* , while the state of both parties can be revealed when the computation of epoch t^* finishes, since at that point the parties' states are independent of a, b . Therefore the simulation is perfect unless $\mathcal{A}$ distinguishes I_{t^*} from the actual CKA key, i.e., distinguishes between H_1, H_2 .

Towards contradiction, assume this happens with probability at least ε , given that $g^{ab'} = g^{a'b}$. This implies that with probability at least ε , $\mathcal{A}$ will query the random oracle H with $g^{ab'}$ (as this is the only way to distinguish I_{t^*} from the actual key), and the triple $(h, h^{1/a}, h^{1/b}) := (g^{ab'}, g^{b'}, g^{a'})$, is a valid solution to the inverse CDH problem. $\mathcal{B}$ simply checks all the queries made by $\mathcal{A}$ to the RO and picks one uniformly at random as a candidate for h , and thus breaks the inverse CDH assumption with probability at least ε/q_{ro} . We have reached a contradiction, thus we conclude that any PPT adversary cannot distinguish between H_1, H_2 , with advantage more than $q_{ro} \cdot \text{Adv}_{\text{inv-cdh}}^G$. $\square$

From the above we conclude that $\text{Adv}_{\Delta, \delta}^{\text{CKA}} \leq q_{ro} \cdot \text{Adv}_{\text{inv-cdh}}^G + \text{Adv}_{\text{prg}}^G$. $\square$

Admissible Attacks	
Parameters: $t^*, \text{ctr}_A, \text{ctr}_B$.	
// Allowed Corruptions for A	// Allowed bad randomness for A
$\begin{cases} t^* \leq \text{ctr}_A \leq t^* - 3, & t^* \text{ is odd} \\ t^* \leq \text{ctr}_A \leq t^* - 4, & t^* \text{ is even} \end{cases}$	$\begin{cases} \text{ctr}_A \neq t^* - 2, & t^* \text{ is odd} \\ \text{ctr}_A \neq t^* - 3, & t^* \text{ is even} \end{cases}$
// Allowed Corruptions for B	// Allowed bad randomness for B
$\begin{cases} t^* \leq \text{ctr}_B \leq t^* - 4, & t^* \text{ is odd} \\ t^* \leq \text{ctr}_B \leq t^* - 3, & t^* \text{ is even} \end{cases}$	$\begin{cases} \text{ctr}_B \neq t^* - 3, & t^* \text{ is odd} \\ \text{ctr}_B \neq t^* - 2, & t^* \text{ is even} \end{cases}$

Figure 11: Admissible corrupt and bad randomness attacks w.r.t. target epoch t^* .

4.3 Subversion Resilience

In the current section we present the firewall (cf. Fig. 12) for the CKA protocol of Fig. 8 and we prove its subversion resilience. At a high level, the firewall (1) mauls outgoing commitments and their openings, i.e., for any outgoing commitment C to a value g^a sent by A, the firewall mauls (and rerandomizes) it to a commitment of $g^{aa_{\text{out}}}$ (for uniformly random a_{out}), mauls the opening to $g^{aa_{\text{out}}}$ and B's response, g^b , to $g^{ba_{\text{out}}}$, and (2) mauls incoming commitments and the corresponding responses, i.e., A's firewall, every incoming commitment to g^b sent by B, is mauled to $g^{ba_{\text{in}}}$ (for uniformly random a_{in}), and then A's response g^a is rerandomized to $g^{aa_{\text{in}}}$, while B's opening g^b is rerandomized to $g^{ba_{\text{in}}}$. B's firewall is symmetric.

A's firewall is depicted in Fig. 12 and works as follows. During initialization, it initializes a_{in} and a_{out} , that will be used to rerandomize incoming and outgoing traffic, respectively. In particular, $a_{\text{in}} = \varepsilon$ (resp. $a_{\text{out}} = \varepsilon$), implies that the firewall hasn't processed any incoming (resp. outgoing) messages, yet. If this is the case (i.e., the firewall is processing A's first outgoing message), it

only rerandomizes the outgoing commitment, by sampling $a_{\text{out}} \xleftarrow{\$} \mathbb{Z}_p^*$, mauling the commitment to $C \leftarrow \text{C.MaulCom}(C, a_{\text{out}})$, which creates a commitment to $g^{aa_{\text{out}}}$ (assuming the original commitment was for g^a), and finally rerandomizes the commitment by computing $C \xleftarrow{\$} \text{C.Rerand}(C)$. The reason why the first call to **Proc-out** is not modifying the response A and the opening op , is that for A 's first message, A and op , are deterministically computed as the output of the PRG, G , over the key k , while the corresponding commitment is also fixed on B 's side, thus given the security of the initial key and G , A , op , do not exfiltrate any information assuming functionality maintaining adversaries. If $a_{\text{out}} \neq \varepsilon$ (which will be the case when A sends its second commitment), a_{out} will be used to consistently rerandomize the opening to the commitment sent in the previous round, by computing $\text{op} \leftarrow \text{C.MaulOp}(\text{op}, a_{\text{out}})$. **Proc-in** symmetrically mauls incoming commitments and openings (besides the first opening which is fixed). Note that **Proc-out** mauls A 's responses using a_{in} , which has been already used by **Proc-in** to maul incoming commitments and their openings, while a_{out} is used to maul both A 's commitments and openings, as well as B 's responses. The firewall for B is symmetric.

W_{CKA} for *resend. If ***resend** has been triggered, which means that parties “resend” the unauthenticated part of the CKA transcript, in addition to the functionality depicted in Fig. 12, the CKA firewall needs to make sure that parties resend the same CKA messages. I.e., if, say A , is the initiator of a CKA session that failed, originally sending (op, A, C) (where C initiates the session), and B responded with (op', B, C') (where B is a response to C), the firewall keeps the original CKA messages as part of its state, and sends them during ***resend**. For simplicity we omit the pseudocode of this very simple functionality from Fig. 12.

Subversion resilience of the composition between the CKA protocol of Fig. 8 and the firewall of Fig. 12 is formally captured in the following theorem.

Theorem 4. *The reverse firewall W_{CKA} of Fig. 12 is functionality maintaining for the protocol of Fig. 8. Furthermore, assuming PRG security of G , the commitment scheme C satisfies tightness, rerandomizability and malleability, and that is statistically hiding and computationally binding (cf. Sec. 2.5), then W_{CKA} preserves security for A and B , and is exfiltration resistant for A against B and for B against A .*

Proof. We prove the theorem by showing that W_{CKA} is functionality preserving, exfiltration resistant and preserves security.

Functionality preservation. First we argue that W_{CKA} preserves functionality. We split our proof into two parts, namely for DH sessions that have been initialized by A or B , respectively.

With regards to DH sessions initiated by A , we observe the following: for each outgoing commitment C over a group element $A := g^a$, the firewall samples the exponent a_{out} and mauls the commitment to $\hat{C}$ (i.e., a commitment of $g^{aa_{\text{out}}}$), and keeps the exponent a_{out} as part of its state. When processing incoming messages, it mauls the response to the commitment, $B := g^b$, to $B^{a_{\text{out}}} = g^{ba_{\text{out}}}$, therefore A will compute the DH key $g^{aba_{\text{out}}}$. In her next message where she sends the opening to C , i.e., A sends g^a and the firewall mauls the opening to $g^{aa_{\text{out}}}$, therefore B will compute the same DH key $g^{aba_{\text{out}}}$.

Now we focus on DH sessions initiated by B . In particular, for each commitment over a group element g^b sent by B , A 's firewall rerandomizes the incoming commitment to a commitment of

Firewall for A W_{CKA}

Parameters: Commitment scheme C , group G , prime p .

```

// Init
Init
   $a_{\text{in}}, a_{\text{out}} \leftarrow \varepsilon$ 

// Process outgoing
Proc-out( $T$ )
   $(\text{op}, A, C) \leftarrow T$ 
  if  $a_{\text{in}} \neq \varepsilon$  then
     $A \leftarrow A^{a_{\text{in}}}$ 
  if  $a_{\text{out}} \neq \varepsilon$  then
     $\text{op} \leftarrow C.\text{MaulOp}(\text{op}, a_{\text{out}})$ 
   $a_{\text{out}} \xleftarrow{\$} \mathbb{Z}_p^*$ 
   $C \leftarrow C.\text{MaulCom}(C, a_{\text{out}})$ 
   $C \xleftarrow{\$} C.\text{Rerand}(C)$ 
   $T \leftarrow (\text{op}, A, C)$ 
  return  $T$ 

// Process incoming
Proc-in( $T$ )
   $(\text{op}, B, C) \leftarrow T$ 
  req  $B \in G \setminus \{1_G\}$ 
  if  $a_{\text{in}} \neq \varepsilon$  then
     $\text{op} \leftarrow C.\text{MaulOp}(\text{op}, a_{\text{in}})$ 
   $a_{\text{in}} \xleftarrow{\$} \mathbb{Z}_p^*$ 
   $C \leftarrow C.\text{MaulCom}(C, a_{\text{in}})$ 
   $B \leftarrow B^{a_{\text{out}}}$ 
   $T \leftarrow (\text{op}, B, C)$ 
  return  $T$ 

```

Figure 12: A's firewall for the CKA protocol of Fig. 8. The firewall for B is symmetric.

$g^{ba_{in}}$, and when A responds with, say $A := g^a$, the firewall rerandomizes it to $A^{a_{in}} = g^{aa_{in}}$, therefore B computes the DH key $g^{aa_{in}b}$. When A receives the opening to B's commitment, g^b , the firewall mauls it to $g^{ba_{in}}$, therefore A computes the correct key $g^{aa_{in}b}$. The case for B's firewall is symmetric. Observe that during the first **Send** operation by A the firewall simply forwards **op** and A since those values have been deterministically and securely computed by the initial state of the CKA protocol, and B already holds a commitment to **op**. For the same reason, the first **Receive** operation by A is not modifying the opening sent by B.

Exfiltration resistance. Now we prove that W is exfiltration resistant for A against B. Since A is functionality maintaining and explainable, we assume that she

- Always reveals the correct openings to her commitments.
- Only triggers ***resend** if the incoming T_a does not match the one that she computes locally.
- During ***resend** she always resends the same values,

since, according to [Definition 19](#), there is no randomness for which an honest party would reveal an invalid opening to the commitment, trigger ***resend**, without receiving an invalid T_a , or sending different values during ***resend**.

We prove that the transcript generated $\text{EX-RE}(\lambda, \Pi, A, B, W)$ for $b = 1$ is indistinguishable from the one generated for $b = 0$. The transcript includes commitments, openings, responses to commitments, as well as messages of ***resend**.

With regards to the commitments computed by A, observe that, after rerandomization and mauling, the commitments are uniformly random commitments of uniformly random group elements, independent of the original commitments sent by A, therefore they are indistinguishable from the commitments that A would send if she would be honest. By the tightness property of the commitment scheme, and since the adversary is functionality maintaining and explainable the openings to commitments are fixed, rerandomized (by the firewall) versions of the original openings, thus they follow the exact same distribution of openings sent by honest parties. With respect to the replies of A to commitments sent by B, since the firewall is rerandomizing the responses of parties to commitments using independent randomness, such responses are uniformly random group elements. W.r.t. the first opening and response, sent by A, recall that those values are fixed, as they are computed via the PRG over the X3DH key k , therefore the adversary cannot send arbitrary values as the protocol would fail. Therefore, for $b = 1$, the commitments, responses and openings, are indistinguishable from those generated when $b = 0$, and this holds also for messages generated during ***resend** since A always resends the same values, as otherwise the firewall would block her messages. The case of B against A is symmetric.

Security preservation. With regards to security preservation, assume w.l.o.g. that t^* is *odd*, i.e., A sends the challenge; the case where t^* is even is handled analogously. Our proof proceeds via sequence of hybrids.

Hybrid H_0 . This hybrid is identical to the CKA security game of figure [Fig. 6](#) w.r.t. $\text{CKA}_{A \rightarrow W \circ \tilde{A}}$ protocol of [Fig. 8](#).

Hybrid H_1 . Let $\tilde{A}^*$ be the truncation of $\tilde{A}$ to the point that computes the output of the PRG, i.e., $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a') \leftarrow G(k)$. If $\tilde{A}^*$ computes a different value H_1 outputs $\perp$ and aborts. The indistinguishability between the hybrids follows from the fact that $\tilde{A}$ is functionality maintaining and explainable, and is formalized in the following lemma.

Lemma 3. *Assuming $\tilde{A}$ is functionality maintaining and explainable, the output of H_0 matches that of H_1 .*

Proof. Observe that if A uses different current and next CKA keys $\text{key}_1, \text{key}_2$, clearly violates functionality as it computes different keys from B , and this will result into different commitments and openings that those agreed with B as the output of X3DH, and this is easily detected by B . $\square$

Hybrid H_2 . This hybrid executes as H_0 with the only difference being that the output of $\tilde{A}^*$, as well as B 's state, are set with $(K_1, K_2, a, \hat{a}, \text{op}, \hat{\text{op}}, a')$ that is sampled uniformly at random from the corresponding space. The indistinguishability between H_2 and H_1 is straightforward given the pseudorandomness property of G and the fact that its input is never exposed to the adversary. We formalize this in the following lemma.

Lemma 4. *Any PPT adversary cannot distinguish between H_2 and H_1 with advantage more than $\text{Adv}_{\text{prg}}^G$, where the probability runs over the randomness used by the two experiments.*

The proof of the lemma is along the lines of the proof of [Lem. 4](#).

Hybrid H_3 . If for epoch t^* , $\tilde{A}$ computes a different key than B during the execution, H_3 outputs $\perp$ and aborts. The indistinguishability between the hybrids follows from the fact that $\tilde{A}$ is functionality maintaining and explainable, and is formalized in the following lemma.

Lemma 5. *Assuming $\tilde{A}$ is functionality maintaining and explainable, the output of H_3 matches that of H_2 .*

Proof. Observe that in order for the CKA adversary to be able to challenge the key of epoch t^* , they need make sure that parties compute the same CKA key, i.e., it does not tamper with the commitments, openings and responses for that epoch. Therefore, if A outputs a different CKA key for epoch t^* she is clearly not functionality maintaining. $\square$

Hybrid H_4 . This hybrid diverges from H_3 as follows: for epoch t^* , it sets the CKA key to a uniformly random element I_{t^*} , instead of $H(g^{ab'}, 1)$, where $g^{ab'}$ is the key computed by A if the adversary tampers with the communication. Recall that we should have $g^{ab'} = g^{ba'}$, otherwise the key would have been rejected at epoch $t^* - 1$. We prove indistinguishability between the two hybrids via a reduction to the hardness of the inverse CDH assumption in G , assuming H is a random oracle. We formalize this idea in the following lemma.

Lemma 6. *Any PPT adversary cannot distinguish between H_4 and H_3 with advantage more than $q_{\text{ro}} \cdot \text{Adv}_{\text{inv-cdh}}^G$, where the probability runs over the randomness used by the two experiments.*

Proof. Let $\mathcal{A}$ be any PPT adversary and let (g, g^a, g^b) be a Diffie-Hellman triple output by the inverse CDH challenger. We define a reduction $\mathcal{B}$ that plays against the challenger and simulates $\text{CKA}_{A \rightarrow W \circ \tilde{A}}$ for $\mathcal{A}$ but embeds the triple into the CKA execution as follows:

- In epoch $t^* - 3$,¹⁹ where $\mathcal{B}$ generates a fresh group element B and sends a commitment of it to $\mathcal{A}$ (initiating a DH key-exchange session), $\mathcal{B}$ embeds part of the CDH challenge by sending a commitment of $B := g^b$. Assume the adversary given B outputs $B' := g^{b'}$, $\mathcal{B}$ executes $\tilde{\mathcal{A}}$ with input $(B')^r$, for freshly sampled r (this simulates the firewall operation).
- In epoch $t^* - 2$, where $\tilde{\mathcal{A}}$ generates and sends a group element A as a reply to the DH key-exchange protocol initiated at epoch $t^* - 3$, the reduction uses $A := g^a$ as the output of $\tilde{\mathcal{A}}$, sends to the adversary A^r , who outputs $A' := g^{a'}$.
- In epoch $t^* - 1$, where $\mathcal{B}$ computes $v \leftarrow H(A'^b, 0)$, the reduction computes a uniformly random v .
- In epoch t^* , where $\tilde{\mathcal{A}}$ (resp. $\mathcal{B}$) computes $I_{t^*} \leftarrow H(B'^{r\bar{a}}, 1)$ (resp. $I_{t^*} \leftarrow H(A'^b, 1)$) for some $\bar{a}$ chosen by $\tilde{\mathcal{A}}$, the reduction sets I_{t^*} to a uniformly random value for both parties (we know that $g^{rab'} = g^{a'b}$, otherwise the adversary would loose the CKA game in the previous epoch).

Observe that the reduction simulates perfectly the CKA execution (up to the point of returning a uniformly random element I_{t^*}) since the adversary never gets access to the private exponents a, b , of the challenge epoch. In particular, by the safety predicate, the adversary cannot control the randomness of $\mathcal{B}$ (resp. $\mathcal{A}$) for epoch $t^* - 3$ (resp. $t^* - 2$), therefore, the reduction successfully embeds g^a, g^b , in the execution. Furthermore, the adversary is not allowed to corrupt $\mathcal{A}$ (resp. $\mathcal{B}$) between epochs $t^* - 2$ (resp. $t^* - 3$) and t^* , while the state of both parties can be revealed when the computation of epoch t^* finishes, since at that point the parties' states are independent of a, b . Furthermore, $\mathcal{A}$'s CKA for epoch t^* is set independently of the real exponent, sat $\bar{a}$ that she might use, but the adversary is not allowed to corrupt her at epoch t^* . Therefore the simulation is perfect unless $\mathcal{A}$ distinguishes I_{t^*} from the actual CKA key, i.e., distinguishes between H_4, H_3 .

Towards contradiction, assume this happens with probability at least ε , given that $g^{rab'} = g^{a'b}$. This implies that with probability at least ε , $\mathcal{A}$ will query the random oracle H with $g^{ab'}$ (as this is the only way to distinguish I_{t^*} from the actual key), and the triple $(h, h^{1/a}, h^{1/b}) := (g^{ab'}, g^{rb'}, g^{a'})$, is a valid solution to the inverse CDH problem. $\mathcal{B}$ simply checks all the queries made by $\mathcal{A}$ to the RO and picks one uniformly at random as a candidate for h , and thus breaks the inverse CDH assumption with probability at least ε/q_{ro} . We have reached a contradiction, thus we conclude that any PPT adversary cannot distinguish between H_3, H_4 , with advantage more than $q_{ro} \cdot \text{Adv}_{\text{inv-cdh}}^G$. $\square$

From the above we conclude that $\text{Adv}_{\Delta, \delta}^{\text{CKA}, A \rightarrow W \circ \tilde{A}} \leq q_{ro} \cdot \text{Adv}_{\text{inv-cdh}}^G + \text{Adv}_{\text{prg}}^G$. $\square$

5 Secure Messaging Scheme

In the current section we present a secure messaging scheme that satisfies [Definition 22](#) while also being subversion resilient according to [Sec. 3.1](#). Similarly to previous Signal-based SM schemes, our scheme relies on continuous key-agreement, forward-secure authenticated encryption with associated data (FS-AEAD) and PRF-PRNGs. Since FS-AEAD and PRF-PRNG are deterministic (as opposed

¹⁹If the challenge key I_{t^*} is computed after an execution of `*resend`, we can simply set as t^* to be the index of “send” oracle query that initiates the DH session that computes the challenge key. Then $\mathcal{B}$ embeds B during that send operation, and the remaining values accordingly.

to CKA), in order for our SM scheme to be subversion resilient, the underlying CKA protocol needs to also be subversion resilient.

SM state. Our scheme keeps an internal state s_A (resp. s_B), which is initialized by `Init-A` (resp. `Init-B`), with input a common key k that is assumed to be the output of the X3DH protocol execution. The state is used and updated by `Send` and `Rcv`, and for party $P \in \{A, B\}$ consists of the following values: an implicit ID field with $\text{id} = P$, the PRF-PRNG state σ_{root} , states $v[0], v[1], v[2], \dots$, of the various FS-AEAD instances, the state γ of the CKA scheme, the current CKA message T_{cur} , and an epoch counter t_{cur} .

In order to remove expired FS-AEAD sessions from memory, we use the variable ℓ_{prv} that remembers the number of messages sent in the second most recent epoch. By the definition of FS-AEAD (cf. Section 2.6), once the maximum number of messages has been set via `FS-Max`, a session “erases” itself from the memory, and similarly for calling `FS-Stop` on a particular FS-AEAD session. The state s_B is defined similarly.

The algorithms. The algorithms of our scheme SM for party A are depicted in Figure 13 and described in more detail below. The algorithms for party B are symmetric. To improve readability, the parties states s_A, s_B , are not made explicit in the description, but we assume they consist of the variables set during initialization.

Initialization. The initialization operation `Init-A` receives a key k shared between A and B, where k is assumed to have been created securely, i.e., via an execution of the X3DH protocol between A and B. The key consists of initialization keys k_{root} and k_{CKA} for the PRF-PRNG state and the CKA state, respectively. Subsequently, the PRF-PRNG state is initialized with k , which is later used to generate a key for the FS-AEAD epoch $v[0]$. A acts as receiver in $v[0]$ and all subsequent even epochs and as sender in all subsequent odd epochs. Furthermore, `Init-A` also initializes the CKA state and sets the initial epoch $t_{\text{cur}} \leftarrow 0$, T_{cur} and ℓ_{prv} , to a default value. Initialization for B is symmetric.

Sending messages. The operation `Send-A` enables party A to send a message to party B. We first cover the case of $\gamma.\text{rs} = \varepsilon$, which is along the lines of [1]. In that case, the operation determines whether it is A’s turn to send the next CKA message, which is the case if t_{cur} is even, and if this holds, `Send-A` runs `Send` to produce the next CKA message T and key I , which is then absorbed into the PRF-PRNG state, outputting the new FS-AEAD key k used as the key for a new FS-AEAD epoch, in which A acts as sender. Finally, `Send-A` creates a header $h = (t, T_a, \ell_{\text{prv}})$, and uses the current epoch $v[t_{\text{cur}}]$ to compute a ciphertext for (h, m) (where h is treated as associated data). Recall, that only T_a is authenticated in the current epoch, while any tampering against T will eventually be detected due to the tamper detection property of the underlying CKA scheme. The case for which $\gamma.\text{rs} \neq \varepsilon$ is discussed below.

If $\gamma.\text{rs} \neq \varepsilon$, this implies that tampering has been detected at the CKA level and parties are resending unauthenticated CKA messages, therefore, the sender is not advancing to a new epoch, neither creates a new FS-AEAD instance, and the call `CKA-Send`(γ) is required in order to compute T_{cur} , which contains the information that needs to be resent. Furthermore, if $\gamma.\text{rs} = \text{“init”}$, this implies that the sender is not the party that detected tampering, therefore it always uses the latest valid FS-AEAD state, which corresponds to epoch t_{cur} , to send messages, while if $\gamma.\text{rs} \neq \text{“init”}$, then

Secure-Messaging Scheme

// Initialization for A

Init-A(k)

```

( $k_{\text{root}}, k_{\text{CKA}}$ )  $\leftarrow k$ 
 $\sigma_{\text{root}} \leftarrow \text{P-Init}(k_{\text{root}})$ 
( $\sigma_{\text{root}}, k$ )  $\leftarrow \text{P-Up}(\sigma_{\text{root}}, \varepsilon)$ 
 $v[\cdot] \leftarrow \varepsilon$ 
 $v[0] \leftarrow \text{FS-Init-R}(k)$ 
 $\gamma \leftarrow \text{Init-A}(k_{\text{CKA}})$ 
 $T_{\text{cur}} \leftarrow \varepsilon$ 
 $\ell_{\text{prv}}, t_{\text{cur}} \leftarrow 0$ 

```

// Receive

Rcv-A(c)

```

( $h, e, T$ )  $\leftarrow c$ ; ( $t, T_a, \ell$ )  $\leftarrow h$ ;  $d \leftarrow \top$ 
req  $t$  even and  $t \leq t_{\text{cur}} + 1$ 
if  $\gamma.\text{rs} \neq \varepsilon$  then
  ( $\gamma, I, d$ )  $\leftarrow \text{CKA-Receive}(\gamma, T_a, T)$ 
else
  if  $t = t_{\text{cur}} + 1$  then
     $t_{\text{cur}}++$ 
     $\text{FS-Max}(v[t-2], \ell)$ 
    ( $\gamma, I, d$ )  $\leftarrow \text{CKA-Receive}(\gamma, T_a, T)$ 
    ( $\sigma_{\text{root}}, k$ )  $\leftarrow \text{P-Up}(\sigma_{\text{root}}, I)$ 
     $v[t] \leftarrow \text{FS-Init-R}(k)$ 
  ( $v[t], i, m$ )  $\leftarrow \text{FS-Rcv}(v[t], h, e)$ 
  if  $m = \perp$  then
    error
  return ( $t, i, m, d$ )

```

// Send

Send-A(m)

```

if  $t_{\text{cur}}$  is even  $\vee \gamma.\text{rs} \neq \varepsilon$  then
  ( $\gamma, T_{\text{cur}}, I$ )  $\xleftarrow{\$} \text{CKA-Send}(\gamma)$ 
  if  $\gamma.\text{rs} = \varepsilon$  then
     $t_{\text{cur}}++$ 
     $\ell_{\text{prv}} \leftarrow \text{FS-Stop}(v[t_{\text{cur}}-1])$ 
    ( $\sigma_{\text{root}}, k$ )  $\leftarrow \text{P-Up}(\sigma_{\text{root}}, I)$ 
     $v[t_{\text{cur}}] \leftarrow \text{FS-Init-S}(k)$ 
  ( $T, T_a$ )  $\leftarrow T_{\text{cur}}$ 
  if  $\gamma.\text{rs} = \text{"init"} \vee \gamma.\text{rs} = \varepsilon$  then
     $t \leftarrow t_{\text{cur}}$ 
  else
     $t \leftarrow t_{\text{cur}} - 1$ 
   $h \leftarrow (t, T_a, \ell_{\text{prv}})$ 
  ( $v[t], e$ )  $\leftarrow \text{FS-Send}(v[t], h, m)$ 
  return ( $h, e, T$ )

```

Figure 13: Algorithms for the secure-messaging scheme $\text{SM} = (\text{Init-A}, \text{Init-B}, \text{Send}, \text{Rcv})$.

$\gamma.rs \neq \text{"inj"}$ and the sender corresponds to the party that detected the tampering, therefore it sends messages using its latest valid FS-AEAD state, which corresponds to the FS-AEAD epoch $t_{\text{cur}} - 1$.

Receiving messages. Assuming $\gamma.rs = \varepsilon$, on input ciphertext $c = (h, e, T)$ with header $h = (t, T_a, \ell)$, the operation Rcv-A accepts c only if t_{cur} is even (recall that even epochs are created by B) and computes as discussed below.

If $t \leq t_{\text{cur}}$, ciphertext c relates to an existing FS-AEAD epoch, in which case FS-Rcv is simply called on $v[t]$ to process e . If the maximum number of messages has been received for session $v[t]$, the session is removed from memory.

If $t = t_{\text{cur}} + 1$, in which case t_{cur} is odd, Rcv-A advances t_{cur} , and computes CKA Receive over inputs T, T_a . This produces I , that is absorbed into the PRF-PRNG state to obtain a key k that initializes the new FS-AEAD state, $v[t_{\text{cur}}]$, for A. Then, e is processed by FS-Rcv with state $v[t_{\text{cur}}]$. Note that Rcv also uses FS-Max to store ℓ as the maximum number of messages in the previous receive epoch. The operation outputs the epoch number, t , the FS-AEAD index i , the tamper detection variable d of CKA, and the message, m , unless decryption fails, in which case the operation terminates with “**error**”, which causes the entire state to be rolled back to what it was before the call (only when $t \leq t_{\text{cur}}$).

If $\gamma.rs \neq \varepsilon$, this implies that tampering has been detected at the CKA level and parties are resending unauthenticated CKA messages, therefore, the receiver is not advancing to a new epoch, neither creates a new FS-AEAD instance, and the call CKA-Receive is required in order to process resent CKA messages and update the CKA state accordingly.

5.1 Security of the SM Scheme

In the current section we prove that the scheme presented in Sec. 5 is secure, i.e., it satisfies *correctness*, *authenticity* and *privacy*. Following [1], proving SM security of our scheme is facilitated by considering three simplified properties, namely *correctness*, *authenticity*, and *privacy*. These properties together imply full SM security, as stated in Sec. 2.4.1. Our result is formalized in the following theorem.

Theorem 5. *Let SM be the construction of Fig. 13. Furthermore, assume that CKA is a $((\Delta_{\text{CKA}}, \delta_{\text{CKA}}), \varepsilon_{\text{cka}})$ -secure CKA scheme with tamper detection, FS-AEAD is a $(q, \varepsilon_{\text{fs-aead}})$ -secure FS-AEAD scheme, and P is a ε_{p} -secure PRF-PRNG. Then, SM is $(q, q_{\text{ep}}, \delta_{\text{SM}}, \Delta_{\text{SM}}, \varepsilon)$ -SM-secure, where $\delta_{\text{SM}} = \delta_{\text{CKA}}$, $\Delta_{\text{SM}} = \Delta_{\text{CKA}}$, and $\varepsilon \leq 2q_{\text{ep}}^2 \cdot (\varepsilon_{\text{cka}} + q \cdot \varepsilon_{\text{fs-aead}} + \varepsilon_{\text{p}})$.*

The proof of the above theorem follows the general ideas of [1], that builds a secure messaging scheme from CKA, FS-AEAD, and PRF-PRG, however, our proofs diverge from [1] in the sense that our reductions should properly handle (1) injections against the unauthenticated part of SM (which in practice are injections against the CKA transcript), (2) the messages of the “resend” protocol executed by parties when injections are detected, and (3) handle the fact that our CKA protocol requires more than two rounds to heal (as opposed to [1], whose definition captures only protocols that heal in two rounds), thus we introduce the parameter δ_{CKA} . Finally, in order to resist denial of service attacks (due to injections against the unauthenticated part of the transcript), we prove a stronger notion of correctness, in which parties successfully exchange messages in the presence of injections, and they can fully recover their state when tampering is detected, by retransmitting the unauthenticated part of the transcript.

Observe that, for any adversary respecting the safety predicates of the SM security definition, an execution of the SM security game with respect to **SM** will feed the PRF-RPG state with CKA keys that are indistinguishable from uniform, thus the FS-AEAD key output by the PRF-PRNG is a secure key. Therefore, the privacy and authenticity properties follow from the corresponding properties of FS-AEAD. In more detail, assuming the adversary targets epoch t^* , and parties heal from the last corruption at epoch t_{heal} , then, if $t_{\text{heal}} = t^*$, at epoch t^* a “secure” CKA key will feed the PRF-PRNG and by the PRG property of the primitive, the output FS-AEAD key will also be secure (even if the state of the primitive is known to the attacker from prior corruptions). If $t_{\text{heal}} < t^*$, security of the FS-AEAD key relies on the PRF security of PRF-PRNG. The full proof of the above theorem is presented below.

Proof. We prove the individual properties of our SM scheme. In particular, in Lemma 7, 8, and 6, we prove correctness, authenticity and privacy, respectively, of our scheme, and by combining them with Theorem 1, we derive the full proof of Theorem 5.

5.1.1 Correctness.

We now prove that **SM** satisfies the correctness property. This is formalized by the lemma below.

Lemma 7. *Assuming perfect correctness of FS-AEAD, **SM** is $(q, q_{\text{ep}}, 0)$ -correct.*

Proof. The correctness variant of the SM security game does not allow any challenges and provides reduced inject oracles that only allow the adversary to deliver compromised messages. The adversary is also allowed to deliver ciphertexts for which the unauthenticated part has not been generated honestly, since our definition captures a stronger notion of correctness in which the adversary is allowed to inject against the unauthenticated part of the ciphertext. We prove that the decryption operation always outputs the correct message (if deliver succeeds).

It’s not hard to see that correctness holds against adversaries that always deliver ciphertexts such that $c.ua$ has been generated by an honest sender, i.e., there is no injection against the unauthenticated part of the ciphertext. By inspection we observe that **A** and **B** compute the same CKA keys in each epoch, which implies that the PRF-PRNG state and output state are also the same. Thus, whenever **A** initializes a new (odd) epoch with a particular key, **B** will initialize the epoch on his end with the same key (and the other way around), and then the ciphertexts are processed by the right FS-AEAD instance. Therefore, assuming perfect correctness of the FS-AEAD scheme, **SM** is correct, assuming there are no injections against $c.ua$.

Assume the adversary tampers with the unauthenticated part of a ciphertext, say T , at epoch t for the first time, in a way such that T contributes to the CKA state and makes the parties compute different keys for some future epoch, say t^* , to which T contributes. Then by the tamper detection property of our CKA scheme, at epoch $t^* - 1$, the receive operation will output $d = \perp$, with overwhelming probability, since the parties compute different keys for epoch t^* . Then the parties will resend the unauthenticated data. Note that for epochs t up to t^* , the CKA protocol is using keys that are not affected by the tampered T , therefore, the PRF-PRNG and FS-AEAD states are consistent between the two parties, and by the FS-AEAD correctness and the fact that the authenticated part of the ciphertext remains intact (since we don’t allow injections), we derive that up to the point where the receive operation fails for one of the parties, correctness holds by the correctness of the FS-AEAD scheme. After failure, the resend protocol is initiated, and parties

exchange messages again using their most recent authenticated states, thus correctness holds due to the correctness of FS-AEAD. $\square$

5.1.2 Authenticity.

We now prove that SM satisfies the authenticity property. This is formalized in the below lemma.

Lemma 8. *Assume that*

- CKA is a $((\Delta_{\text{CKA}}, \delta_{\text{CKA}}), \varepsilon_{\text{cka}})$ -secure CKA scheme,
- FS-AEAD is a $(q, \varepsilon_{\text{fs-aead}})$ -secure FS-AEAD scheme, and
- P is a ε_{p} -secure PRF-PRNG.

Then, SM is $(q, q_{\text{ep}}, \delta_{\text{SM}}, \Delta_{\text{SM}}, \varepsilon)$ -authentic where $\Delta_{\text{SM}} = \Delta_{\text{CKA}}$, $\delta_{\text{SM}} = \delta_{\text{CKA}}$, and

$$\varepsilon \leq \varepsilon_{\text{cka}} + q \cdot \varepsilon_{\text{fs-aead}} + \varepsilon_{\text{p}}.$$

Proof. Let $\mathcal{A}$ be any adversary against the authenticity property of SM. $\mathcal{A}$ is required to initially specify (t_{L}^*, t^*) , i.e., the epoch of the last corruption and the epoch that it will try to inject against, respectively. Without loss of generality we assume that t^* is even, that is, in the epoch being attacked, B acts as sender and A as receiver. By definition, the authenticity game provides no challenge oracles and inject oracles that are *reduced* (as defined in Section 2.4.1), except for ciphertexts corresponding to epoch t^* . Moreover, if at any point during the execution of the game it happens to be that $t_{\text{L}} \in \{t_{\text{L}}^* + 1, \dots, t^* - 1\}$, i.e., if $\mathcal{A}$ does violates its commitment w.r.t. the last corruption epoch, or if A (the receiver in epoch t^*) is corrupted any time once $t_{\text{A}} > t_{\text{L}}^*$, the attacker loses the game immediately.

In the following, let t_{heal} be the epoch between t_{L}^* and t^* in which parties heal after corruption. By definition $t_{\text{heal}} = t_{\text{L}}^* + \delta_{\text{CKA}}$, i.e., the protocols state heals when the CKA state heals from the last corruption. The proof proceeds via a sequence of hybrid experiments H_0 , H_1 , and H_2 . The indistinguishability of H_0 and H_1 follows from the security of the CKA scheme, that of H_1 and H_2 from the security of the PRF-PRNG, and finally we bound the probability of winning in H_2 by the probability of breaking the FS-AEAD security of FS-AEAD.

Hybrid H_0 . This is the original authenticity game w.r.t. SM.

Hybrid H_1 . Let $I_1, I_2, \dots$, be the CKA keys computed by A and B during the execution of the protocol. H_1 works as H_0 , with the only difference being that the CKA key $I_{t_{\text{heal}}}$ is replaced by a value chosen uniformly at random. The indistinguishability between the hybrids follows from the security of CKA. We formalize this in the following lemma.

Lemma 9. *There exists an attacker $\mathcal{B}_1$ such that*

$$\text{Adv}_{\text{H}_0}^{\text{SM}}(\mathcal{A}) \leq \text{Adv}_{\text{H}_1}^{\text{SM}}(\mathcal{A}) + \text{Adv}_{(\Delta_{\text{CKA}}, \delta_{\text{CKA}})}^{\text{CKA}}(\mathcal{B}_1).$$

Proof. For simplicity we assume that t_{heal} is even, i.e., the first “good” key is generated by B. $\mathcal{B}_1$ plays against the CKA security game and internally runs $\mathcal{A}$ and simulates the authenticity game for $\mathcal{A}$. The simulation is along the lines of [1], however special care is required in order to handle the case in which the adversary tampers with the unauthenticated part of the transmitted ciphertexts. $\mathcal{B}_1$ executes as follows:

- **init**(t_L^*, t^*): $\mathcal{B}_1$ simply makes a call to its own CKA oracle **Init**(t_{heal}) and simulates initialization for both parties as in H_0 , except that the CKA initialization procedures are now called internally by the CKA security game.
- **transmit-A**(m, r): $\mathcal{B}_1$ simulates the oracle as in H_0 , but, whenever a new epoch is created, and if messages are not generated via ***resend**, it creates the corresponding CKA messages and keys by calling the corresponding oracle of the CKA game: to create (T_t, I_t) it calls **Send-A**. If the call fails (due to the **req** condition of **Send-A**), $\mathcal{B}_1$ aborts.
- **transmit-B**(m, r): $\mathcal{B}_1$ simulates as in H_0 , but, whenever needed, it creates CKA messages and keys (T_t, I_t) by calling **Chall-B** if $t = t_{\text{heal}}$, and **Send-B** otherwise. If the call fails (due to the **req** condition), $\mathcal{B}_1$ aborts. If $r \neq \perp$ and $t = t_{\text{heal}}$, $\mathcal{B}_1$ also aborts. For messages generated via ***resend**, simulation proceeds as with **transmit-A**, i.e., as in H_0 .
- **deliver-A**(c) and **inject-A**(c): for **deliver-A**(c) if the unauthenticated part of c matches the one output by B, $\mathcal{B}_1$ simulates by calling **Receive-A**, otherwise, it calls **Inject-A**. In both cases it advances the CKA game and uses the CKA keys I_t already returned by the CKA game when $\mathcal{B}_1$ was handling a previous transmit query for B. For calls to **inject-A**(c), $\mathcal{B}_1$ simulates as in H_0 .
- **corr-A**: $\mathcal{B}_1$ simulates as in H_0 , but uses the corruption oracle **Corr-A** of the CKA game to get the state γ^A ; if the call fails (due to the **req** condition of **Corr-A**), $\mathcal{B}_1$ aborts.

The oracles for B that are not covered above are similar. Finally, as soon as a party reaches epoch $t_{\text{heal}} + \Delta_{\text{CKA}}$, $\mathcal{B}_1$ issues a corrupt query on the CKA security game to obtain its state, which is used to complete the simulation.

We now show that the simulation is perfect. By inspection we observe that whenever the bit inside the CKA game is $b = 0$, $\mathcal{B}_1$ simulates H_0 , while whenever the bit is 1, $\mathcal{B}_1$ simulates H_1 , until $\mathcal{A}$ either wins or loses the game. Furthermore, $\mathcal{B}_1$ consistently calls the CKA challenge oracle **Chall-B** in epoch t_{heal} , and by requiring that $t^* \geq t_{\text{heal}} + \Delta_{\text{CKA}}$, injections are allowed only after the CKA security game has ended and $\mathcal{B}_1$ uses the leaked CKA states to simulate the remaining queries. With respect to bad randomness, if the adversary uses $r \neq \perp$ to send a message for epoch $t \in \{t_L^* + 1, \dots, t_{\text{heal}} + \Delta_{\text{CKA}}\}$, then either t_L is set to a value in $\{t_L^* + 1, \dots, t^* - 1\}$, or t_L is set to t^* but before B sends any epoch- t^* messages, i.e., all epoch- t^* ciphertexts will be compromised and the attacker loses the authenticity game. I.e., simulation fails only when $\mathcal{A}$ would trivially lose the authenticity game. $\square$

Hybrid H_2 . H_2 works as H_1 , but the FS-AEAD keys output by the PRF-PRNG at epoch t^* are replaced by uniformly random values in the key space. We formally prove the below.

Lemma 10. *There exists an attacker $\mathcal{B}_2$ such that*

$$\text{Adv}_{H_1}^{\text{SM}}(\mathcal{A}) \leq \text{Adv}_{H_2}^{\text{SM}}(\mathcal{A}) + \text{Adv}_{\text{PP}}^{\text{P}}(\mathcal{B}_2).$$

Proof. The attacker $\mathcal{B}_2$ is playing against the PRF-PRNG security game and internally runs $\mathcal{A}$, and simulates the authenticity game for $\mathcal{A}$. $\mathcal{B}_2$ is defined as follows.

- **init**(t_L^*, t^*): $\mathcal{B}_2$ executes the initialization for both parties as in H_2 , omitting the PRF-PRNG initialization, which is handled by the PRF-PRNG security game.
- **transmit-A**(m, r): $\mathcal{B}_2$ executes as in H_2 and only makes calls to the PRF-PRNG security game oracles if needed for the creation of new FS-AEAD sessions. Let I be the input to PRF-PRNG as defined by the hybrid execution. We have that,
 - For $t = t_{\text{heal}}, t \neq t^*$, **process** is called without any input.
 - For $t = t_{\text{heal}}, t = t^*$, **chall-prng** is called without any argument.
 - For $t \neq t_{\text{heal}}, t = t^*$, **chall-prf**(I) is called.
 - Otherwise, **process**(I) is called.

For messages generated via ***resend** no call to the PRF-PRNG game is required, since they are using existing FS-AEAD keys. If **chall-prng** or **chall-prf** fail (due to the **req** condition), $\mathcal{B}_2$ aborts.

- **deliver-A**(c): $\mathcal{B}_2$ executes as in H_2 using the relevant FS-AEAD key derived from the PRF-PRNG game.
- **inject-A**(c): $\mathcal{B}_2$ executes as in H_2 , but when an FS-AEAD key for epoch t^* is needed, it creates it similarly to the transmit oracles above.
- **corr-A**: $\mathcal{B}_2$ simulates as in H_2 , but makes a call to the corruption oracle of the PRF-PRNG game to recover the state σ . If $\neg\text{prf}$ doesn't hold, $\mathcal{B}_2$ aborts.
- **corr-B**: $\mathcal{B}_2$ executes as in **corr-A**, but if $\neg\text{prf}$ is false, it proceeds as follows: if B is in some epoch $t \in \{t^* - \delta_{\text{CKA}}, \dots, t^* - 1\}$, it aborts; if B is in epoch t^* or later, use σ_t^* mentioned above or whatever state it has evolved to.

The oracles corresponding to B (that are not defined above) work similarly. It follows by inspection that whenever the bit inside the PRF-PRNG game is $b = i$ (for $i \in \{1, 2\}$), $\mathcal{B}_2$ simulates H_i , up to the point where $\mathcal{A}$ either wins or loses. In particular, the transmit oracles fail if **chall-prng** or **chall-prf**(I) cannot be called due to the **req** condition, which, implies that $t_L \in \{t_L^* + 1, \dots, t^* - 1\}$. Failures w.r.t. **corr-A** and **corr-B** are similar. $\square$

The reduction to the FS-AEAD security. By the definition of H_2 , all the FS-AEAD keys are generated uniformly at random at epoch $t = t^*$. Therefore, one can define hybrids, such that in the i -th hybrid (for $t = t^*$), injecting against the first i FS-AEAD sessions results into the FS-AEAD decryption outputting $\perp$. Clearly, each two successive hybrids are close by a straight-forward reduction to the FS-AEAD security. This is formalized in the following lemma.

Lemma 11. *There exists an attacker $\mathcal{B}_3$ such that*

$$\text{Adv}_{H_2}^{\text{SM}}(\mathcal{A}) \leq q \cdot \text{Adv}_{\text{fs-aead}}^{\text{FS-AEAD}}(\mathcal{B}_3).$$

The proof is via a straightforward reductions to the FS-AEAD security similar to the proof of [Lem. 19](#), and therefore omitted. $\square$

5.1.3 Privacy

We now prove that SM satisfies the privacy property. This is formalized in the below theorem.

Theorem 6. *Assume that*

- CKA is a $((\Delta_{\text{CKA}}, \delta_{\text{CKA}}), \varepsilon_{\text{cka}})$ -secure CKA scheme,
- FS-AEAD is a $(q, \varepsilon_{\text{fs-aead}})$ -secure FS-AEAD scheme, and
- P is a ε_{p} -secure PRF-PRNG.

Then, SM is $(q, q_{\text{ep}}, \delta_{\text{SM}}, \Delta_{\text{SM}}, \varepsilon)$ -private for $\Delta_{\text{SM}} = \Delta_{\text{CKA}}$, $\delta_{\text{SM}} = \delta_{\text{CKA}}$, and

$$\varepsilon \leq \varepsilon_{\text{cka}} + q \cdot \varepsilon_{\text{fs-aead}} + \varepsilon_{\text{p}}.$$

Let $\mathcal{A}$ be any PPT adversary against privacy. Recall that $\mathcal{A}$ is required to specify values (t_{L}^*, t^*) , where w.l.o.g. t^* is assumed to be *even* in the following. That is, in the epoch being attacked, B acts as sender and A as receiver. Recall further that the privacy game provides only *reduced* inject oracles and that the challenge oracles only work in epoch t^* . Moreover, if at any point $t_{\text{L}} \in \{t_{\text{L}}^* + 1, \dots, t^* - 1\}$, the attacker loses the game immediately (i.e., the outcome of the game is a uniformly random bit). In the following, let $t_{\text{heal}} = t_{\text{L}}^* + \delta_{\text{CKA}}$.

The proof proceeds in hybrids $\text{H}_0, \text{H}_1, \text{H}_2$, similar to the authenticity proof. As with authenticity, the indistinguishability of H_0 and H_1 follows from the security of the CKA scheme, that of H_1 and H_2 from the security of the PRF-PRNG, and there is a reduction from winning the FS-AEAD security game to winning in H_2 .

Hybrid H_0 . This is the original privacy game w.r.t. SM.

Hybrid H_1 . Let $I_1, I_2, \dots$ be the CKA keys computed by A and B during the execution of the protocol. H_1 works as H_0 , with the only difference being that the CKA key $I_{t_{\text{heal}}}$ is replaced by a value chosen uniformly at random. The indistinguishability between the hybrids follows from the security of CKA. We formalize this in the following lemma.

Lemma 12. *There exists an attacker $\mathcal{B}_1$ such that*

$$\text{Adv}_{\text{H}_0}^{\text{SM}}(\mathcal{A}) \leq \text{Adv}_{\text{H}_1}^{\text{SM}}(\mathcal{A}) + \text{Adv}_{(\Delta_{\text{CKA}}, \delta_{\text{CKA}})}^{\text{CKA}}(\mathcal{B}_1).$$

The proof of the above lemma is identical to the proof of [Lem. 9](#).

Hybrid H_2 . H_2 works as H_1 , but the FS-AEAD keys output by the PRF-PRNG at epoch t^* are replaced by uniformly random values in the key space. We formally prove the below.

Lemma 13. *There exists an attacker $\mathcal{B}_2$ such that*

$$\text{Adv}_{\text{H}_1}^{\text{SM}}(\mathcal{A}) \leq \text{Adv}_{\text{H}_2}^{\text{SM}}(\mathcal{A}) + \text{Adv}_{\text{PP}}^{\text{P}}(\mathcal{B}_2).$$

The proof of the above lemma is identical to the proof of [Lem. 10](#).

By the definition of H_2 , all the FS-AEAD keys are generated uniformly at random at epoch $t = t^*$. Therefore, one can define hybrids, such that in the i -th hybrid (for $t = t^*$), attacking the first i FS-AEAD sessions is not a win for the adversary against privacy. Clearly, each two successive hybrids are close by a straight-forward reduction to the FS-AEAD security. This is formalized in the following lemma.

Lemma 14. *There exists an attacker $\mathcal{B}_3$ such that*

$$\text{Adv}_{H_2}^{\text{SM}}(\mathcal{A}) \leq q \cdot \text{Adv}_{\text{fs-aead}}^{\text{FS-AEAD}}(\mathcal{B}_3).$$

Proof. We define an adversary $\mathcal{B}_3^i$ that plays against the privacy property of the FS-AEAD security game and simulates H_3^i . $\mathcal{B}_3^i$ executes H_3^i and diverges only as follows: when the adversary issues a **transmit-A** or **transmit-B** operation that creates the epoch $t = t^*$ $\mathcal{B}_3^i$ handles queries by $\mathcal{A}$ using the oracles of the FS-AEAD security game. In particular, on **transmit-A**(m), **transmit-B**(m), **deliver-A**(c), **deliver-B**(c), it forwards the query to the FS-AEAD security game normally and on **chall-A**(m_0, m_1) (assuming that for simplicity $\mathcal{A}$ is challenging a message sent by A), it queries the FS-AEAD game with **chall-A**(a, m_0, m_1), where a is the associated data computed according to H_3^i . $\mathcal{B}_3^i$ uses the ciphertext returned by FS-AEAD game as the output of the **chall-A** operation. Clearly, if in the FS-AEAD game $b = 0$, then $\mathcal{B}_3^i$ simulates H_3^i with $b = 0$, otherwise, it simulates H_3^i with $b = 1$, and the indistinguishability between the hybrids follows from the privacy of the FS-AEAD scheme. By applying the same argument for $i \in [q]$, the proof of the lemma is concluded. $\square$

By combining Theorems [7](#), [8](#), and [6](#) with Theorem [1](#), the proof of [Theorem 5](#) is concluded. $\square$

5.2 Subversion Resilience of the SM Scheme

Since CKA is the only non-deterministic component in our SM construction (PRF-PRNGs and FS-AEAD are deterministic), in order for our SM scheme to be subversion resilient, it requires the underlying CKA scheme to also be subversion resilient. It follows that the firewall of the final SM construction simply implements the CKA firewall, i.e., it only sanitizes/mauls the CKA transcript as specified by its firewall, while during retransmission of unauthenticated CKA messages (if tampering is detected), the firewall enforces that parties retransmit the same messages. Recall that the adversary could potentially inject against the unauthenticated part of the SM transcripts (which corresponds to the CKA transcript) in order trigger the “resend” protocol. In this setting, a subverted party, say A, being in advance aware of B’s responses to the CKA protocol, could potentially diverge, and, instead of resending the same CKA messages (as the protocol specifies), she modifies her transcript in order to mount a rejection sampling attack against the CKA key (i.e., continuously trigger the “resend” protocol and choose her CKA messages until the CKA key is fixed to a specific value). The CKA firewall prevents this attack, since during “resend” it enforces parties to retransmit the CKA messages sent by the parties before the initiation of the “resend” protocol. Given, the above and the fact that the CKA output feeds the PRF-PRNG, where the latter is deterministic, it follows that the PRF-PRNG output follows the right distribution.

Another attack vector would be for the adversary to exfiltrate information by creating a channel between A and B using the FS-AEAD layer, e.g., by encrypting arbitrary messages related to the parties’ private state, however the uniqueness property of encryption (requiring

that, for every message m , associated data ad and key sk , there is exactly one ciphertext such that $\text{Dec}(\text{sk}, c, \text{ad}) = m$ together with the fact that subverted parties should be functionality maintaining, prevents the creation of this type of channel: if a subverted client encrypts a message m' other than the message m that the sender wishes to send, due to the uniqueness property, the output of decryption would output m' , and this breaks functionality. Note that the fact that we require subverted parties to be functionality maintaining, prohibits subverted parties from selectively creating valid/invalid ciphertexts in order to trigger selective decryption failures that create a channel between the parties. Given the above we derive the following theorem whose full proof is presented next.

Theorem 7. *Let CKA be a subversion resilient CKA scheme with reverse firewall W_{CKA} ,²⁰ FS-AEAD be an FS-AEAD scheme with unique ciphertexts, and let $W_{\text{SM}} := W_{\text{CKA}}$. Then, W_{SM} is functionality maintaining for the scheme SM of Fig. 13, preserves security for A and B, and is exfiltration resistant for A against B and for B against A, in SM.*

Proof. Since the firewall for SM is simply the firewall of CKA, W_{CKA} , and W_{CKA} is functionality maintaining for CKA, we derive that $W_{\text{SM}} := W_{\text{CKA}}$ is functionality maintaining for SM. We now proceed to prove that W_{SM} preserves the security (i.e., authenticity and privacy) of SM.

5.2.1 Authenticity

We now prove that SM satisfies the authenticity property in the presence of functionality maintaining and explainable subversions. This is formalized in the below theorem.

Lemma 15. *Assume that*

- CKA is a $((\Delta_{\text{CKA}}, \delta_{\text{CKA}}), \varepsilon_{\text{cka}})$ -secure subversion-resilient CKA scheme,
- FS-AEAD is a $(q, \varepsilon_{\text{fs-aead}})$ -secure FS-AEAD scheme with unique ciphertexts, and
- P is a ε_{p} -secure PRF-PRNG.

Then, for $P \in \{A, B\}$, $\text{SM}_{P \rightarrow W_{\text{SM}} \circ \tilde{P}}$ is $(q, q_{\text{ep}}, \delta_{\text{SM}}, \Delta_{\text{SM}}, \varepsilon)$ -authentic where, $\Delta_{\text{SM}} = \Delta_{\text{CKA}}$, $\delta_{\text{SM}} = \delta_{\text{CKA}}$, and

$$\varepsilon \leq \varepsilon_{\text{cka}} + q \cdot \varepsilon_{\text{fs-aead}} + \varepsilon_{\text{p}}.$$

Proof. We prove the needed for $P := A$. The case of B is symmetric. Let $\tilde{A}$ be a functionality maintaining and explainable subverted implementation of A and $\mathcal{A}$ be any adversary against the authenticity property of SM. We need to prove that authenticity holds against $\mathcal{A}$ if we substitute the code of A with $\tilde{A}$, in the presence of W_{SM} .

Recall that $\mathcal{A}$ is required to initially specify (t_{L}^*, t^*) , i.e., the epoch of the last corruption and the epoch that it will try to inject against, respectively. Without loss of generality we assume that t^* is even, that is, in the epoch being attacked, B acts as sender and A as receiver. By definition, the authenticity game provides no challenge oracles and inject oracles that are *reduced* (as defined in Section 2.4.1), except for ciphertexts corresponding to epoch t^* . Moreover, if at any point during the execution of the game it happens to be that $t_{\text{L}} \in \{t_{\text{L}}^* + 1, \dots, t^* - 1\}$, i.e., if $\mathcal{A}$ does violates its

²⁰I.e., the firewall is functionality maintaining for CKA, preserves security and provides exfiltration resistance to parties.

commitment w.r.t. the last corruption epoch, or if A (the receiver in epoch t^*) is corrupted any time once $t_A > t_L^*$, the attacker loses the game immediately.

In the following, let t_{heal} be the epoch between t_L^* and t^* in which parties heal after corruption when both A and B are not subverted. By definition $t_{\text{heal}} = t_L^* + \delta_{\text{CKA}}$, i.e., the protocols state heals when the CKA state heals from the last corruption. We prove that even the code of A is substituted with a functionality maintaining and explainable $\tilde{A}$, the protocol would still heal in t_{heal} .

The proof proceeds via a sequence of hybrid experiments.

Hybrid H_0 . This is the original authenticity game w.r.t. $\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}$ and arbitrary $\mathcal{A}$.

Hybrid H_1 . Let $I_1, I_2, \dots$, be the CKA keys computed by A and B during the execution of the protocol. Since $\tilde{A}$ is functionality maintaining and explainable, the I_i are consistent for A, B . The current hybrid on a **transmit-A** query by $\mathcal{A}$ with input message m , internally executes the honest **Send** operation for A to compute the FS-AEAD ciphertext and if this is not equal with the ciphertext output by $\tilde{A}$, H_1 aborts and outputs $\perp$. Assuming $\tilde{A}$ is functionality maintaining and explainable, i.e., always aims at delivering the right message, and its transcript is explainable, i.e., generates valid ciphertexts, and the uniqueness property of FS-AEAD, the current hybrid is indistinguishable from the previous one. This is formalized in the following lemma.

Lemma 16. *Assuming $\tilde{A}$ is functionality maintaining and explainable, there exists an attacker $\mathcal{B}_1$ such that*

$$\text{Adv}_{H_0}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_1}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) + \text{Adv}_{\text{UE}}^{\text{FS-AEAD}}(\mathcal{B}_1).$$

Proof. We define an adversary $\mathcal{B}_1$ that simulates H_0 against $\mathcal{A}$ while playing against the uniqueness property of the encryption scheme FS-AEAD. $\mathcal{B}_1$ is defined as follows: it simulates H_0 until the point in which $\tilde{A}$ outputs a ciphertext $(h', e') \neq (h, e)$, where (h, e) is the ciphertext that should have been generated by an honest execution of **Send**. $\mathcal{B}_1$ outputs $((h, e), (h', e'), m, v[t])$, as an attack against the uniqueness property of FS-AEAD, where m is the input message to the **transmit-A** oracle, and $v[t]$ is the FS-AEAD state that will be used for decryption by honest B . Assuming $\tilde{A}$ is functionality maintaining and explainable, we should have that $\text{FS-Rcv}(v[t], h', e') = \text{FS-Rcv}(v[t], h, e) \neq \perp$, and $\mathcal{B}_1$ breaks the uniqueness property of the encryption scheme. $\square$

Hybrid H_2 . Let $I_1, I_2, \dots$ be the CKA keys computed by A and B during the execution of the protocol. H_2 works as H_1 , with the only difference being that the CKA key $I_{t_{\text{heal}}}$ is replaced by a value chosen uniformly at random.

Let A^* (resp. $\tilde{A}^*$) be the “truncation” of A (resp. $\tilde{A}$), to the CKA protocol. The indistinguishability between the hybrids follows from the security of $\text{CKA}_{A^* \rightarrow W_{\text{CKA}} \circ \tilde{A}^*}$. We formalize this in the following lemma.

Lemma 17. *There exists an attacker $\mathcal{B}_2$ such that*

$$\text{Adv}_{H_1}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_2}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) + \text{Adv}_{(\Delta_{\text{CKA}}, \delta_{\text{CKA}})}^{\text{CKA}_{A^* \rightarrow W_{\text{CKA}} \circ \tilde{A}^*}}(\mathcal{B}_2).$$

Proof. For simplicity we assume that t_{heal} is even, i.e., the first “good” key is generated by B. $\mathcal{B}_2$ plays against the CKA security game with respect to $\text{CKA}_{A^* \rightarrow W_{\text{CKA}} \circ \tilde{A}^*}$ and internally runs $\mathcal{A}$ and simulates the authenticity game for $\mathcal{A}$. $\mathcal{B}_2$ executes as follows:

- **init**(t_L^*, t^*): $\mathcal{B}_2$ simply makes a call to its own CKA oracle **Init**(t_{heal}) and simulates initialization for both parties as in H_1 , except that the CKA initialization procedures are now called internally by the CKA security game.
- **transmit-A**(m, r): $\mathcal{B}_2$ simulates the oracle as in H_1 , but, whenever a new epoch is created, and if messages are not generated via ***resend**, it creates the corresponding CKA messages and keys by calling the corresponding oracle of the CKA game: to create (T_t, I_t) it calls **Send-A**, which, instead of calling the original **Send-A** operation, it executes the operation as defined by the code of $\tilde{A}^*$. If the call fails (due to the **req** condition of **Send-A**), $\mathcal{B}_2$ aborts. Subsequently, $\mathcal{B}_2$ calls $\tilde{A}$ to retrieve the output of **transmit-A**. Here the CKA part of $\tilde{A}$ ’s and $\tilde{A}^*$, match (i.e., they are executed with the same random tape).
- **transmit-B**(m, r): $\mathcal{B}_2$ simulates as in H_1 , but, whenever needed, it creates CKA messages and keys (T_t, I_t) by calling **Chall-B** if $t = t_{\text{heal}}$, and **Send-B** otherwise. If the call fails (due to the **req** condition), $\mathcal{B}_2$ aborts. If $r \neq \perp$ and $t = t_{\text{heal}}$, $\mathcal{B}_2$ also aborts. For messages generated via ***resend**, simulation proceeds as with **transmit-A**, i.e., as in H_1 .
- **deliver-A**(c) and **inject-A**(c): for **deliver-A**(c) if the unauthenticated part of c matches the one output by B, $\mathcal{B}_2$ simulates by calling **Receive-A**, otherwise, it calls **Inject-A**. In both cases it advances the CKA game which executes the code of $\tilde{P}^*$ and uses the CKA keys I_t already returned by the CKA game when $\mathcal{B}_2$ was handling a previous transmit query for B. For calls to **inject-A**(c), $\mathcal{B}_2$ simulates as in H_1 .
- **corr-A**: $\mathcal{B}_2$ simulates as in H_1 , but uses the corruption oracle **Corr-A** of the CKA game to get the state γ^A ; if the call fails (due to the **req** condition of **Corr-A**), $\mathcal{B}_2$ aborts.

The oracles for B that are not covered above are similar. Finally, as soon as a party reaches epoch $t_{\text{heal}} + \Delta_{\text{CKA}}$ ($\Delta_{\text{CKA}} = 0$ in our protocols), $\mathcal{B}_2$ issues a corrupt query on the CKA security game to obtain its state, which is used to complete the simulation. We now show that the simulation is perfect. By inspection we observe that whenever the bit inside the CKA game is $b = 0$, $\mathcal{B}_2$ simulates H_1 , while whenever the bit is 1, $\mathcal{B}_2$ simulates H_2 , until $\mathcal{A}$ either wins or loses the game. Furthermore, $\mathcal{B}_2$ consistently calls the CKA challenge oracle **Chall-B** in epoch t_{heal} , and by requiring that $t^* \geq t_{\text{heal}} + \Delta_{\text{CKA}}$, injections are allowed only after the CKA security game has ended and $\mathcal{B}_1$ uses the leaked CKA states to simulate the remaining queries. With respect to bad randomness, if the adversary uses $r \neq \perp$ to send a message for epoch $t \in \{t_L^* + 1, \dots, t_{\text{heal}} + \Delta_{\text{CKA}}\}$, then either t_L is set to a value in $\{t_L^* + 1, \dots, t^* - 1\}$, or t_L is set to t^* but before B sends any epoch- t^* messages, i.e., all epoch- t^* ciphertexts will be compromised and the attacker loses the authenticity game. I.e., simulation fails only when $\mathcal{A}$ would trivially lose the authenticity game. $\square$

Hybrid H_3 . H_3 works as H_2 , but the FS-AEAD keys output by the PRF-PRNG for honest B at epoch t^* are replaced by uniformly random values in the key space. We formally prove the below.

Lemma 18. *There exists an attacker $\mathcal{B}_3$ such that*

$$\text{Adv}_{H_2}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_3}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) + \text{Adv}_{\text{PP}}^{\text{P}}(\mathcal{B}_3).$$

Proof. The attacker $\mathcal{B}_3$ is playing against the PRF-PRNG security game and internally runs $\mathcal{A}$, and simulates the authenticity game for $\mathcal{A}$ with respect to $\text{SM}_{\text{P} \rightarrow \text{W}_{\text{SM}} \circ \tilde{\text{P}}}$. $\mathcal{B}_3$ is defined as follows.

- **init**(t_{L}^*, t^*): $\mathcal{B}_3$ executes the initialization for both parties as in H_3 , omitting the PRF-PRNG initialization, which is handled by the PRF-PRNG security game.
- **transmit-A**(m, r): $\mathcal{B}_3$ executes as in H_3 and only makes calls to the PRF-PRNG security game oracles if needed for the creation of new FS-AEAD sessions. Let I be the input to PRF-PRNG as defined by the hybrid execution. We have that,
 - For $t = t_{\text{heal}}$, $t \neq t^*$, **process** is called without any input.
 - For $t = t_{\text{heal}}$, $t = t^*$, **chall-prng** is called without any argument.
 - For $t \neq t_{\text{heal}}$, $t = t^*$, **chall-prf**(I) is called.
 - Otherwise, **process**(I) is called.

For a subverted A , $\tilde{\text{P}}$, we already know that she will never output a ciphertext different than the one that the honest **Send** operation would output, therefore, $\mathcal{B}_3$ uses the PRF-PRNG output to fully simulate the FS-AEAD ciphertext for $\tilde{\text{P}}$. For messages generated via ***resend** no call to the PRF-PRNG game is required, since they are using existing FS-AEAD keys. If **chall-prng** or **chall-prf** fail (due to the **req** condition), $\mathcal{B}_3$ aborts.

- **deliver-A**(c): $\mathcal{B}_3$ executes as in H_2 using the relevant FS-AEAD key derived from the PRF-PRNG game.
- **inject-A**(c): $\mathcal{B}_3$ executes as in H_2 , but when an FS-AEAD key for epoch t^* is needed, it creates it similarly to the transmit oracles above.
- **corr-A**: $\mathcal{B}_3$ simulates as in H_2 , but makes a call to the corruption oracle of the PRF-PRNG game to recover the state σ . If $\neg \text{prf}$ doesn't hold, $\mathcal{B}_3$ aborts.
- **corr-B**: $\mathcal{B}_3$ executes as in **corr-A**, but if $\neg \text{prf}$ is false, it proceeds as follows: if B is in some epoch $t \in \{t^* - \delta_{\text{CKA}}, \dots, t^* - 1\}$, it aborts; if B is in epoch t^* or later, use σ_t^* mentioned above or whatever state it has evolved to.

The oracles corresponding to B (that are not defined above) work similarly. It follows by inspection that whenever the bit inside the PRF-PRNG game is $b = i$ (for $i \in \{1, 2\}$), $\mathcal{B}_3$ simulates H_i , up to the point where $\mathcal{A}$ either wins or loses. In particular, the transmit oracles fail if **chall-prng** or **chall-prf**(I) cannot be called due to the **req** condition, which, implies that $t_{\text{L}} \in \{t_{\text{L}}^* + 1, \dots, t^* - 1\}$. Failures w.r.t. **corr-A** and **corr-B** are similar. $\square$

Hybrid H_4 . By the definition of H_3 , all the FS-AEAD keys are generated uniformly at random at epoch $t = t^*$. Therefore, we define hybrids H_4^i , $i \in [q]$, such that in the i -th hybrid (for $t = t^*$), injecting against the first i FS-AEAD sessions always results into the FS-AEAD decryption outputting $\perp$. We set $\text{H}_4^q := \text{H}_4$. Clearly, each two successive hybrids are close by a straight-forward reduction to the FS-AEAD security. This is formalized in the following lemma.

Lemma 19. *There exists an attacker $\mathcal{B}_4$ such that*

$$\text{Adv}_{\mathcal{H}_3}^{\text{SM}_{\mathbf{A} \rightarrow \mathbf{W}_{\text{SM} \circ \tilde{\mathbf{A}}}}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{H}_4}^{\text{SM}_{\mathbf{A} \rightarrow \mathbf{W}_{\text{SM} \circ \tilde{\mathbf{A}}}}} + q \cdot \text{Adv}_{\text{fs-aead}}^{\text{FS-AEAD}}(\mathcal{B}_4).$$

Proof. We define an adversary $\mathcal{B}_4^i$ that plays against the FS-AEAD security game and simulates $\mathcal{H}_4^i$. $\mathcal{B}_4^i$ executes $\mathcal{H}_4^i$ and diverges only as follows: when the adversary issues a **transmit-A** or **transmit-B** operation that creates the i -th FS-AEAD session, $\mathcal{B}_4^i$ handles queries by $\mathcal{A}$ using the oracles of the FS-AEAD security game. In particular, on **transmit-A**(m), **transmit-B**(m), **deliver-A**(c), **deliver-B**(c), **inject-A**(c), **inject-B**(c), it forwards the query to the FS-AEAD security game. Clearly, the adversary cannot mount a successful injection in the presence of a trivial corruption, and hybrid $\mathcal{H}_4^i$ is distinguishable from $\mathcal{H}_4^{i-1}$ only if $\mathcal{A}$ makes the decryption operation of during the call to **inject-B**(c) to output a non- $\perp$ value, diverging from $\mathcal{H}_4^i$ that always outputs $\perp$ in that case. Clearly, this can only happen if $\mathcal{A}$ mounts a successful injection against the FS-AEAD security game.

By applying the same argument for $i \in [q]$, and the fact that in $\mathcal{H}_4^i$ the advantage of the adversary in breaking authenticity is zero, the proof of the lemma is concluded. $\square$

Hybrid $\mathcal{H}_5$. This is the authenticity game w.r.t. $\text{SM}_{\mathbf{A} \rightarrow \mathbf{W}_{\text{SM} \circ \mathbf{A}}}$ and arbitrary $\mathcal{A}$, i.e., this hybrid switches from the subverted $\tilde{\mathbf{P}}$ to the honest $\mathbf{A}$. The indistinguishability between the hybrids is based on the exfiltration resistance of property $\mathbf{W}_{\text{CKA}}$ and is formalized on the following lemma.

Lemma 20. *There exists an attacker $\mathcal{B}_5$ such that*

$$\text{Adv}_{\mathcal{H}_4}^{\text{SM}_{\mathbf{A} \rightarrow \mathbf{W}_{\text{SM} \circ \tilde{\mathbf{A}}}}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{H}_5}^{\text{SM}_{\mathbf{A} \rightarrow \mathbf{W}_{\text{SM} \circ \mathbf{A}}}}(\mathcal{A}) + \text{Adv}_{\text{ER}}^{\mathbf{W}_{\text{CKA}}}(\mathcal{B}_5).$$

Proof. We define an adversary $\mathcal{B}_5$ that plays against the exfiltration resistance property of $\mathbf{W}_{\text{CKA}}$, i.e., it plays against $\text{EX-RE}(\lambda, \mathbf{II}, \mathbf{A}, \mathbf{B}, \mathbf{W}_{\text{CKA}})$ ²¹ while simulating $\mathcal{H}_4$ for $\mathcal{A}$. For convenience we provide $\mathcal{B}_5$ with access to the CKA operations via oracles similar to those provided by the CKA security game. $\mathcal{B}_5$ is defined as follows:

- Calls EX-RE to set $\mathbf{A}'$, where $\mathbf{A}'$ is either $\mathbf{W}_{\text{CKA}} \circ \mathbf{A}$ or $\mathbf{W}_{\text{CKA}} \circ \tilde{\mathbf{A}}$, depending on the bit sampled by EX-RE. In the next steps it simulates $\text{SM}_{\mathbf{A} \rightarrow \mathbf{W}_{\text{SM} \circ \mathbf{A}'}}$.
- **init**(t_{L}^*, t^*): $\mathcal{B}_5$ simply makes a call to its own CKA oracle **Init**(t_{heal}) and simulates initialization for both parties as in $\mathcal{H}_4$, except that the CKA initialization procedures are now called internally by the CKA oracles.
- **transmit-A**(m, r): $\mathcal{B}_5$ simulates the oracle as in $\mathcal{H}_4$, but, whenever a new epoch is created, it creates the corresponding CKA messages and keys by calling the corresponding oracle of CKA: to create (T_t, I_t) it calls **Send-A**, which, instead of calling the original **Send-A** operation, it executes the operation as defined by $\mathbf{A}'$ (clearly if $b = 0$ the oracle computes as $\tilde{\mathbf{A}}^*$, otherwise computes as $\mathbf{A}$). Subsequently, $\mathcal{B}_5$ uses the output of $\mathbf{A}'$ to honestly execute the PRF-PRNG and the FS-AEAD ciphertext.

²¹The input X on that game is the X3DH key, while $S_{\mathbf{B}}$ is $\mathbf{B}$'s state after the protocol terminates. The state can be arbitrary and the only restriction is that $\mathbf{B}$ should be functionality maintaining and explainable during the protocol execution.

- **transmit-B**(m, r): similar to **transmit-A**.
- **deliver-A**(c) and **inject-A**(c): for **deliver-A**(c) if the unauthenticated part of c matches the one output by B , $\mathcal{B}_5$ simulates by calling **Receive-A**, otherwise, it calls **Inject-A**. In both cases it advances the CKA game which executes the code of A' and uses the CKA keys I_t already returned by the CKA game when $\mathcal{B}_5$ was handling a previous transmit query for B . For calls to **inject-A**(c), $\mathcal{B}_5$ simulates as in H_4 .

Observe that ciphertexts simulated by the $\mathcal{B}_5$ are always computed correctly given the CKA outputs, and this is consistent with the definitions of H_4 and H_5 (recall that due to the uniqueness property of the encryption scheme the adversary should always transmit the right ciphertext). Therefore, the FS-AEAD part of the transmitted ciphertext are not affected by the bit b chosen by the EX-RE. Furthermore, the CKA part of the transmitted messages is generated according to A' for A and according to $\tilde{B}$ for B . Therefore, if $b = 0$, $\mathcal{B}_5$ simulates $SM_{W_{CKA} \circ \tilde{A}, B}$, i.e., H_4 , while if $b = 1$, it simulates $SM_{W_{CKA} \circ A, B}$, i.e., H_5 , and the distinguishing advantage of the $\mathcal{A}$ against the two hybrids is bounded by the advantage of $\mathcal{B}_5$ against the exfiltration resistance of W_{CKA} . $\square$

The proof of the lemma is concluded. $\square$

5.2.2 Privacy

We now prove that SM satisfies the privacy property in the presence of functionality maintaining and explainable subversions. This is formalized in the lemma below.

Lemma 21. *Assume that*

- CKA is a $((\Delta_{CKA}, \delta_{CKA}), \varepsilon_{cka})$ -secure CKA scheme,
- FS-AEAD is a $(q, \varepsilon_{fs-aead})$ -secure FS-AEAD scheme, and
- P is a ε_P -secure PRF-PRNG.

Then, for $P \in \{A, B\}$, $SM_{P \rightarrow W_{SM} \circ \tilde{P}}$ is $(q, q_{ep}, \delta_{SM}, \Delta_{SM}, \varepsilon)$ -private for $\Delta_{SM} = \Delta_{CKA}$, $\delta_{SM} = \delta_{CKA}$, and

$$\varepsilon \leq \varepsilon_{cka} + q \cdot \varepsilon_{fs-aead} + \varepsilon_P.$$

Proof. We prove the needed for $P := A$. The case of B is symmetric. Let $\tilde{A}$ be a functionality maintaining and explainable subverted implementation of A and $\mathcal{A}$ be any adversary against the privacy property of SM . We need to prove that privacy holds against $\mathcal{A}$ if we substitute the code of A with $\tilde{A}$, in the presence of W_{SM} .

Let $\mathcal{A}$ be any PPT adversary against privacy. Recall that $\mathcal{A}$ is required to specify values (t_L^*, t^*) , where w.l.o.g. t^* is assumed to be *even* in the following. That is, in the epoch being attacked, B acts as sender and A as receiver. Recall further that the privacy game provides only *reduced* inject oracles and that the challenge oracles only work in epoch t^* . Moreover, if at any point $t_L \in \{t_L^* + 1, \dots, t^* - 1\}$, the attacker loses the game immediately (i.e., the outcome of the game is a uniformly random bit). In the following, let $t_{heal} = t_L^* + \delta_{CKA}$.

The proof proceeds in hybrids, similar to the authenticity proof.

Hybrid H_0 . This is the original authenticity game w.r.t. $SM_{A \rightarrow W_{SM} \circ \tilde{A}}$ and arbitrary $\mathcal{A}$.

Hybrid H₁. Let $I_1, I_2, \dots$ be the CKA keys computed by A and B during the execution of the protocol. Since $\tilde{A}$ is functionality maintaining and explainable, the I_i are consistent for A, B. The current hybrid on a **transmit-A** query by $\mathcal{A}$ with input message m , internally executes the honest **Send** operation for A to compute the FS-AEAD ciphertext and if this is not equal with the ciphertext output by $\tilde{A}$, H₁ aborts and outputs $\perp$. Assuming $\tilde{A}$ is functionality maintaining and explainable and the uniqueness property of FS-AEAD, the current hybrid is indistinguishable from the previous one. This is formalized in the following lemma.

Lemma 22. *Assuming $\tilde{A}$ is functionality maintaining and explainable, there exists an attacker $\mathcal{B}_1$ such that*

$$\text{Adv}_{H_0}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_1}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) + \text{Adv}_{\text{UE}}^{\text{FS-AEAD}}(\mathcal{B}_1).$$

The proof of the lemma is identical to the proof of [Lem. 16](#).

Hybrid H₂. Let $I_1, I_2, \dots$ be the CKA keys computed by A and B during the execution of the protocol. H₂ works as H₁, with the only difference being that the CKA key $I_{t_{\text{heal}}}$ is replaced by a value chosen uniformly at random.

Let A^* (resp. $\tilde{A}^*$) be the “truncation” of A (resp. $\tilde{A}$), to the CKA protocol. The indistinguishability between the hybrids follows from the security of $\text{CKA}_{A^* \rightarrow W_{\text{CKA}} \circ \tilde{A}^*}$. We formalize this in the following lemma.

Lemma 23. *There exists an attacker $\mathcal{B}_2$ such that*

$$\text{Adv}_{H_1}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_2}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) + \text{Adv}_{(\Delta_{\text{CKA}}, \delta_{\text{CKA}})}^{\text{CKA}_{A^* \rightarrow W_{\text{CKA}} \circ \tilde{A}^*}}(\mathcal{B}_2).$$

The proof of the lemma is identical to the proof of [Lem. 17](#).

Hybrid H₃. H₃ works as H₂, but the FS-AEAD keys output by the PRF-PRNG for honest B at epoch t^* are replaced by uniformly random values in the key space. We formally prove the below.

Lemma 24. *There exists an attacker $\mathcal{B}_3$ such that*

$$\text{Adv}_{H_2}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_3}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) + \text{Adv}_{\text{PP}}^{\text{P}}(\mathcal{B}_3).$$

The proof of the lemma is identical to the proof of [Lem. 18](#).

Hybrid H₄. By the definition of H₃, all the FS-AEAD keys are generated uniformly at random at epoch $t = t^*$. Therefore, we define hybrids H_4^i , $i \in [q]$, such that attacking the i -th hybrid (for $t = t^*$), is not a win against privacy. We set $H_4^q := H_4$. Clearly, each two successive hybrids are close by a straight-forward reduction to the FS-AEAD privacy. This is formalized in the following lemma.

Lemma 25. *There exists an attacker $\mathcal{B}_4$ such that*

$$\text{Adv}_{H_3}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_4}^{\text{SM}_{A \rightarrow W_{\text{SM}} \circ \tilde{A}}} + q \cdot \text{Adv}_{\text{fs-aead}}^{\text{FS-AEAD}}(\mathcal{B}_4).$$

Proof. We define an adversary $\mathcal{B}_4^i$ that plays against the privacy property of the FS-AEAD security game and simulates H_4^i . $\mathcal{B}_4^i$ executes H_4^i and diverges only as follows: when the adversary issues a **transmit-A** or **transmit-B** operation that creates the epoch $t = t^*$ $\mathcal{B}_4^i$ handles queries by $\mathcal{A}$ using the oracles of the FS-AEAD security game. In particular, on **transmit-A**(m), **transmit-B**(m), **deliver-A**(c), **deliver-B**(c), it forwards the query to the FS-AEAD security game normally and on **chall-A**(m_0, m_1) (assuming that for simplicity $\mathcal{A}$ is challenging a message sent by A), it queries the FS-AEAD game with **chall-A**(a, m_0, m_1), where a is the associated data computed according to H_4^i . $\mathcal{B}_4^i$ uses the ciphertext returned by FS-AEAD game as the output of the **chall-A** operation. Clearly, if in the FS-AEAD game $b = 0$, then $\mathcal{B}_4^i$ simulates H_4^i with $b = 0$, otherwise, it simulates H_4^i with $b = 1$, and the indistinguishability between the hybrids follows from the privacy of the FS-AEAD scheme.

By applying the same argument for $i \in [q]$, the proof of the lemma is concluded. $\square$

Hybrid H_5 . This is the privacy game w.r.t. $SM_{A \rightarrow W_{SM} \circ A}$ and arbitrary $\mathcal{A}$, i.e., this hybrid switches from the subverted $\tilde{P}$ to the honest A. The indistinguishability between the hybrids is based on the exfiltration resistance of property W_{CKA} and is formalized on the following lemma.

Lemma 26. *There exists an attacker $\mathcal{B}_5$ such that*

$$\text{Adv}_{H_4}^{SM_{A \rightarrow W_{SM} \circ \tilde{A}}}(\mathcal{A}) \leq \text{Adv}_{H_5}^{SM_{A \rightarrow W_{SM} \circ A}}(\mathcal{A}) + \text{Adv}_{ER}^{W_{CKA}}(\mathcal{B}_5).$$

Observe that $\text{Adv}_{H_5}^{SM_{A \rightarrow W_{SM} \circ A}}(\mathcal{A}) = 0$. The proof of the lemma is identical to the proof of [Lem. 20](#). $\square$

By combining [Lem. 15](#) and [Lem. 21](#) with [Theorem 1](#), the proof of [Theorem 7](#) is concluded. $\square$

6 Subversion-Resilient X3DH

Reverse firewalls require some level of interaction between the parties so it can consistently re-randomize the messages in a way that correctness is preserved and parties end up with the same session key. The novel component that we add to our version of X3DH (and that we call “firewall-friendly” X3DH) is that the protocol has one additional round of communication between the parties and the server. More specifically, the server now actively contributes a random factor to the ephemeral key of Alice and Bob. This is what makes the X3DH firewall-friendly, as it allows the protocol to work with a reverse-firewall for either Alice and/or Bob that is able to re-randomize the messages exchanged between the parties in a systematic and transparent way. Moreover, we note that the server *is not* required to be honest. As we show in our proof, a malicious server that samples $r \neq 0$ and $r' \neq 0$ is not able to bias the final session key. In case either r (resp. r') is equal to zero Alice (resp. Bob) can just ignore r (resp. r') and keep her ephemeral key as g^x (resp. g^y). The security relies on the Gap Diffie-Hellman assumption (in [2.3.3](#)), the inverse CDH assumption (in [2.3.2](#)), unforgeability of signature, and the security of the pseudo-random function.

Next, we give a formal description of the protocol (in [Fig. 14](#)) including parameters generation and building blocks, followed by a security proof. Later, in [Sec. 6.3](#) we present the firewalls for Alice and Bob and demonstrate that they guarantee functionality preservation, exfiltration resistance and security preservation.

Alice: $\text{lpk}_A := (\text{pk}_A, g^a)$ $\text{lsk}_A := (\text{sk}_A, a)$	Server	Bob: $\text{lpk}_B := (\text{pk}_B, g^b)$ $\text{lsk}_B := (\text{sk}_B, b)$
$x \xleftarrow{\$} \mathbb{Z}_p^*$ $\sigma_A \leftarrow \Sigma.\text{Sign}(\text{sk}_A, g^{xr})$ – go offline –	$\xrightarrow{g^x}$ $\xleftarrow{r}$ $r \xleftarrow{\$} \mathbb{Z}_p^* \setminus \{0\}$ $(g^{xr}, \sigma_A) \xrightarrow{\quad}$ $(A, (g^{xr}, \sigma_A))$ $r' \xleftarrow{\$} \mathbb{Z}_p^* \setminus \{0\}$ $(A, B, (g^{yr'}, \sigma_B))$	$y \xleftarrow{\$} \mathbb{Z}_p^*$ $\xleftarrow{A, g^y}$ $\xrightarrow{r'}$ $(g^{xr}, \sigma_A) \xrightarrow{\quad}$ $\text{req } \Sigma.\text{Ver}(\text{pk}_A, g^{xr}, \sigma_A)$ $\text{DH}_1 \leftarrow (g^{xr})^b$ $\text{DH}_2 \leftarrow (g^a)^{yr'}$ $\text{DH}_3 \leftarrow (g^{xr})^{yr'}$ $\text{DH} \leftarrow H(\text{DH}_1, \text{DH}_2, \text{DH}_3)$ $\text{sid}_B := A B g^a g^b g^{xr} g^{yr'}$ $k_B \leftarrow \text{PRF}_{\text{DH}}(\text{sid}_B)$ $\sigma_B \leftarrow \Sigma.\text{Sign}(\text{sk}_B, g^{yr'})$
– come online – $\text{req } \Sigma.\text{Ver}(\text{pk}_B, g^{yr'}, \sigma_B)$ $\text{DH}_1 \leftarrow (g^b)^{xr}$ $\text{DH}_2 \leftarrow (g^{yr'})^a$ $\text{DH}_3 \leftarrow (g^{yr'})^{xr}$ $\text{DH} \leftarrow H(\text{DH}_1, \text{DH}_2, \text{DH}_3)$ $\text{sid}_A := A B g^a g^b g^{xr} g^{yr'}$ $k_A \leftarrow \text{PRF}_{\text{DH}}(\text{sid}_A)$	$\xleftarrow{(g^{yr'}, \sigma_B)}$	

Building blocks. The protocol consists of the following building blocks. (1) $\Sigma = (\text{KeyGen}, \text{Sign}, \text{Ver})$ is a unique signature scheme²² that is EU-CMA secure and perfectly correct, (2) $\text{PRF} : \{0, 1\}^\lambda \times$

62

$\{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a pseudo-random function family with key-space $\{0, 1\}^\lambda$, and (3) $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ is a hash function modeled as a random oracle.

Public parameters. All the parties are provided with the public parameters at the beginning of the protocol. G is a cyclic group, g is a generator of G , q is the (prime) order of the group, and $\lambda = \log(q)$ is the security parameter.

Key generation. At the setup phase of the protocol Alice (resp. Bob) samples its long-term keys as follows: samples key pair for the signature scheme $(\text{ssk}_a, \text{spk}_a) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$, samples a random element $a \leftarrow \mathbb{Z}_p^*$, and finally sets its secret and public long-term keys as $\text{lsk}_A = (\text{ssk}_a, a)$ and $\text{lpk}_A = (\text{spk}_a, g^a)$. The public keys are shared with the other parties of the protocol.

Protocol execution. Alice starts by generating an ephemeral key g^x and sending it to the server. The server replies with a random nonzero value r , which Alice uses to update her key to g^{xr} . She then signs this updated key and uploads the signature to the server for Bob to retrieve before going offline. When she reconnects, Alice receives Bob's ephemeral key and its signature, verifies the signature, and computes the shared secret k_A . Specifically, she derives key material $\text{DH} := H((g^{xr})^b, (g^a)^{yr'}, (g^{xr})^{yr'})$ and then obtains k_A by applying a pseudorandom function PRF with DH as the key and the session identifier sid as the input. Bob's execution mirrors this process.

6.2 Security

We adopt the security model from [42], with a detailed description of the model provided in Sec. B. We first establish the protocol's correctness below (according to Definition 25), and subsequently prove its security within the same model.

Theorem 8. *Let the signature scheme Σ be perfectly correct. Then, the protocol of Fig. 14 is 1-correct.*

Proof. It is easy to see that Alice's and Bob's oracles become partners when they execute the protocol faithfully and the server honestly samples random elements in $\mathbb{Z}_p^*$. In that case, the values $\text{DH}_1, \text{DH}_2, \text{DH}_3$ will be identical for both Alice and Bob, and thus the final keys k_A and k_B will match. The probability of the final keys not matching or the protocol not finishing for a pair of oracles is bounded by the correctness error probability of the signature scheme δ_Σ . Since the signature scheme is perfectly correct, the protocol in Fig. 14 is also perfectly correct. $\square$

We now state the security of the protocol followed by a full proof.

Theorem 9. *Let H be a random oracle, and let qs denote the number of queries the adversary makes to H . For any PPT adversary $\mathcal{A}$ playing $G_{\Pi_{\text{AKE}}}^{\text{FS}}(\mu, \ell)$ with μ parties and at most ℓ sessions per party, there exist PPT algorithms that break the GDH assumption, the security of PRF, the CDH assumption, the inverse CDH assumption, and the unforgeability of Σ such that,*

$$\text{Adv}_{\Pi_{\text{AKE}}}^{\text{FS}}(\mathcal{A}) \leq \max \begin{cases} \mu^2 \ell^2 \cdot (qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}''), \\ \mu^2 \ell \cdot (\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}''), \\ \mu \cdot \text{Adv}_{\text{EU-CMA}}^{\Sigma}(\mathcal{B}) \end{cases}$$

Strategy	Tested	Partner?	Alice		Bob		Lemma
			long-term	ephemeral	long-term	ephemeral	
Type-1	Alice/Bob	Yes	✗	✓	✗	✓	lemma 27
Type-2	Alice/Bob	Yes	✗	✓	✓	✗	lemma 28
Type-3	Alice/Bob	Yes	✓	✗	✗	✓	lemma 28
Type-4	Alice/Bob	Yes	✓	✗	✓	✗	no security
Type-5	Alice	No	✗	✓	✓	-	lemma 29
Type-6	Alice	No	✓	✗	✓	-	lemma 29
Type-7	Bob	No	✓	-	✗	✓	lemma 29
Type-8	Bob	No	✓	-	✓	✗	lemma 29

Table 1: This table outlines the adversary’s strategy in a security game targeting a freshly tested oracle. The “Tested” column specifies the tested oracle’s role, which can be either the initiator (Alice), or the responder (Bob). The “Partner” column indicates whether the tested oracle is associated with a partner oracle during the game. Symbols ✓ and ✗ denote whether the long-term or ephemeral (i.e., **state**) key material remains secure or has been compromised by the adversary, respectively. In strategies Type-5 and Type-6 (respectively, Type-7 and Type-8), when Alice’s (respectively, Bob’s) oracle is being tested without a partner oracle, the ephemeral key material for Bob (respectively, Alice) is undefined, thus marked as “-”. The final column specifies the lemma that addresses the respective attack strategy.

Proof intuition. The proof follows the adversary’s possible strategies, summarized in Table 1, and considers three main cases. Type-1: Security relies on both parties ephemeral keys and is based on the CDH assumption, as the adversary cannot compute $\text{DH}_3 = (g^{x^r})^{y^{r'}}$ to query the random oracle. Types 2-3: Security depends on one party’s ephemeral key and the other’s long-term key, requiring the GDH assumption, since Bob’s (resp. Alice’s) long-term key g^b (resp. g^a) is used across sessions, which must be simulated without knowing b (resp. a). Types 5-8: The tested oracle has no partner, thus the security relies on the unforgeability of the signature scheme, as the adversary should not be able to forge a signature for a message that was not signed by the tested party’s partner. By combining these results, the proof is complete. We present the full proof below.

Proof. Our proof follows the same structure as the proof in [41]. Let $\mathcal{A}$ be a PPT adversary that plays the security game $G_{\Pi_{\text{AKE}}}^{\text{FS}}(\mu, \ell)$ with the challenger $\mathcal{C}$ and has advantage $\text{Adv}_{\Pi_{\text{AKE}}}^{\text{FS}}(\mathcal{A})$. To prove Theorem 9 we analyse all the relevant strategies that can be taken by the adversary $\mathcal{A}$. More specifically, $\mathcal{A}$ ’s strategy can be divided into eight different types of strategies listed in Table 1, where each strategy is mutually independent. For our X3DH protocol, strategies Type-2 and Type-3 are identical with the roles of Alice (i.e. initiator) and Bob (i.e. responder) reversed. Additionally, Type-5 and Type-6 (resp. Type-7 and Type-8) strategies can be analyzed together as the proof simply relies on the non-leakage of Bob’s (resp. Alice’s) long-term keys.

Although, in our protocol, Alice and Bob send their ephemeral keys to the server as part of the protocol, we define protocol messages in the security game as those intended for Alice and Bob, excluding the server. In all adversarial strategies we model the server as a potentially malicious entity capable of influencing the parties’ ephemeral keys. This approach accounts for the worst-case scenario, where the server can try to bias these keys to maximize the adversary’s advantage.

We note that our protocol does not have security in the Type-4 attack (as the original X3DH) since in the Type-4 strategy the ephemeral keys of both parties are leaked, and thus all values DH_1, DH_2 and DH_3 can be computed by the adversary.

For each strategy in Table 1 we construct an algorithm that breaks an underlying assumption of the protocol by using adversary $\mathcal{A}$ as a subroutine. Our proof is structured in a sequence of games. Without loss of generality we assume that $\mathcal{A}$ always issues a **Test** query. We denote by $\Pr[G_j]$ the probability that the adversary outputs $b' = b$ in game G_j (which is the winning condition of the game described in Sec. B) and we let $\text{Adv}_{G_j}^{\mathcal{A}} := |\Pr[G_j] - \frac{1}{2}|$ denote the advantage of the adversary in game G_j . For all strategies of $\mathcal{A}$ that we analyse the proofs share a common game sequence G_0 - G_1 that we describe next.

Game G_0 . This game is identical to the original security game. We thus have

$$\text{Adv}_{G_0}^{\mathcal{A}} = \text{Adv}_{\Pi_{\text{AKE}}}^{\text{FS}}(\mathcal{A}).$$

Game G_1 . This game is identical to G_0 except that the game aborts whenever there exist two partner oracles π_i^s and π_j^t that do not agree on the same session key. From Theorem 8, the probability of an error occurring is zero. Thus, we have

$$\Pr[G_0] = \Pr[G_1].$$

Adversary using strategy Type-1.

Lemma 27. *For any PPT adversary $\mathcal{A}$ using the Type-1 strategy, there exists PPT algorithms $\mathcal{B}$ breaking the CDH assumption and $\mathcal{B}'$ breaking the security of the PRF function such that*

$$\text{Adv}_{G_1}^{\mathcal{A}} \leq \mu^2 \ell^2 \cdot \left(\text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}') \right).$$

Proof. **Game G_2 .** This game is identical to the previous one, except that we introduce an abort condition. Let E_{coll} be the event that there exists an oracle with more than one partner oracle. If E_{coll} occurs, the game aborts. Since G_2 and G_3 are identical when E_{coll} does not occur, we obtain

$$|\text{Adv}_{G_1}^{\mathcal{A}} - \text{Adv}_{G_2}^{\mathcal{A}}| \leq \Pr[\text{E}_{\text{coll}}].$$

Next, we bound the probability of E_{coll} . Fix $i \in [\mu]$ and consider the set of oracles $S_i = \{\pi_j^t \mid \text{Pid}_j^t = i\}$. For any $\pi_j^t \in S_i$, if there exist two oracles π_i^s and $\pi_i^{s'}$ with $s \neq s'$ such that both are partners of π_j^t , then by definition, we must have

$$\text{sid}_j^t = \text{sid}_i^s = \text{sid}_i^{s'}.$$

Consider the case where π_j^t is an oracle for Bob (resp. Alice), and π_i^s and $\pi_i^{s'}$ are oracles for Alice (resp. Bob). Since the server is malicious and contributes to the ephemeral keys of both parties, we need to bound the probability that the server can force π_i^s and $\pi_i^{s'}$ to share the same ephemeral key, thereby triggering E_{coll} .

For this to happen, the server must ensure that for two independently chosen values g^x and $g^{x'}$, it can compute values r and r' such that

$$g^{xr} = g^{x'r'},$$

causing a collision in the session identifiers (sid) of the two Alice oracles.

To analyze this, we construct an algorithm $\mathcal{B}''$ that interacts with the malicious server and breaks the inverse CDH assumption whenever E_{coll} occurs. The algorithm $\mathcal{B}''$ receives an inverse CDH tuple (g, g^α, g^β) from its challenger and initializes the ephemeral keys of π_i^s and $\pi_i^{s'}$ as g^α and g^β , respectively. The server then (possibly maliciously) samples r and sets the final ephemeral key of π_i^s as $g^{\alpha r}$, and samples r' to set the final ephemeral key of $\pi_i^{s'}$ as $g^{\beta r'}$.

$\mathcal{B}''$ receives both ephemeral keys and computes

$$h := g^\alpha, \quad h^{1/\alpha} := g, \quad \text{and} \quad h^{1/\beta} := g^{r/r'} = g^{\alpha/\beta}.$$

$\mathcal{B}''$ then returns $(g^\alpha, g, g^{r/r'})$ to its inverse CDH challenger.

It is clear that $\mathcal{B}''$ wins the inverse CDH game whenever the event E_{coll} happens, i.e., whenever the server successfully causes a collision between the ephemeral keys of π_i^s and $\pi_i^{s'}$ for $s \neq s'$. Consequently, we obtain

$$\Pr[E_{\text{coll}}] \leq \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

Game G_3 . At the beginning of the game, the challenger C uniformly at random picks an oracle for Alice $\pi_i^{\hat{s}}$ (i.e. initiator) and an oracle for Bob $\pi_j^{\hat{t}}$ (i.e. responder) from the set of all $\mu\ell$ oracles. We let E_{testO} be the event that the tested oracle is neither $\pi_i^{\hat{s}}$ nor $\pi_j^{\hat{t}}$, or the oracles $\pi_i^{\hat{s}}$ and $\pi_j^{\hat{t}}$ are not partners. The event E_{testO} can be checked efficiently and C aborts the game as soon as it detects E_{testO} occurs. C correctly guesses the choice made by $\mathcal{A}$ with probability at least $1/\mu^2\ell^2$, thus

$$\text{Adv}_{G_3}^{\mathcal{A}} \geq \frac{\text{Adv}_{G_1}^{\mathcal{A}}}{\mu^2\ell^2}.$$

Game G_4 . This game is identical to G_3 except that the game samples a random $v \xleftarrow{\$} \{0, 1\}^\lambda$ and in the test session it sets $\text{DH} := v$ instead of computing $\text{DH} := H(\text{DH}_1, \text{DH}_2, \text{DH}_3)$. We show that G_3 and G_4 are indistinguishable assuming the CDH assumption holds. For that, we build an algorithm $\mathcal{B}$ that violates the CDH assumption by using $\mathcal{A}$ as a subroutine.

$\mathcal{B}$ receives a CDH tuple (g, g^α, g^β) from its challenger and samples all the public parameters of Π_{AKE} . $\mathcal{B}$ then computes $(\text{lpk}_i, \text{lsk}_i)$ for all $i \in [\mu]$ by running the protocol honestly, and samples $(\hat{i}, \hat{j}, \hat{s}, \hat{t})$ uniformly at random from $[\mu]^2 \times [\ell]^2$. It initializes the list $\mathcal{H}$ as empty, and let the total number of queries the adversary can make to H be denoted by qs . It then invokes $\mathcal{A}$ on the public parameters of Π_{AKE} and $\{\text{lpk}_i | i \in [\mu]\}$ and answers queries made by $\mathcal{A}$ as follows:

- $H(v)$: To simulate the random oracle H , the algorithm $\mathcal{B}$ maintains a list $\mathcal{H}$ of pairs (v, u) , where each pair represents a queried value $v := (\text{DH}_1, \text{DH}_2, \text{DH}_3)$ and its corresponding response u from the random oracle H . Whenever the adversary queries H with an input v , if v is already present in the list $\mathcal{H}$, $\mathcal{B}$ retrieves and returns the previously assigned response u . Otherwise, $\mathcal{B}$ samples a random $u \xleftarrow{\$} \{0, 1\}^\lambda$, adds the pair (v, u) to the list $\mathcal{H}$, and returns u to the adversary.
- $\text{Send}(i, s, \langle \text{START: role}, j \rangle)$: Depending on the values (i, s, j) proceed as following.

- If $(i, s, j) = (\hat{i}, \hat{s}, \hat{j})$ then $\mathcal{B}$ returns g^α to $\mathcal{A}$ and receives r back from the adversary $\mathcal{A}$. Here we let adversary $\mathcal{A}$ to sample r on behalf of the server, what models r being chosen maliciously. If $r = 0$ then set $r := 1$. $\mathcal{B}$ implicitly sets $\text{state}_i^s \leftarrow \alpha \cdot r$ (note that $\mathcal{B}$ does not know α).
- If $(i, s, j) = (\hat{j}, \hat{t}, \hat{i})$ then $\mathcal{B}$ returns g^β to $\mathcal{A}$ and receives r' back from the adversary $\mathcal{A}$. Here we let adversary $\mathcal{A}$ to sample r' on behalf of the server, what models r' being chosen maliciously. If $r' = 0$ then set $r' := 1$. $\mathcal{B}$ implicitly sets $\text{state}_i^s \leftarrow \beta \cdot r'$ (note that $\mathcal{B}$ does not know α).
- In any other case, respond as in G_3 .
- **Send** $(i, s, m = (g^c, \sigma_j))$: Let $\text{Pid}_i^s := j$. Depending on the values (i, s, j) proceed as follows:
 - If $(i, s) = (\hat{i}, \hat{s})$ and $j \neq \hat{j}$ then oracles $\pi_i^{\hat{s}}$ and $\pi_j^{\hat{t}}$ are not partner oracles and the game aborts (event E_{testO} happens).
 - If $(i, s) = (\hat{j}, \hat{t})$ and $j \neq \hat{i}$ then oracles $\pi_i^{\hat{s}}$ and $\pi_j^{\hat{t}}$ are not partner oracles and the game aborts (event E_{testO} happens).
 - If $(i, s, j) = (\hat{i}, \hat{s}, \hat{j})$ then $\mathcal{B}$ checks if $g^c = g^{\beta r'}$; if not then $\mathcal{B}$ aborts (event E_{testO} happens). Otherwise it proceeds as in game G_3 except that it fixes $\text{DH} \leftarrow v$.
 - If $(i, s, j) = (\hat{j}, \hat{t}, \hat{i})$ then $\mathcal{B}$ checks if $g^c = g^{\alpha r}$; if not then $\mathcal{B}$ aborts (event E_{testO} happens). Otherwise it proceeds as in game G_3 except that it fixes $\text{DH} \leftarrow v$.
 - In any other case, $\mathcal{B}$ responds as in game G_3 .
- **RevLTK** (i) , **RegLTK** (i) , **RevState** (i, s) , **RevSessKey** (i, s) : $\mathcal{B}$ proceeds as in the previous game. We note that since $\mathcal{A}$ is following the Type-1 strategy $\mathcal{B}$ can answer all the **RevState** queries *except* **RevState** $(\hat{i}, \hat{s})$ or **RevState** $(\hat{j}, \hat{t})$.
- **Test** (i, s) : $\mathcal{B}$ responds as in G_2 . Note that in case $(i, s) \notin \{(\hat{i}, \hat{s}), (\hat{j}, \hat{t})\}$ the game aborts.

When $\mathcal{A}$ outputs a guess b' then $\mathcal{B}$ also outputs b' . By the random oracle model, we have that $\mathcal{A}$ can win the game G_4 if and only if it queries the random oracle with the value $v := (\text{DH}_1, \text{DH}_2, g^{(\alpha\beta)rr'})$. Algorithm $\mathcal{B}$ now picks a uniformly random pair $(v, u) \in \mathcal{H}$ and provides $(\text{DH}_3)^{1/rr'}$ to its challenger, thereby violating the CDH assumption. Consequently, we obtain

$$|\Pr[G_3] - \Pr[G_4]| \leq qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}).$$

Game G_5 . In this game we modify how the session key k is generated by the tested oracle (which is either $\pi_i^{\hat{s}}$ or $\pi_j^{\hat{t}}$ for some $\hat{t} \in [\ell]$). Instead of computing $\text{PRF}_{\text{DH}}(\text{sid})$, the tested oracle computes the session key as $k \xleftarrow{\$} \{0, 1\}^\lambda$. Due to the modification in game G_3 we have that DH is a random bit string. Hence, games G_4 and G_5 are indistinguishable assuming the pseudo-randomness of the PRF function. In particular, we can construct a PRF adversary $\mathcal{B}'$ such that

$$|\Pr[G_4] - \Pr[G_5]| \leq \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}').$$

Putting it all together, we have

$$\text{Adv}_{G_1}^A \leq \mu^2 \ell^2 \cdot (qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')).$$

□

Adversary using strategy Type-2 or Type-3

Lemma 28. *For any PPT adversary $\mathcal{A}$ using the Type-2 or Type-3 strategies, there exists PPT algorithms $\mathcal{B}$ breaking the GDH assumption, $\mathcal{B}'$ breaking the security of the PRF function, and $\mathcal{B}''$ breaking the inverse CDH assumption such that*

$$\text{Adv}_{G_1}^{\mathcal{A}} \leq \mu^2 \ell \cdot \left(\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}') \right) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

Proof. We give the full proof below for the Type-2 strategy. For the Type-3 strategy we point out that the proof is identical to the proof below (therefore we omit it here) with the exception that the roles of Alice and Bob are reversed.

Game G_2 . This game is identical to the previous one, except that we introduce an abort condition.

Let E_{coll} be the event that there exists an oracle with more than one partner oracle. If E_{coll} occurs, the game aborts. Since G_2 and G_3 are identical when E_{coll} does not occur, we obtain

$$|\text{Adv}_{G_1}^{\mathcal{A}} - \text{Adv}_{G_2}^{\mathcal{A}}| \leq \Pr[\text{E}_{\text{coll}}].$$

Next, we bound the probability of E_{coll} . Fix $i \in [\mu]$ and consider the set of oracles $S_i = \{\pi_j^t \mid \text{Pid}_j^t = i\}$. For any $\pi_j^t \in S_i$, if there exist two oracles π_i^s and $\pi_i^{s'}$ with $s \neq s'$ such that both are partners of π_j^t , then by definition, we must have

$$\text{sid}_j^t = \text{sid}_i^s = \text{sid}_i^{s'}.$$

Consider the case where π_j^t is an oracle for Bob (resp. Alice), and π_i^s and $\pi_i^{s'}$ are oracles for Alice (resp. Bob). Since the server is malicious and contributes to the ephemeral keys of both parties, we need to bound the probability that the server can force π_i^s and $\pi_i^{s'}$ to share the same ephemeral key, thereby triggering E_{coll} .

For this to happen, the server must ensure that for two independently chosen values g^x and $g^{x'}$, it can compute values r and r' such that

$$g^{xr} = g^{x'r'},$$

causing a collision in the session identifiers (sid) of the two Alice oracles.

To analyze this, we construct an algorithm $\mathcal{B}''$ that interacts with the malicious server and breaks the inverse CDH assumption whenever E_{coll} occurs. The algorithm $\mathcal{B}''$ receives an inverse CDH tuple (g, g^α, g^β) from its challenger and initializes the ephemeral keys of π_i^s and $\pi_i^{s'}$ as g^α and g^β , respectively. The server then (possibly maliciously) samples r and sets the final ephemeral key of π_i^s as $g^{\alpha r}$, and samples r' to set the final ephemeral key of $\pi_i^{s'}$ as $g^{\beta r'}$. $\mathcal{B}''$ receives both ephemeral keys and computes

$$h := g^\alpha, \quad h^{1/\alpha} := g, \quad \text{and} \quad h^{1/\beta} := g^{r/r'} = g^{\alpha/\beta}.$$

$\mathcal{B}''$ then returns $(g^\alpha, g, g^{r/r'})$ to its inverse CDH challenger.

It is clear that $\mathcal{B}''$ wins the inverse CDH game whenever the event E_{coll} happens, i.e., whenever the server successfully causes a collision between the ephemeral keys of π_i^s and $\pi_i^{s'}$ for $s \neq s'$. Consequently, we obtain

$$\Pr[\text{E}_{\text{coll}}] \leq \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

Game G_3 . At the beginning of the game, the challenger C uniformly randomly picks a random party P_j to act as Bob (i.e. responder) from the μ parties and a random oracle for Alice $\pi_i^{\hat{s}}$ (i.e. initiator) from the $\mu\ell$ possible oracles. We let E_{testO} be the event that the tested oracle is not $\pi_j^{\hat{t}}$ for some $\hat{t} \in [\ell]$ and its partner oracle is $\pi_i^{\hat{s}}$, or the tested oracle is not $\pi_i^{\hat{s}}$ and its peer is P_j . Since E_{testO} can be tested efficiently the challenger C aborts the game as soon as E_{testO} occurs. Thus, C guesses the choice made by $\mathcal{A}$ with probability $1/\mu^2\ell$, so we have

$$\text{Adv}_{G_3}^{\mathcal{A}} = \frac{1}{\mu^2\ell} \cdot \text{Adv}_{G_2}^{\mathcal{A}}.$$

Game G_4 . This game is identical to G_3 except that the game samples a random value $\text{DH} \xleftarrow{\$} \{0, 1\}^\lambda$ to be used in the test session instead of computing $\text{DH} \leftarrow H(\text{DH}_1, \text{DH}_2, \text{DH}_3)$. We show that G_3 and G_4 are indistinguishable assuming the GDH assumption holds. For this, we build an algorithm $\mathcal{B}$ that breaks the CDH assumption (given access to a DDH oracle) with the same probability $\mathcal{A}$ does (by using $\mathcal{A}$ as a subroutine).

$\mathcal{B}$ receives a tuple (g, g^α, g^β) from the GDH challenger, samples $(\hat{i}, \hat{j}, \hat{s})$ uniformly at random from $[\mu]^2 \times [\ell]$, samples all the public parameters of Π_{AKE} , and generates the long-term keys as follows. For party P_j algorithm $\mathcal{B}$ runs $(\text{spk}_j, \text{ssk}_j) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$ and sets the long-term public keys as $\text{lpk} := (g^\beta, \text{spk}_j)$ and *implicitly* sets the long-term secret keys as $\text{spk} := (\beta, \text{ssk}_j)$ since $\mathcal{B}$ does not know β that is the exponent in the GDH challenge. For all other parties $i \in [\mu \setminus \hat{j}]$ $\mathcal{B}$ computes $(\text{lpk}_i, \text{lsk}_i)$ by running the protocol honestly. It initializes the lists $\mathcal{H}$ and $\mathcal{L}$ as empty. It then invokes $\mathcal{A}$ on the public parameters of Π_{AKE} and $\{\text{lpk}_i | i \in [\mu]\}$ and answers queries made by $\mathcal{A}$ as follows:

- $H(v)$: To simulate the random oracle H , the algorithm $\mathcal{B}$ maintains a list $\mathcal{H}$ of pairs (v, u) , where each pair represents a queried value v and its corresponding response u from the random oracle H . Whenever the adversary queries H with an input v , the algorithm $\mathcal{B}$ proceeds as follows:
 - If v is already present in the list $\mathcal{H}$, $\mathcal{B}$ retrieves and returns the previously assigned response u .
 - Otherwise, algorithm $\mathcal{B}$ parses $v := (\text{DH}_1, \text{DH}_2, \text{DH}_3)$ and for all tuples $(g^c, \text{DH}'_2, \text{DH}'_3, \text{DH}) \in \mathcal{L}$ it sends the triple $(g^\beta, g^c, \text{DH}_1)$ to the DDH oracle. If the oracle replies with 1 then $\mathcal{B}$ returns the corresponding value DH in $\mathcal{L}$ as response. Otherwise, if the DDH oracle replies with 0 or if $\mathcal{L}$ is empty, then $\mathcal{B}$ samples a fresh random value $u \leftarrow \{0, 1\}^\lambda$, adds the pair (v, u) to $\mathcal{H}$, and returns u as the response.
- $\text{Send}(i, s, m = \langle \text{START: role}, j \rangle)$: Depending on the values (i, s, j) proceed as following.
 - If $(i, s, j) = (\hat{i}, \hat{s}, \hat{j})$ then $\mathcal{B}$ returns g^α to $\mathcal{A}$ and receives r back from the adversary $\mathcal{A}$. Here we let adversary $\mathcal{A}$ to sample r on behalf of the server, what models r being chosen maliciously. If $r = 0$ then set $r := 1$. $\mathcal{B}$ implicitly sets $\text{state}_i^{\hat{s}} = \alpha \cdot r$ (note that $\mathcal{B}$ does not know α).
 - In any other case, $\mathcal{B}$ responds as in game G_3 .
- $\text{Send}(i, s, m = (g^c, \sigma_j))$: Let $j = \text{pid}_i^s$. Depending on the values (i, s, j) proceed as follows:
 - If $i = \hat{j}$, then $\mathcal{B}$ proceeds as follows:

- * At this point, the values DH_2 and DH_3 are fixed for session s and can be computed by $\mathcal{B}$.
- * For each pair $(v, u) \in \mathcal{H}$ parse v as $(\text{DH}'_1, \text{DH}'_2, \text{DH}'_3)$.
 - Check if $\text{DH}_2 = \text{DH}'_2$ and $\text{DH}_3 = \text{DH}'_3$.
 - If the check passes, send the triple $(g^\beta, g^c, \text{DH}'_1)$ to the DDH oracle.
 - If the DDH oracle returns 1, set the session key DH to the corresponding value u from $\mathcal{H}$.
 - Otherwise (i.e., if the DDH oracle returns 0 or if DH'_2 and DH'_3 do not match DH_2 and DH_3), generate a fresh random key $\text{DH} \leftarrow \{0, 1\}^\lambda$.
 - Finally, store the tuple $(g^c, \text{DH}'_2, \text{DH}'_3, \text{DH})$ in the list $\mathcal{L}$.
- In any other case, $\mathcal{B}$ responds as in game G_3 .
- $\text{RevLTK}(i), \text{RegLTK}(i), \text{RevState}(i, s), \text{RevSessKey}(i, s)$: $\mathcal{B}$ proceeds as in the previous game. We note that since $\mathcal{A}$ is following the Type-2 strategy $\mathcal{B}$ can answer all the RevState queries *except* $\text{RevState}(\hat{i}, \hat{s})$ and it can answer all RevLTK queries except $\text{RevLTK}(\hat{j})$.
- $\text{Test}(i, s)$: $\mathcal{B}$ responds as in G_3 . Note that in case $i \neq \hat{j}$ or $(i, s) \neq (\hat{i}, \hat{s})$ the game aborts (i.e., event E_{testO} is triggered).

When $\mathcal{A}$ outputs a guess b' then $\mathcal{B}$ also outputs b' . We note that $\mathcal{B}$ consistently simulates the random oracle queries for $\mathcal{A}$. If $\mathcal{A}$ wins in game G_4 it must have queried the random oracle with the value $v = (g^{(\alpha\beta)r}, \text{DH}_2, \text{DH}_3)$. At this point, $\mathcal{B}$ searches through the list $\mathcal{H}$ and, for each tuple, sends the query $(g^\alpha, g^\beta, \text{DH}_1)$ to its DDH oracle; if the DDH oracle returns 1 then AdvB returns DH_1 to its GDH challenger. Consequently, we obtain

$$|\Pr[G_3] - \Pr[G_4]| \leq \text{Adv}_{\text{gdh}}(\mathcal{B}).$$

Game G_5 . In this game we sample a random function $f : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ and whenever the game needs to compute a session key $k \leftarrow \text{PRF}_{\text{DH}}(\text{sid})$ it instead compute as $k := f(\text{DH}, \text{sid})$. Due to the modification in game G_4 the value DH is a random bit string. Hence, games G_4 and G_5 are indistinguishable assuming the pseudo-randomness of the PRF function. In particular, we can construct an adversary $\mathcal{B}'$ for the PRF function such that

$$|\Pr[G_4] - \Pr[G_5]| \leq \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}').$$

Lastly, we argue that the session key output by the tested oracle in game G_5 is uniformly random regardless of the bit b sampled by the challenger. Conditioned on event E_{testO} not happening, the tested oracle computes the session key as $k \leftarrow f(\text{DH}, \text{sid}^*)$ for some sid^* . Note that the PRF key DH used is a random bit string due to the changes in game G_5 . Moreover, conditioned on E_{coll} not happening, the only oracles that share the same sid^* are the tested oracle and its partner. Therefore, $f(\text{DH}, \text{sid}^*)$ is only used to compute the session key of the tested oracle and its partner. Since the output of f is uniformly random for different inputs, we can conclude that $\Pr[G_5] = 1/2$. Thus, combining all the arguments together, we have

$$\text{Adv}_{G_1}^{\mathcal{A}} \leq \mu^2 \ell \cdot \left(\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}') \right) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

□

Adversary using strategies Type-5 or Type-6. (resp. Type-7 or Type-8)

Lemma 29. *For any PPT adversary $\mathcal{A}$ using the Type-5, Type-6, Type-7 or Type-8 strategies, there exists a PPT algorithm $\mathcal{B}$ breaking the unforgeability of Σ such that*

$$\text{Adv}_{G_1}^{\mathcal{A}} \leq \mu \cdot \text{Adv}_{\text{EU-CMA}}^{\Sigma}(\mathcal{B}).$$

Proof. We give the full proof below for the Type-5 and Type-6 strategies. For the Type-7 and Type-8 strategies we point out that the proof is identical to the proof below (therefore we omit it here) with the exception that the roles of Alice and Bob are reversed.

Game G_2 . At the beginning of the game, the challenger C picks a party $P_{\hat{j}}$ uniformly at random from the set of μ parties to act as Bob (i.e. responder). The game aborts if the *peer* of the tested oracle is not $P_{\hat{j}}$. Thus,

$$\text{Adv}_{G_2}^{\mathcal{A}} = \frac{1}{\mu} \text{Adv}_{G_1}^{\mathcal{A}}.$$

Game G_3 . This game is identical to the previous game, except that we add another abort condition. Let S be the list of message-signature pairs that $P_{\hat{j}}$ generated as being a responder oracle. More specifically, every time $\pi_{\hat{j}}^t$ for some $t \in [\ell]$ is invoked as a responder, the game updates the list S to include the message-signature pair $(g^c, \sigma_{\hat{j}}^t)$ that it generates. Then when an initiator oracle π_i^s for any $(i, s) \in [\mu] \times [\ell]$ is invoked on input (g^c, σ) from party $P_{\hat{j}}$ where $\text{Pid}_i^s = \hat{j}$ it performs the following checks: If $P_{\hat{j}}$ is not corrupt and $\Sigma.\text{Ver}(\text{spk}_{\hat{j}}, g^c, \sigma) = 1$ then it must be that $(g^c, \sigma) \in S$ or the game aborts. Otherwise, it proceeds as in the previous game. Let's call this abort event E_{sig} . Games G_2 and G_3 are identical if event E_{sig} does not happen, thus

$$|\Pr[G_2] - \Pr[G_3]| \leq \Pr[\text{E}_{\text{sig}}].$$

We argue that no adversary $\mathcal{A}$ following strategy Type-5 or Type-6 has a winning advantage in game G_3 . Assume $\mathcal{A}$ issues a test-query $\text{Test}(i^*, s^*)$ and receives a session key $k \neq \perp$. This implies that $\pi_{i^*}^{s^*}$ is in the **accept** state (otherwise the adversary would only receive $k = \perp$). Due to the modification made in game G_2 and the definition of Type-5 or Type-6 strategies the tested oracle $\pi_{i^*}^{s^*}$ has no partner oracle $\pi_{\hat{j}}^t$ for any $t \in [\ell]$ and the peer $P_{\hat{j}}$ was not corrupted before $\pi_{i^*}^{s^*}$ finishes the protocol. However, if $\pi_{i^*}^{s^*}$ reaches the **accept** state then the event E_{sig} must have not happened. Thus, there exists an oracle $\pi_{\hat{j}}^t$ for $t \in [\ell]$ that sent the message (g^{c^*}, σ^*) that was used by the tested oracle. However, if oracle $\pi_{\hat{j}}^t$ exists then it would be a partner of oracle $\pi_{i^*}^{s^*}$ (the **sid**'s would be the same). This is clearly a contradiction and thus $\mathcal{A}$ could have only received $\perp$ when it issues a $\text{Test}(i^*, s^*)$ query. Therefore $\Pr[G_3] = 1/2$.

We can bound the probability of event E_{sig} happening by constructing an algorithm $\mathcal{B}$ against the EU-CMA security of Σ as follows: $\mathcal{B}$ samples all the public parameters of Π_{AKE} , and receives the public parameters of Σ and the challenge verification key spk^* . Algorithm $\mathcal{B}$ randomly samples $\hat{j} \in [\mu]$ and sets the long-term public and secret keys of party $P_{\hat{j}}$ by computing $b_{\hat{j}} \xleftarrow{\$} \mathbb{Z}_p^*$ and sets $\text{lpk}_{\hat{j}} := (\text{spk}^*, g^{b_{\hat{j}}})$ and $\text{lsk}_{\hat{j}} := (\text{ssk}^*, b_{\hat{j}})$. Note that ssk^* is unknown to $\mathcal{B}$ and thus this is only set implicitly. For all other parties $P_j \in [\mu \setminus \hat{j}]$ algorithm $\mathcal{B}$ generates the keys as in G_2 . Finally $\mathcal{B}$ invokes $\mathcal{A}$ on input the public parameters of Π_{AKE} and $\{\text{lpk}_j | j \in [\mu]\}$ and answer the queries of $\mathcal{A}$ as follows:

- $\text{Send}(i, s, m = \langle \text{START: role}, j \rangle)$: Depending on the value i proceed as following.
 - If $i = \hat{j}$ then $\mathcal{B}$ samples g^y sends it to $\mathcal{A}$ and receives r' back from the adversary $\mathcal{A}$. Here we let adversary $\mathcal{A}$ to sample r' on behalf of the server, what models r' being chosen maliciously. $\mathcal{B}$ then sends the value $g^{yr'}$ to its signing oracle and receives back a signature σ' under the signing key ssk^* . Algorithm $\mathcal{B}$ then responds as in G_2 except that it sets $\sigma := \sigma'$.
 - In any other case responds as in game G_2
- $\text{Send}(j, t, m = (g^c, \sigma_i))$: Responds as in the previous game.
- $\text{RevLTK}(i), \text{RegLTK}(i), \text{RevState}(i, s), \text{RevSessKey}(i, s)$: $\mathcal{B}$ proceeds as in the previous game. We note that since $\mathcal{A}$ is following the Type-5 or Type-6 strategy $\mathcal{B}$ can answer all the RevLTK queries *except* $\text{RevLTK}(\hat{j})$.
- $\text{Test}(i, s)$: $\mathcal{B}$ responds as in G_2 . In case $\text{Pid}_s^i \neq \hat{j}$ then the game aborts.

Note that $\mathcal{B}$ perfectly simulates the view of game G_2 to $\mathcal{A}$. Assume that $\mathcal{A}$ issues a $\text{Test}(i^*, s^*)$ query. Let the message received by $\pi_{i^*}^{s^*}$ be $(g^{yr'}, \sigma_{\hat{j}})$. By the definition of Type-5 and Type-6 strategies we are assured that the tested oracle $\pi_{i^*}^{s^*}$ satisfies: (1) $\text{role} = \text{init}$ and $\text{Pid}_{i^*}^{s^*} = \hat{j}$, (2) $\pi_{i^*}^{s^*}$ is in an *accept* state; in particular, this implies that $\Sigma.\text{Ver}(\text{spk}^*, g^{yr'}, \sigma_{\hat{j}})$. (3) $P_{\hat{j}}$ is not corrupted before $\pi_{i^*}^{s^*}$ completes the protocol execution, and finally (4) $\pi_{i^*}^{s^*}$ has no partner oracles.

Since $\pi_{i^*}^{s^*}$ has no partner oracles, there exists no oracle $\pi_{\hat{j}}^t$ that outputs $(g^{yr'}, \sigma_{\hat{j}})$. Therefore, no oracle $\pi_{\hat{j}}^t$ signed the message $g^{yr'}$. Note that this is exactly event E_{sig} : an initiator oracle $\pi_{i^*}^{s^*}$ receives a valid signature that was not produced by oracle $\pi_{\hat{j}}^t$ for any $t \in [\ell]$ and $P_{\hat{j}}$ was not corrupted at that time. Moreover, note that triggering event E_{sig} requires the adversary $\mathcal{A}$ to produce a forgery of a signature on message $g^{yr'}$. Thus, we conclude that $\Pr[\text{E}_{\text{sig}}] = \text{Adv}_{\text{EU-CMA}}^{\Sigma}(\mathcal{B})$. Combining all the games together, we have that

$$\text{Adv}_{G_1}^A \leq \mu \cdot \text{Adv}_{\text{EU-CMA}}^{\Sigma}(\mathcal{B}).$$

□

Thus, combining all the arguments from all adversarial strategies together, we have that

$$\text{Adv}_{\Pi_{\text{AKE}}}^{\text{FS}}(\mathcal{A}) \leq \max \begin{cases} \mu^2 \ell^2 \cdot (q_s \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}''), \\ \mu^2 \ell \cdot (\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}''), \\ \mu \cdot \text{Adv}_{\text{EU-CMA}}^{\Sigma}(\mathcal{B}) \end{cases}$$

□

6.3 Subversion Security for X3DH

In this section, we describe the reverse firewalls for both Alice and Bob within the context of the protocol illustrated in [Fig. 14](#). Since Alice and Bob's firewalls are identical in both design and

functionality, our discussion will primarily focus on Alice's firewall. We provide a formal description of Alice's firewall W_A in Fig. 15 and subsequently demonstrate its security. As Bob's firewall is identical to Alice's, we omit its detailed discussion.

Alice's Firewall. Given the firewall-friendly architecture of our X3DH protocol, the firewall for Alice (and for Bob) is notably straightforward. Alice's ephemeral key g^x is intercepted by the firewall, which re-randomizes it to $g^{x\alpha}$ before forwarding it to the server. Per the protocol in Fig. 14, the server replies with a random nonzero value r ; the firewall captures this and modifies it to αr ,²³ which is then delivered to Alice. She uses αr to compute her final ephemeral key $g^{x\alpha r}$. This extra interaction enables the firewall to systematically alter the ephemeral key without affecting correctness. In the next phase, Alice signs her ephemeral key and sends the signature to the server. The firewall verifies that the received key matches the one computed with αr and that the signature is valid, ensuring that a compromised Alice cannot leak information via the key or its signature. The remaining of the protocol is exactly the same as the protocol in Fig. 14.

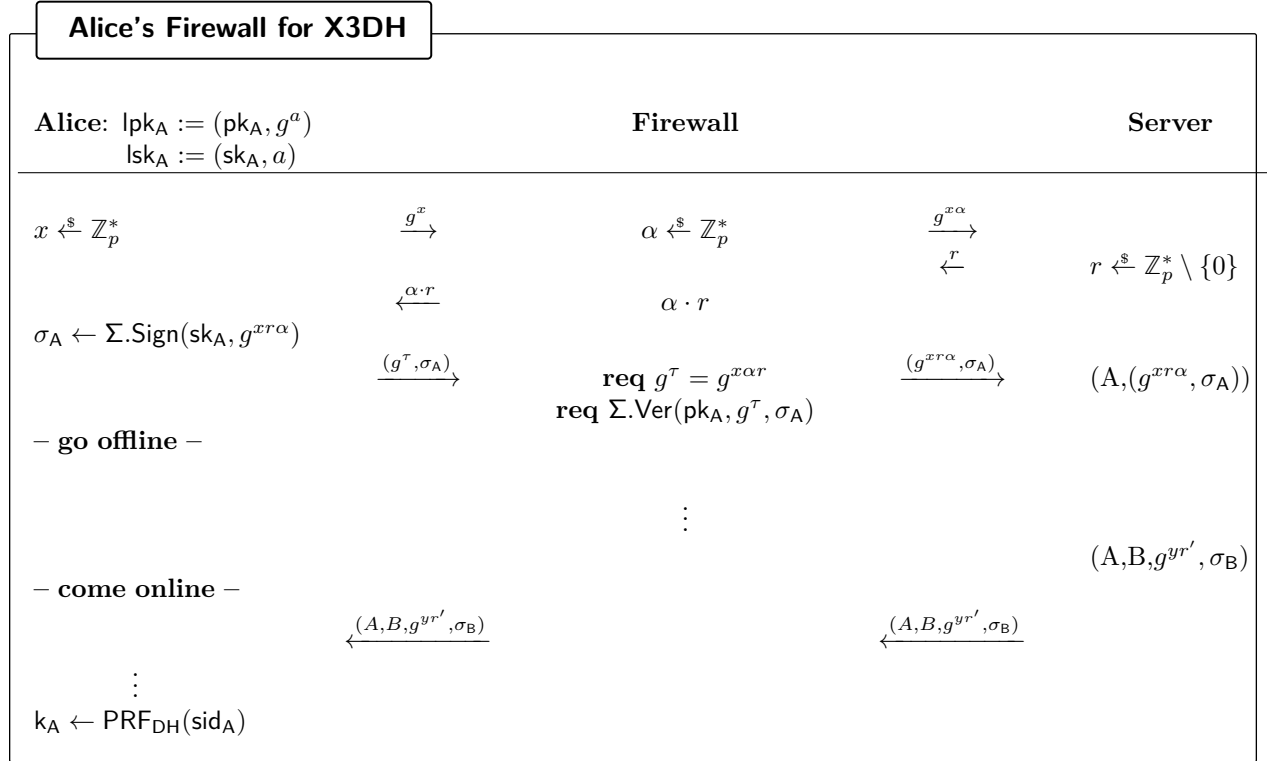


Figure 15: Firewall W_A for Alice in the X3DH protocol.

Theorem 10. *The reverse firewall W_A in Fig. 15 is functionality maintaining for the protocol of Fig. 14. Furthermore, assuming unforgeability of the unique signature scheme Σ then W_A preserves security for A and B, and is exfiltration resistant for A against B and for B against A.*

²³If $r = 0$, the firewall ignores it and simply returns α to Alice.

As the proof shares many similarities with the proof of [Theorem 9](#), we defer the full proof to [Sec. C](#).

References

- [1] J. Alwen, S. Coretti, and Y. Dodis. The double ratchet: Security notions, proofs, and modularization for the Signal protocol. In Y. Ishai and V. Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 129–158. Springer, Cham, May 2019.
- [2] J. Alwen, a. shelat, and I. Visconti. Collusion-free protocols in the mediated model. In D. Wagner, editor, *CRYPTO 2008*, volume 5157 of *LNCS*, pages 497–514. Springer, Berlin, Heidelberg, Aug. 2008.
- [3] M. Armour and B. Poettering. Substitution attacks against message authentication. *IACR Trans. Symm. Cryptol.*, 2019(3):152–168, 2019.
- [4] G. Ateniese, D. Francati, B. Magri, and D. Venturi. Public immunization against complete subversion without random oracles. In R. H. Deng, V. Gauthier-Umaña, M. Ochoa, and M. Yung, editors, *ACNS 2019*, volume 11464 of *LNCS*, pages 465–485. Springer, Cham, June 2019.
- [5] G. Ateniese, B. Magri, and D. Venturi. Subversion-resilient signature schemes. In I. Ray, N. Li, and C. Kruegel, editors, *ACM CCS 2015*, pages 364–375. ACM Press, Oct. 2015.
- [6] B. Auerbach, M. Bellare, and E. Kiltz. Public-key encryption resistant to parameter subversion and its realization from efficiently-embeddable groups. In M. Abdalla and R. Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 348–377. Springer, Cham, Mar. 2018.
- [7] F. Balli, P. Rösler, and S. Vaudenay. Determining the core primitive for optimally secure ratcheting. In S. Moriai and H. Wang, editors, *ASIACRYPT 2020, Part III*, volume 12493 of *LNCS*, pages 621–650. Springer, Cham, Dec. 2020.
- [8] M. Bellare, G. Fuchsbauer, and A. Scafuro. NIZKs with an untrusted CRS: Security in the face of parameter subversion. In J. H. Cheon and T. Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 777–804. Springer, Berlin, Heidelberg, Dec. 2016.
- [9] M. Bellare, J. Jaeger, and D. Kane. Mass-surveillance without the state: Strongly undetectable algorithm-substitution attacks. In I. Ray, N. Li, and C. Kruegel, editors, *ACM CCS 2015*, pages 1431–1440. ACM Press, Oct. 2015.
- [10] M. Bellare, K. G. Paterson, and P. Rogaway. Security of symmetric encryption against mass surveillance. In J. A. Garay and R. Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 1–19. Springer, Berlin, Heidelberg, Aug. 2014.
- [11] M. Bellare, A. C. Singh, J. Jaeger, M. Nyayapati, and I. Stepanovs. Ratcheted encryption and key exchange: The security of messaging. In J. Katz and H. Shacham, editors, *CRYPTO 2017, Part III*, volume 10403 of *LNCS*, pages 619–650. Springer, Cham, Aug. 2017.

- [12] P. Bemmman, R. Chen, and T. Jager. Subversion-resilient public key encryption with practical watchdogs. In J. Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 627–658. Springer, Cham, May 2021.
- [13] S. Berndt and M. Liskiewicz. Algorithm substitution attacks from a steganographic perspective. In B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, editors, *ACM CCS 2017*, pages 1649–1660. ACM Press, Oct. / Nov. 2017.
- [14] A. Bienstock, J. Fairuze, S. Garg, P. Mukherjee, and S. Raghuraman. A more complete analysis of the Signal double ratchet algorithm. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 784–813. Springer, Cham, Aug. 2022.
- [15] A. Bienstock, P. Rösler, and Y. Tang. Asmesh: Anonymous and secure messaging in mesh networks using stronger, anonymous double ratchet. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1–15, 2023.
- [16] N. Borisov, I. Goldberg, and E. Brewer. Off-the-record communication, or, why not to use pgp. In *Proceedings of the 2004 ACM workshop on Privacy in the electronic society*, pages 77–84, 2004.
- [17] A. Bossuat, X. Bultel, P.-A. Fouque, C. Onete, and T. van der Merwe. Designing reverse firewalls for the real world. In L. Chen, N. Li, K. Liang, and S. A. Schneider, editors, *ESORICS 2020, Part I*, volume 12308 of *LNCS*, pages 193–213. Springer, Cham, Sept. 2020.
- [18] J. Brendel, R. Fiedler, F. Günther, C. Janson, and D. Stebila. Post-quantum asynchronous deniable key exchange and the Signal handshake. In G. Hanaoka, J. Shikata, and Y. Watanabe, editors, *PKC 2022, Part II*, volume 13178 of *LNCS*, pages 3–34. Springer, Cham, Mar. 2022.
- [19] J. Brendel, M. Fischlin, F. Günther, C. Janson, and D. Stebila. Towards post-quantum security for signal’s x3dh handshake. In *Selected Areas in Cryptography: 27th International Conference, Halifax, NS, Canada (Virtual Event), October 21-23, 2020, Revised Selected Papers 27*, pages 404–430. Springer, 2021.
- [20] A. Caforio, F. B. Durak, and S. Vaudenay. Beyond security and efficiency: On-demand ratcheting with security awareness. In J. Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 649–677. Springer, Cham, May 2021.
- [21] R. Canetti, P. Jain, M. Swanberg, and M. Varia. Universally composable end-to-end secure messaging. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 3–33. Springer, Cham, Aug. 2022.
- [22] S. Chakraborty, S. Dziembowski, and J. B. Nielsen. Reverse firewalls for actively secure MPCs. In D. Micciancio and T. Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 732–762. Springer, Cham, Aug. 2020.
- [23] S. Chakraborty, C. Ganesh, M. Pancholi, and P. Sarkar. Reverse firewalls for adaptively secure MPC without setup. In M. Tibouchi and H. Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 335–364. Springer, Cham, Dec. 2021.

- [24] S. Chakraborty, B. Magri, J. B. Nielsen, and D. Venturi. Universally composable subversion-resilient cryptography. In O. Dunkelman and S. Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 272–302. Springer, Cham, May / June 2022.
- [25] R. Chen, Y. Mu, G. Yang, W. Susilo, F. Guo, and M. Zhang. Cryptographic reverse firewall via malleable smooth projective hash functions. In J. H. Cheon and T. Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 844–876. Springer, Berlin, Heidelberg, Dec. 2016.
- [26] S. S. M. Chow, A. Russell, Q. Tang, M. Yung, Y. Zhao, and H.-S. Zhou. Let a non-barking watchdog bite: Cliptographic signatures with an offline watchdog. In D. Lin and K. Sako, editors, *PKC 2019, Part I*, volume 11442 of *LNCS*, pages 221–251. Springer, Cham, Apr. 2019.
- [27] K. Cohn-Gordon, C. Cremers, B. Dowling, L. Garratt, and D. Stebila. A formal security analysis of the Signal messaging protocol. *Journal of Cryptology*, 33(4):1914–1983, Oct. 2020.
- [28] J. P. Degabriele, P. Farshim, and B. Poettering. A more cautious approach to security against mass surveillance. In G. Leander, editor, *FSE 2015*, volume 9054 of *LNCS*, pages 579–598. Springer, Berlin, Heidelberg, Mar. 2015.
- [29] J. P. Degabriele, K. G. Paterson, J. C. N. Schuldt, and J. Woodage. Backdoors in pseudorandom number generators: Possibility and impossibility results. In M. Robshaw and J. Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 403–432. Springer, Berlin, Heidelberg, Aug. 2016.
- [30] S. Dobson and S. D. Galbraith. Post-quantum signal key agreement from sidh. In *International Conference on Post-Quantum Cryptography*, pages 422–450. Springer, 2022.
- [31] Y. Dodis, C. Ganesh, A. Golovnev, A. Juels, and T. Ristenpart. A formal treatment of backdoored pseudorandom generators. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 101–126. Springer, Berlin, Heidelberg, Apr. 2015.
- [32] Y. Dodis, I. Mironov, and N. Stephens-Davidowitz. Message transmission with reverse firewalls—secure communication on corrupted machines. In M. Robshaw and J. Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 341–372. Springer, Berlin, Heidelberg, Aug. 2016.
- [33] B. Dowling, F. Günther, and A. Poirrier. Continuous authentication in secure messaging. In V. Atluri, R. Di Pietro, C. D. Jensen, and W. Meng, editors, *ESORICS 2022, Part II*, volume 13555 of *LNCS*, pages 361–381. Springer, Cham, Sept. 2022.
- [34] B. Dowling and B. Hale. There can be no compromise: The necessity of ratcheted authentication in secure messaging. Cryptology ePrint Archive, Report 2020/541, 2020.
- [35] B. Dowling and B. Hale. Secure messaging authentication against active man-in-the-middle attacks. In *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 54–70. IEEE, 2021.
- [36] B. Dowling, E. Hauck, D. Riepel, and P. Rösler. Strongly anonymous ratcheted key exchange. In S. Agrawal and D. Lin, editors, *ASIACRYPT 2022, Part III*, volume 13793 of *LNCS*, pages 119–150. Springer, Cham, Dec. 2022.

- [37] F. B. Durak and S. Vaudenay. Bidirectional asynchronous ratcheted key agreement with linear complexity. In *Advances in Information and Computer Security: 14th International Workshop on Security, IWSEC 2019, Tokyo, Japan, August 28–30, 2019, Proceedings 14*, pages 343–362. Springer, 2019.
- [38] M. Fischlin, C. Janson, and S. Mazaheri. Backdoored hash functions: Immunizing HMAC and HKDF. In S. Chong and S. Delaune, editors, *CSF 2018 Computer Security Foundations Symposium*, pages 105–118. IEEE Computer Society Press, 2018.
- [39] C. Ganesh, B. Magri, and D. Venturi. Cryptographic reverse firewalls for interactive proof systems. In A. Czumaj, A. Dawar, and E. Merelli, editors, *ICALP 2020*, volume 168 of *LIPIcs*, pages 55:1–55:16. Schloss Dagstuhl, July 2020.
- [40] I. Giacomelli, R. F. Olimid, and S. Ranellucci. Security of linear secret-sharing schemes against mass surveillance. In M. Reiter and D. Naccache, editors, *CANS 15*, LNCS, pages 43–58. Springer, Cham, Dec. 2015.
- [41] K. Hashimoto, S. Katsumata, K. Kwiatkowski, and T. Prest. An efficient and generic construction for Signal’s handshake (X3DH): Post-quantum, state leakage secure, and deniable. In J. Garay, editor, *PKC 2021, Part II*, volume 12711 of *LNCS*, pages 410–440. Springer, Cham, May 2021.
- [42] K. Hashimoto, S. Katsumata, K. Kwiatkowski, and T. Prest. An efficient and generic construction for Signal’s handshake (X3DH): Post-quantum, state leakage secure, and deniable. *Journal of Cryptology*, 35(3):17, July 2022.
- [43] J. Jaeger and I. Stepanovs. Optimal channel security against fine-grained state compromise: The safety of messaging. In H. Shacham and A. Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 33–62. Springer, Cham, Aug. 2018.
- [44] D. Jost, U. Maurer, and M. Mularczyk. Efficient ratcheting: Almost-optimal guarantees for secure messaging. In Y. Ishai and V. Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 159–188. Springer, Cham, May 2019.
- [45] N. Kobeissi, K. Bhargavan, and B. Blanchet. Automated verification for secure messaging protocols and their implementations: A symbolic and computational approach. In *2017 IEEE European symposium on security and privacy (EuroS&P)*, pages 435–450. IEEE, 2017.
- [46] N. Kostyuk and S. Landau. Dueling over DUALEC_DRBG: The consequences of corrupting a cryptographic standardization process. In *Harvard National Security Journal*, 2022.
- [47] J. Liu, R. Chen, Y. Wang, X. Tang, and J. Su. Subversion-resilient authenticated key exchange with reverse firewalls. In J. K. Liu, L. Chen, S.-F. Sun, and X. Liu, editors, *ProvSec 2023, Part II*, volume 14904 of *LNCS*, pages 181–200. Springer, Singapore, Sept. 2024.
- [48] M. Marlinspike and T. Perrin. The double ratchet algorithm, 2016. <https://whispersystems.org/docs/specifications/doubleratchet/doubleratchet.pdf>.
- [49] M. Marlinspike and T. Perrin. The x3dh key agreement protocol, 2016. <https://signal.org/docs/specifications/x3dh/x3dh.pdf>.

- [50] G. Miller. The intelligence coup of the century, 2020. <https://www.washingtonpost.com/graphics/2020/world/national-security/cia-crypto-encryption-machines-espionage/>.
- [51] I. Mironov and N. Stephens-Davidowitz. Cryptographic reverse firewalls. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part II*, volume 9057 of *LNCS*, pages 657–686. Springer, Berlin, Heidelberg, Apr. 2015.
- [52] T. Okamoto and D. Pointcheval. The gap-problems: A new class of problems for the security of cryptographic schemes. In K. Kim, editor, *PKC 2001*, volume 1992 of *LNCS*, pages 104–118. Springer, Berlin, Heidelberg, Feb. 2001.
- [53] B. Poettering and P. Rösler. Towards bidirectional ratcheted key exchange. In H. Shacham and A. Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 3–32. Springer, Cham, Aug. 2018.
- [54] M. Ringerud. Note on subversion-resilient key exchange. Cryptology ePrint Archive, Report 2023/749, 2023.
- [55] J. Robertson and M. Riley. The big hack: How china used a tiny chip to infiltrate u.s. companies, 2018. <https://www.bloomberg.com/news/features/2018-10-04/the-big-hack-how-china-used-a-tiny-chip-to-infiltrate-america-s-top-companies>.
- [56] A. Russell, Q. Tang, M. Yung, and H.-S. Zhou. Cliptography: Clipping the power of kleptographic attacks. In J. H. Cheon and T. Takagi, editors, *ASIACRYPT 2016, Part II*, volume 10032 of *LNCS*, pages 34–64. Springer, Berlin, Heidelberg, Dec. 2016.
- [57] A. Russell, Q. Tang, M. Yung, and H.-S. Zhou. Generic semantic security against a kleptographic adversary. In B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, editors, *ACM CCS 2017*, pages 907–922. ACM Press, Oct. / Nov. 2017.
- [58] A. Russell, Q. Tang, M. Yung, and H.-S. Zhou. Correcting subverted random oracles. In H. Shacham and A. Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 241–271. Springer, Cham, Aug. 2018.
- [59] G. J. Simmons. The prisoners’ problem and the subliminal channel. In D. Chaum, editor, *CRYPTO’83*, pages 51–67. Plenum Press, New York, USA, 1983.
- [60] G. J. Simmons. Authentication theory/coding theory. In G. R. Blakley and D. Chaum, editors, *CRYPTO’84*, volume 196 of *LNCS*, pages 411–431. Springer, Berlin, Heidelberg, Aug. 1984.
- [61] O. W. Systems. The signal application, 2023. <https://github.com/signalapp>.
- [62] N. Unger and I. Goldberg. Deniable key exchanges for secure messaging. In I. Ray, N. Li, and C. Kruegel, editors, *ACM CCS 2015*, pages 1211–1223. ACM Press, Oct. 2015.
- [63] N. Unger and I. Goldberg. Improved strongly deniable authenticated key exchanges for secure messaging. *PoPETs*, 2018(1):21–66, Jan. 2018.

- [64] N. Vatandas, R. Gennaro, B. Ithurburn, and H. Krawczyk. On the cryptographic deniability of the Signal protocol. In M. Conti, J. Zhou, E. Casalicchio, and A. Spognardi, editors, *ACNS 2020, Part II*, volume 12147 of *LNCS*, pages 188–209. Springer, Cham, Oct. 2020.
- [65] A. Young and M. Yung. The dark side of “black-box” cryptography, or: Should we trust capstone? In N. Kobitz, editor, *CRYPTO’96*, volume 1109 of *LNCS*, pages 89–103. Springer, Berlin, Heidelberg, Aug. 1996.
- [66] A. Young and M. Yung. Kleptography: Using cryptography against cryptography. In W. Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 62–74. Springer, Berlin, Heidelberg, May 1997.

A Comparison with [32]

In the current section we discuss how our results extend far beyond [32] by discussing the underlying challenges and the ways we address them.

Insufficiency of functionality and security of [32]. The work of [32] considers a one-time primitive and does not model state leakage attacks, therefore, does not account for forward secrecy and post-compromise security, which are fundamental properties for modern secure messaging protocols. The current state of the art in the secure messaging regime requires a stronger notion of key agreement, called continuous key agreement (CKA), providing PCS and FS in the presence of state leakage. However, CKA is not firewall friendly, as it requires the entire transcript to be authenticated.

We address the insufficiency of functionality and security by using state-of-the-art definitions that model state leakage attacks, thus accounting for forward secrecy and post-compromise security. Furthermore, we manage to overcome the limitation of the CKA notion by introducing the notion of continuous key agreement with tamper detection, which is a firewall friendly CKA notion that allows partial mauling of the CKA transcript, while also ensuring tamper detection.

Forward security in one round, deniability, pairing-based computations and immediate decryption. The naive approach in building CKA with tamper detection would be to apply sequential composition on the one-time protocol of [32], deriving a protocol in the continuous setting. However, this would require four rounds of interaction in order for the new key to be computed and for forward security to kick in. Furthermore, the protocol of [32] requires the computation of signatures, which contradicts deniability (which is an important property for secure messaging protocols), and also requires costly, pairing-based computations. Finally, [32] does not achieve immediate decryption, by being highly interactive.

In our work we build a CKA protocol that executes a “Commit”, “Respond”, “Open Commit”, type of protocol, but diverges significantly from [32], as it: (A) executes multiple sessions in an interleaved manner in order to achieve forward security in one round as well as immediate decryption, (B) avoids the use of digital signatures offering deniability, and (C) avoids the use of bilinear pairing computations.

Protection against denial of service attacks. The work of [32] does not provide any recovery against tampering attacks. In particular, if a network adversary tampers with the communication, the protocol of [32] halts and there is no recovery mechanism. This leads to a very low cost denial of service attack for the attacker: insert one malformed message and parties cannot recover forever. This is insufficient for real-world messaging protocols, where parties should recover shortly after the attacker stops tampering with the channel.

In our work we present a protocol that enables parties to recover from denial of service attacks, such that, if tampering has been detected, a recovery subprotocol is activated, enabling parties to reach consistent states. Parties can still exchange messages during the execution of the recovery protocol and the adversary is allowed to tamper with the communication. As long as the adversary remains passive for a number of rounds, which is a standard assumption in secure messaging, parties compute consistent states.

X3DH and re-randomization. The standard X3DH protocol doesn't support firewalls re-randomizing a party's message flow, while maintaining correctness. In this work we design a firewall-friendly X3DH variant, in which the server plays an extra role, re-randomizing keys sent to it, thus allowing firewalls to sanitize communication between parties and maintain protocol functionality. Despite the server's active participation, we prove that the protocol is secure against a malicious server.

B Security Model for Signal-Conforming AKE Protocols

In this section we describe the Signal-Conforming authenticated key-exchange (AKE) security model from [42] that we adopt for our firewall-friendly X3DH protocol in section 6.

B.0.1 Execution Environment.

We consider a system of μ parties $P_1, \dots, P_\mu$. Each party P_i is represented by a set of ℓ oracles $\{\pi_i^1, \dots, \pi_i^\ell\}$, where each oracle corresponds to a single execution of a protocol, and $\ell \in \mathbb{N}$ is the maximum number of protocol sessions per party. Each oracle is equipped with fixed randomness but is otherwise deterministic. Each oracle π_i^s has access to the long-term key pair $(\text{pk}_i, \text{sk}_i)$ of P_i and the public keys of all other parties, and maintains a list of the following local variables:

- rand_i^s is the randomness hard-wired to π_i^s ;
- sid_i^s stores the identity of the session as specified by the protocol;
- π_i^s stores the identity of the intended communication partner;
- $\Psi_i^s \in \{\perp, \text{accept}, \text{reject}\}$ indicates whether oracle π_i^s has successfully completed the protocol execution and accepted the resulting key;
- k_i^s stores the session key computed by π_i^s ;
- state_i^s holds the (secret) session-state values and intermediary results required by the session;
- $\text{role}_i^s \in \{\perp, \text{init}, \text{resp}\}$ indicates π_i^s 's role during the protocol execution.

For each oracle π_i^s , these variables, except the randomness, are initialized to $\perp$. An AKE protocol is executed interactively between two oracles. An oracle that first sends a message is called an initiator ($\text{role} = \text{init}$) and a party that first receives a message is called a responder ($\text{role} = \text{resp}$). The computed session key is assigned to the variable $\mathbf{k}_i^s$ if and only if π_i^s reaches the accept state, that is, $\mathbf{k}_i^s \neq \perp \iff \Psi_i^s = \text{accept}$.

Partnering . To exclude trivial attacks in the security model, we need to define a notion of “partnering” of two oracles. Intuitively, this dictates which oracles can be corrupted without trivializing the security game.

Definition 24. For any $(i, j, s, t) \in [\mu]^2 \times [\ell]^2$ with $i \neq j$, we say that oracles π_i^s and π_j^t are partners if (1) $\text{Pid}_i^s = j$ and $\text{Pid}_j^t = i$; (2) $\text{role}_i^s \neq \text{role}_j^t$; and (3) $\text{sid}_i^s = \text{sid}_j^t$.

For correctness, we require that two oracles executing the AKE without adversarial interaction derive identical session-identifiers. We also require that two such oracles reach the accept state and derive identical session keys except with negligible probability. We call a set $S \subseteq ([\mu] \times [\ell])^2$ to have a valid pairing if the following properties hold:

- For all $((i, s), (j, t)) \in S, i \leq j$.
- For all $(i, s) \in [\mu] \times [\ell]$, there exists a unique $(j, t) \in [\mu] \times [\ell]$ such that $i \neq j$ and either $((i, s), (j, t)) \in S$ or $((j, t), (i, s)) \in S$.

In other words, a set with a valid pairing S partners off each oracle π_i^s and π_j^t in a way that the pairing is unique and no oracle is left out without a pair. We define correctness of an AKE protocol as follows.

Definition 25 (AKE Correctness). An AKE protocol Π_{AKE} is $(1 - \delta)$ -correct if for any set with a valid pairing $S \subseteq ([\mu] \times [\ell])^2$, when we execute the AKE protocol without adversarial interaction between all the oracle pairs included in S , it holds that

$$(1 - \delta) \leq \Pr \left[\begin{array}{l} \pi_i^s \text{ and } \pi_j^t \text{ are partners} \wedge \Psi_i^s = \Psi_j^t = \text{accept} \\ \wedge \mathbf{k}_i^s = \mathbf{k}_j^t \neq \perp \text{ for all } ((i, s), (j, t)) \in S \end{array} \right],$$

where the probability is taken over the randomness used in the oracles.

B.0.2 Security Game.

We define the security of an AKE protocol Π_{AKE} via a game $G_{\Pi_{\text{AKE}}}^{\text{FS}}(\mu, \ell)$ played between an adversary $\mathcal{A}$ and a challenger C . More formally, the security game is parameterized by two integers μ (the number of honest parties) and ℓ (the maximum number of protocol executions per party), and proceeds as follows:

Setup: C first chooses a challenge bit $b \in \{0, 1\}$ at random. C then generates the public parameter of Π_{AKE} and μ long-term key pair $\{(\mathbf{pk}_i, \mathbf{sk}_i) \mid i \in [\mu]\}$, and initializes the collection of oracles $\{\pi_i^s \mid i \in [\mu], s \in [\ell]\}$. C runs $\mathcal{A}$ providing the public parameter and all the long-term public keys $\{\mathbf{pk}_i \mid i \in [\mu]\}$ as input.

Phase 1: $\mathcal{A}$ adaptively issues the following queries any number of times in an arbitrary order:

- **Send**(i, s, m): This query allows $\mathcal{A}$ to send an arbitrary message m to oracle π_i^s . The oracle will respond according to the protocol specification and its current internal state. To start a new oracle, the message m takes a special form:
 $\langle \text{START: role}, j \rangle$; C initializes π_i^s in the role **role**, having party P_j as its peer, that is, C sets $\text{Pid}_i^s := j$ and $\text{role}_i^s := \text{role}$. If π_i^s is an initiator (i.e., $\text{role} = \text{init}$), then C returns the first message of the protocol.
- **RevLTK**(i): For $i \in [\mu]$, this query allows $\mathcal{A}$ to learn the long-term secret key sk_i of party P_i . After this query, P_i is said to be corrupted.
- **RegLTK**(i, pk_i): For $i \in \mathbb{N} \setminus [\mu]$, this query allows $\mathcal{A}$ to register a new party P_i with public key pk_i . We do not require that the adversary knows the corresponding secret key. After the query, the pair (i, pk_i) is distributed to all other oracles. Parties registered by **RegLTK** are corrupted by definition.
- **RevState**(i, s): This query allows $\mathcal{A}$ to learn the session-state state_i^s of oracle π_i^s . After this query, state state_i^s is said to be revealed.
- **RevSessKey**(i, s): This query allows $\mathcal{A}$ to learn the session key k_i^s of oracle π_i^s .

Test: Once $\mathcal{A}$ decides that Phase 1 is over, it issues the following special Test-query which returns a real or random key depending on the challenge bit b .

- **Test**(i, s): If $(i, s) \notin [\mu] \times [\ell]$ or $\Psi_i^s \neq \text{accept}$, C returns $\perp$. Else, C returns k_b , where $k_0 := k_i^s$ and $k_1 \leftarrow \leftarrow^{\$} \mathcal{K}$ (where $\mathcal{K}$ is the session key space). After this query, π_i^s is said to be tested.

Phase 2: $\mathcal{A}$ adaptively issues queries as in Phase 1.

Guess: Finally, $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$. At this point, the tested oracle must be fresh. Here, an oracle π_i^s with $\Psi_i^s = j$ is fresh if all the following conditions hold:

- **RevSessKey**(i, s) has not been issued;
- if π_i^s has a partner π_j^t for some $t \in [\ell]$, then **RevSessKey**(j, t) has not been issued;
- P_i is not corrupted or state state_i^s is not revealed;
- if π_i^s has a partner π_j^t for some $t \in [\ell]$, then P_j is not corrupted or state state_j^t is not revealed;
- if π_i^s has no partner oracle, then P_j is corrupted only after π_i^s finishes the protocol execution.

If the tested oracle is not fresh, C aborts the game and outputs a random bit b' on behalf of $\mathcal{A}$. We say that $\mathcal{A}$ wins the game if $b = b'$. The advantage of $\mathcal{A}$ in the security game above is defined as

$$\text{Adv}_{\Pi_{\text{AKE}}}^{\text{AKE-FS}}(\mathcal{A}) := \left| \Pr [b = b'] - \frac{1}{2} \right|.$$

Definition 26 (AKE Security). *An AKE protocol Π_{AKE} is secure with perfect forward secrecy if $\text{Adv}_{\Pi_{\text{AKE}}}^{\text{AKE-FS}}(\mathcal{A})$ is negligible for any PPT adversary $\mathcal{A}$.*

Definition 27 (AKE Receiver obliviousness). *An AKE protocol is receiver oblivious (or Signal-conforming) if the initiator can compute the first-message without knowledge of the peer id and long-term public key of the communication peer.*

C Proof of Alice’s X3DH Firewall

Proof. We divide the proof in three parts: functionality preservation, exfiltration resistance, and security preservation.

Functionality preservation. First we argue that W_A is functionality preserving. Let Π be the protocol in Fig. 14. Then, we need to show that if we replace A in Π_A for $\Pi_{A \rightarrow W \circ \tilde{A}}$ the protocol still satisfies correctness. We observe that in the first outgoing message flow, the firewall receives g^x from $\tilde{A}$, samples a random value α and re-randomizes the ephemeral key of $\tilde{A}$ to $g^{x\alpha}$ and keeps α as its state. When the firewall receives an incoming message r from the server destined to $\tilde{A}$, the firewall will compute αr and deliver it to $\tilde{A}$. This allows $\tilde{A}$ to set her ephemeral key locally as $g^{xr\alpha}$ and to send the same ephemeral key to B . The remaining message flows the firewall does not alter any messages and thus have no influence over functionality preservation.

Exfiltration resistance. Now we prove that W_A is exfiltration resistant for a functionality-maintaining $\tilde{A}$ against B . More formally, we must show that no PPT adversary can distinguish sanitized transcripts (i.e., with a firewall) generated by running the honest A with a malicious B from a sanitized transcript generated by running a functionality-maintaining $\tilde{A}$ with a malicious B . For that we observe that in the first outgoing message the firewall re-randomizes the ephemeral key of $\tilde{A}$ g^x to be the random group element $g^{x\alpha}$ which in particular is uniformly random and independent from $\tilde{A}$ ’s initial key g^x and has the exact same distribution. In the second outgoing message the firewall verifies that the ephemeral key sent by $\tilde{A}$ matches the ephemeral key that she was supposed to compute from the values αr and that the signature on the ephemeral key is valid under $\tilde{A}$ ’s signing key. Note that the firewall can perform this check efficiently as the firewall knows αr and g^x and thus can compute $g^{xr\alpha}$. It follows that the transcripts are indistinguishable when these checks succeed.

Security preservation. To argue security preservation we must revisit the security proof of the original protocol and show that by replacing A in the protocol of Fig. 14 with a functionality-maintaining version $\tilde{A}$ together with the firewall W_A , the security guarantees of the protocol are preserved. Since many parts of the proof would be identical (or very similar) to the proof of Theorem 9, we will present next, for each strategy of the adversary, only the games that differ from the proof of Theorem 9.

Let $\mathcal{A}$ be a PPT adversary that plays the security game $G_{\Pi_{\text{AKE}}}^{\text{FS}}(\mu, \ell)$ with the challenger $\mathcal{C}$ and has advantage $\text{Adv}_{\Pi_{\text{AKE}}(A \rightarrow W \circ \tilde{A})}^{\text{FS}}(\mathcal{A})$. As before, we must analyze all the relevant strategies that can be taken by the adversary $\mathcal{A}$ as listed in Table 1.

First, we define game G_1 as an intermediate game between G_1 and G_2 that differs from G_1 in the following way.

Game $G_{1'}$. In $G_{1'}$ the game aborts if $\tilde{A}$ does not send a valid signature (of the intended message) in its first message to Bob. The rest of the game is identical to G_1 . We argue that G_1 and $G_{1'}$ have identical views for the adversary $\mathcal{A}$ as long as $\tilde{A}$ is functionality maintaining and Σ produces unique signatures. If $\tilde{A}$ signs a different message than the one she is supposed to sign the firewall aborts and thus functionality is not maintained. If $\tilde{A}$ signs the correct message then the signature must be unique and thus the game aborts with the same probability as in G_1 .

Next we proceed to analyze the different strategies of the adversary $\mathcal{A}$ by only describing the games that differ from the proof of [Theorem 9](#).

Lemma 30 (Adversary using strategy Type-1.). *For any PPT adversary $\mathcal{A}$ using the Type-1 strategy, there exists PPT algorithms $\mathcal{B}$ breaking the CDH assumption and $\mathcal{B}'$ breaking the security of the PRF function such that*

$$\text{Adv}_{\Pi_{\text{AKE}}(\mathcal{A} \rightarrow \mathcal{W} \circ \tilde{\mathcal{A}})}^{\text{FS}}(\mathcal{A}) \leq \mu^2 \ell^2 \cdot \left(qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}') \right).$$

Proof. We describe next game G_4 that differs from the proof of [Lem. 27](#). The remaining games are exactly the same and thus omitted.

Game G_4 . This game is identical to G_3 except that the game samples a random $v \xleftarrow{\$} \{0, 1\}^\lambda$ and in the test session it sets $\text{DH} := v$ instead of computing $\text{DH} := H(\text{DH}_1, \text{DH}_2, \text{DH}_3)$. We show that G_3 and G_4 are indistinguishable assuming the CDH assumption holds. For that, we build an algorithm $\mathcal{B}$ that violates the CDH assumption by using $\mathcal{A}$ as a subroutine.

$\mathcal{B}$ receives a CDH tuple (g, g^γ, g^κ) from its challenger and samples all the public parameters of Π_{AKE} . $\mathcal{B}$ then computes $(\text{lpk}_i, \text{lsk}_i)$ for all $i \in [\mu]$ by running the protocol honestly, and samples $(\hat{i}, \hat{j}, \hat{s}, \hat{t})$ uniformly at random from $[\mu]^2 \times [\ell]^2$. It initializes the list $\mathcal{H}$ as empty, and let the total number of queries the adversary can make to H be denoted by qs . It then invokes $\mathcal{A}$ on the public parameters of Π_{AKE} and $\{\text{lpk}_i | i \in [\mu]\}$ and answers queries made by $\mathcal{A}$ as follows:

- $H(v)$: To simulate the random oracle H , the algorithm $\mathcal{B}$ maintains a list $\mathcal{H}$ of pairs (v, u) , where each pair represents a queried value $v := (\text{DH}_1, \text{DH}_2, \text{DH}_3)$ and its corresponding response u from the random oracle H . Whenever the adversary queries H with an input v , if v is already present in the list $\mathcal{H}$, $\mathcal{B}$ retrieves and returns the previously assigned response u . Otherwise, $\mathcal{B}$ samples a random $u \xleftarrow{\$} \{0, 1\}^\lambda$, adds the pair (v, u) to the list $\mathcal{H}$, and returns u to the adversary.
- $\text{Send}(i, s, \langle \text{START: role}, j \rangle)$: Depending on the values (i, s, j) proceed as follows.
 - If $(i, s, j) = (\hat{i}, \hat{s}, \hat{j})$ then $\mathcal{B}$ samples random $\alpha \in \mathbb{Z}_p^*$ to simulate the value sampled by the firewall, and then returns $g^{\gamma\alpha}$ to $\mathcal{A}$. It receives r back from the adversary $\mathcal{A}$ (as it models r being chosen maliciously). $\mathcal{B}$ implicitly sets $\text{state}_i^s = \gamma \cdot \alpha \cdot r$ (note that $\mathcal{B}$ does not know γ). Otherwise, $\mathcal{B}$ responds as in G_3 .
 - If $(i, s, j) = (\hat{j}, \hat{t}, \hat{i})$ then $\mathcal{B}$ returns g^κ to $\mathcal{A}$ and receives r' back from the adversary $\mathcal{A}$ (as it models r' being chosen maliciously). $\mathcal{B}$ implicitly sets $\text{state}_i^s = \kappa \cdot r'$ (note that $\mathcal{B}$ does not know b). Otherwise, $\mathcal{B}$ responds as in G_3 .
- $\text{Send}(i, s, m = (g^c, \sigma_j))$: Let $\text{Pid}_i^s := j$. Depending on the values (i, s, j) proceed as follows:

- If $(i, s) = (\hat{i}, \hat{s})$ and $j \neq \hat{j}$ then oracles $\pi_{\hat{i}}^{\hat{s}}$ and $\pi_{\hat{j}}^{\hat{t}}$ are not partner oracles and the game aborts (event E_{testO} happens).
- If $(i, s) = (\hat{j}, \hat{t})$ and $j \neq \hat{i}$ then oracles $\pi_{\hat{i}}^{\hat{s}}$ and $\pi_{\hat{j}}^{\hat{t}}$ are not partner oracles and the game aborts (event E_{testO} happens).
- If $(i, s, j) = (\hat{i}, \hat{s}, \hat{j})$ then $\mathcal{B}$ checks if $g^c = g^{\beta\kappa r'}$. If not then $\mathcal{B}$ aborts (event E_{testO} happens), otherwise it proceeds as in game G_3 except that it fixes $\text{DH} := v$.
- If $(i, s, j) = (\hat{j}, \hat{t}, \hat{i})$ then $\mathcal{B}$ checks if $g^c = g^{\alpha\gamma r}$. If not then $\mathcal{B}$ aborts (event E_{testO} happens), otherwise it proceeds as in game G_3 except that it fixes $\text{DH} := v$.
- In any other case, $\mathcal{B}$ responds as in game G_3 .
- $\text{RevLTK}(i), \text{RegLTK}(i), \text{RevState}(i, s), \text{RevSessKey}(i, s)$: $\mathcal{B}$ proceeds as in the previous game. We note that since $\mathcal{A}$ is following the Type-1 strategy $\mathcal{B}$ can answer all the RevState queries *except* $\text{RevState}(\hat{i}, \hat{s})$ or $\text{RevState}(\hat{j}, \hat{t})$.
- $\text{Test}(i, s)$: $\mathcal{B}$ responds as in G_3 . Note that in case $(i, s) \notin \{(\hat{i}, \hat{s}), (\hat{j}, \hat{t})\}$ the game aborts.

When $\mathcal{A}$ outputs a guess b' then $\mathcal{B}$ also outputs b' . By the random oracle model, we have that $\mathcal{A}$ can win the game G_4 if and only if it queries the random oracle with the value $v = (\text{DH}_1, \text{DH}_2, g^{(\gamma\kappa)\alpha\beta rr'})$. Algorithm $\mathcal{B}$ now picks a uniformly random pair $(v, u) \in \mathcal{H}$ and provides $(\text{DH}_3)^{1/rr'}$ to its challenger, thereby violating the CDH assumption. Consequently, we obtain

$$|\Pr[G_3] - \Pr[G_4]| \leq qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}).$$

Combining all the arguments (including the omitted games), we have

$$\text{Adv}_{\Pi_{\text{AKE}}(\mathcal{A} \rightarrow \mathcal{W} \circ \tilde{\mathcal{A}})}^{\text{FS}}(\mathcal{A}) \leq \mu^2 \ell^2 \cdot (qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

□

Lemma 31 (Adversary using strategy Type-2 or Type-3). *For any PPT adversary $\mathcal{A}$ using the Type-2 or Type-3 strategies, there exists PPT algorithms $\mathcal{B}$ breaking the GDH assumption, $\mathcal{B}'$ breaking the security of the PRF function, and $\mathcal{B}''$ breaking the inverse CDH assumption such that*

$$\text{Adv}_{\Pi_{\text{AKE}}(\mathcal{A} \rightarrow \mathcal{W} \circ \tilde{\mathcal{A}})}^{\text{FS}}(\mathcal{A}) \leq \mu^2 \ell \cdot (\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

Proof. We describe game G_4 that differs from the proof of [Lem. 28](#). The remaining games are exactly the same and thus omitted.

Game G_4 . This game is identical to G_3 except that the game samples a random value $\text{DH} \xleftarrow{\$} \{0, 1\}^\lambda$ to be used in the test session instead of computing $\text{DH} \leftarrow H(\text{DH}_1, \text{DH}_2, \text{DH}_3)$. We show that G_3 and G_4 are indistinguishable assuming the GDH assumption holds. For this, we build an algorithm $\mathcal{B}$ that breaks the CDH assumption (given access to a DDH oracle) with the same probability $\mathcal{A}$ does (by using $\mathcal{A}$ as a subroutine).

$\mathcal{B}$ receives a tuple (g, g^γ, g^κ) from the GDH challenger, samples $(\hat{i}, \hat{j}, \hat{s})$ uniformly at random from $[\mu]^2 \times [\ell]$, samples all the public parameters of Π_{AKE} , and generates the long-term keys as follows. For party $P_{\hat{j}}$ algorithm $\mathcal{B}$ runs $(\text{spk}_{\hat{j}}, \text{ssk}_{\hat{j}}) \leftarrow \Sigma.\text{KeyGen}(1^\lambda)$ and sets the long-term public keys as $\text{lpk} := (g^\kappa, \text{spk}_{\hat{j}})$ and *implicitly* sets the long-term secret keys as $\text{spk} := (\kappa, \text{ssk}_{\hat{j}})$

since $\mathcal{B}$ does not know κ that is the exponent in the GDH challenge. For all other parties $i \in [\mu \setminus \hat{j}]$ $\mathcal{B}$ computes $(\text{lpk}_i, \text{lsk}_i)$ by running the protocol honestly. It initializes the lists $\mathcal{H}$ and $\mathcal{L}$ as empty. It then invokes $\mathcal{A}$ on the public parameters of Π_{AKE} and $\{\text{lpk}_i | i \in [\mu]\}$ and answers queries made by $\mathcal{A}$ as follows:

- $H(v)$: To simulate the random oracle H , the algorithm $\mathcal{B}$ maintains a list $\mathcal{H}$ of pairs (v, u) , where each pair represents a queried value v and its corresponding response u from the random oracle H . Whenever the adversary queries H with an input v , the algorithm $\mathcal{B}$ proceeds as follows:
 - If v is already present in the list $\mathcal{H}$, $\mathcal{B}$ retrieves and returns the previously assigned response u .
 - Otherwise, algorithm $\mathcal{B}$ parses $v := (\text{DH}_1, \text{DH}_2, \text{DH}_3)$ and for all tuples $(g^c, \text{DH}'_2, \text{DH}'_3, \text{DH}) \in \mathcal{L}$ it sends the triple $(g^\beta, g^c, \text{DH}_1)$ to the DDH oracle. If the oracle replies with 1 then $\mathcal{B}$ returns the corresponding value DH in $\mathcal{L}$ as response. Otherwise, if the DDH oracle replies with 0 or if $\mathcal{L}$ is empty, then $\mathcal{B}$ samples a fresh random value $u \leftarrow \{0, 1\}^\lambda$, adds the pair (v, u) to $\mathcal{H}$, and returns u as the response.
- $\text{Send}(i, s, \langle \text{START: role}, j \rangle)$: Depending on the values (i, s, j) proceed as following.
 - If $(i, s, j) = (\hat{i}, \hat{s}, \hat{j})$ then $\mathcal{B}$ samples α to simulate the value sampled by the firewall and then send $g^{\gamma\alpha}$ to $\mathcal{A}$. It then receives r back from the adversary $\mathcal{A}$ (as it models r being chosen maliciously). $\mathcal{B}$ implicitly sets $\text{state}_i^{\hat{s}} = \gamma \cdot \alpha \cdot r$ (note that $\mathcal{B}$ does not know γ). Otherwise, $\mathcal{B}$ responds as in G_4 .
 - In any other case, $\mathcal{B}$ responds as in game G_3 .
- $\text{Send}(i, s, m = (g^c, \sigma_j))$: Depending on the values (i, s, j) proceed as follows:
 - If $i = \hat{j}$, then $\mathcal{B}$ proceeds as follows:
 - * At this point, the values DH_2 and DH_3 are fixed for session s and can be computed by $\mathcal{B}$.
 - * For each pair $(v, u) \in \mathcal{H}$ parse v as $(\text{DH}'_1, \text{DH}'_2, \text{DH}'_3)$.
 - Check if $\text{DH}_2 = \text{DH}'_2$ and $\text{DH}_3 = \text{DH}'_3$.
 - If the check passes, send the triple $(g^\kappa, g^c, \text{DH}'_1)$ to the DDH oracle.
 - If the DDH oracle returns 1, set the session key DH to the corresponding value u from $\mathcal{H}$.
 - Otherwise (i.e., if the DDH oracle returns 0 or if DH'_2 and DH'_3 do not match DH_2 and DH_3), generate a fresh random key $\text{DH} \leftarrow \{0, 1\}^\lambda$.
 - Finally, store the tuple $(g^c, \text{DH}'_2, \text{DH}'_3, \text{DH})$ in the list $\mathcal{L}$.
 - In any other case, $\mathcal{B}$ responds as in game G_3 .
- $\text{RevLTK}(i), \text{RegLTK}(i), \text{RevState}(i, s), \text{RevSessKey}(i, s)$: $\mathcal{B}$ proceeds as in the previous game. We note that since $\mathcal{A}$ is following the Type-2 strategy $\mathcal{B}$ can answer all the RevState queries *except* $\text{RevState}(\hat{i}, \hat{s})$ and it can answer all RevLTK queries except $\text{RevLTK}(\hat{j})$.
- $\text{Test}(i, s)$: $\mathcal{B}$ responds as in G_4 . Note that in case $i \neq \hat{j}$ or $(i, s) \neq (\hat{i}, \hat{s})$ the game aborts (i.e., event E_{testO} is triggered).

When $\mathcal{A}$ outputs a guess b' then $\mathcal{B}$ also outputs b' . We note that $\mathcal{B}$ consistently simulates the random oracle queries for $\mathcal{A}$. If $\mathcal{A}$ wins in game G_4 it must have queried the random oracle

with the value $v = (g^{(\gamma\kappa)r\alpha}, \text{DH}_2, \text{DH}_3)$. At this point, $\mathcal{B}$ searches through the list $\mathcal{H}$ and, for each tuple, sends the query $(g^\gamma, g^\kappa, \text{DH}_1)$ to its DDH oracle; if the DDH oracle returns 1 then AdvB returns DH_1 to its GDH challenger. Consequently, we obtain

$$|\Pr[G_3] - \Pr[G_4]| \leq \text{Adv}_{\text{gdh}}(\mathcal{B}).$$

Thus, combining all the arguments together (from the omitted games), we have

$$\text{Adv}_{\Pi_{\text{AKE}}(\mathbf{A} \rightarrow \mathbf{W} \circ \tilde{\mathbf{A}})}^{\text{FS}}(\mathcal{A}) \leq \mu^2 \ell \cdot (\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}'').$$

□

The proof for the adversary strategies Type-5 and Type-6 (resp. Type-7 and Type-8) is again identical to the proof of [Lem. 29](#) and we omit it here. The argument there is only about the unforgeability of the signature scheme Σ . However, additionally we require that Σ produces unique signatures. The intuition then is that $\mathbf{A}$ is not able to leak information by choosing signatures in a specific way.

Thus, combining all the arguments together, we have that the advantage of any PPT adversary $\mathcal{A}$ in violating the security of the protocol in [Fig. 14](#) for a functionality-maintaining $\mathbf{A}$ combined with the firewall $\mathbf{W}_{\mathbf{A}}$ is bounded by

$$\text{Adv}_{\Pi_{\text{AKE}}}^{\text{FS}}(\mathcal{A}) \leq \max \left\{ \begin{array}{l} \mu^2 \ell^2 \cdot (qs \cdot \text{Adv}_{\text{cdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}''), \\ \mu^2 \ell \cdot (\text{Adv}_{\text{gdh}}(\mathcal{B}) + \text{Adv}_{\text{prf}}^{\text{PRF}}(\mathcal{B}')) + \text{Adv}_{\text{inv-cdh}}(\mathcal{B}''), \\ \mu \cdot \text{Adv}_{\Sigma}^{\Sigma}(\mathcal{B}) \end{array} \right\}$$

Thus, the firewall $\mathbf{W}_{\mathbf{A}}$ preserves the security of the protocol in [Fig. 14](#) against Type-1, Type-2, Type-3, Type-5, Type-6, Type-7, and Type-8 adversaries when $\mathbf{A}$ is functionality maintaining. This concludes the proof.

□